INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
SÃO PAULO
Câmpus São Carlos

TTEC - Tópicos Tec. Emergentes

Introdução a Linguagem RUST p.1

Conteúdo 02

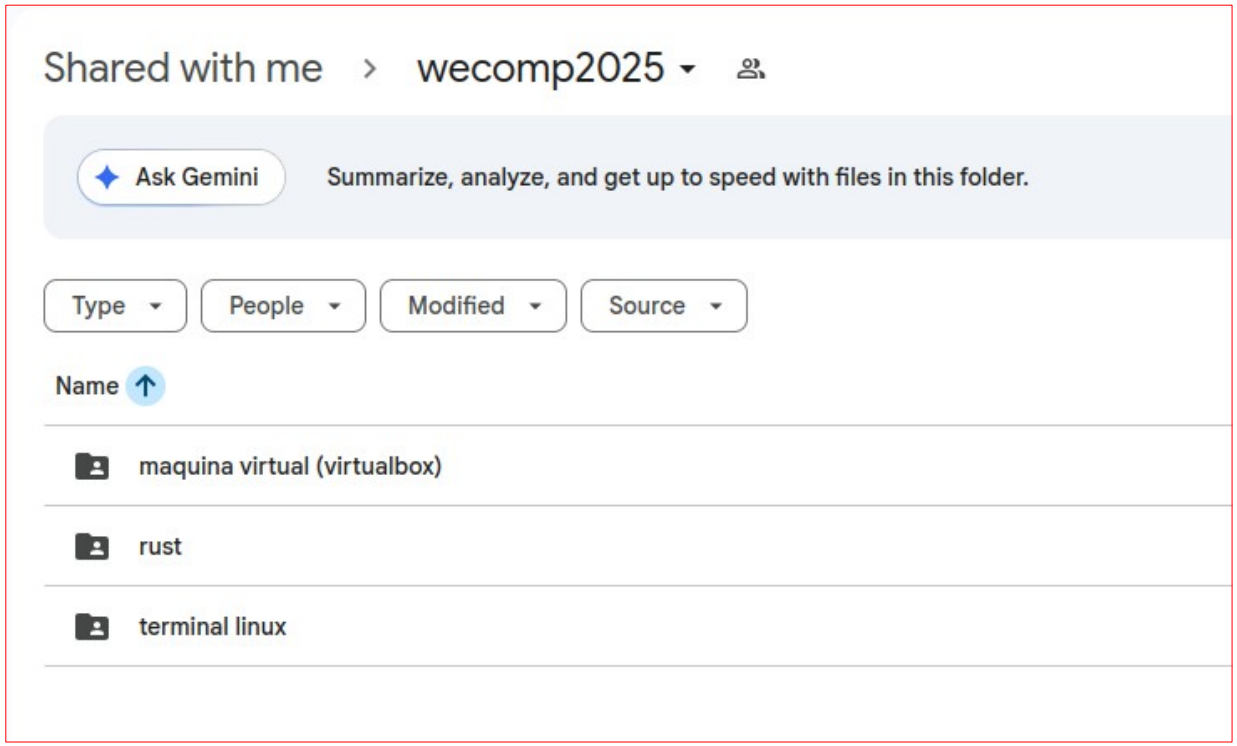
INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
SÃO PAULO
Câmpus São Carlos

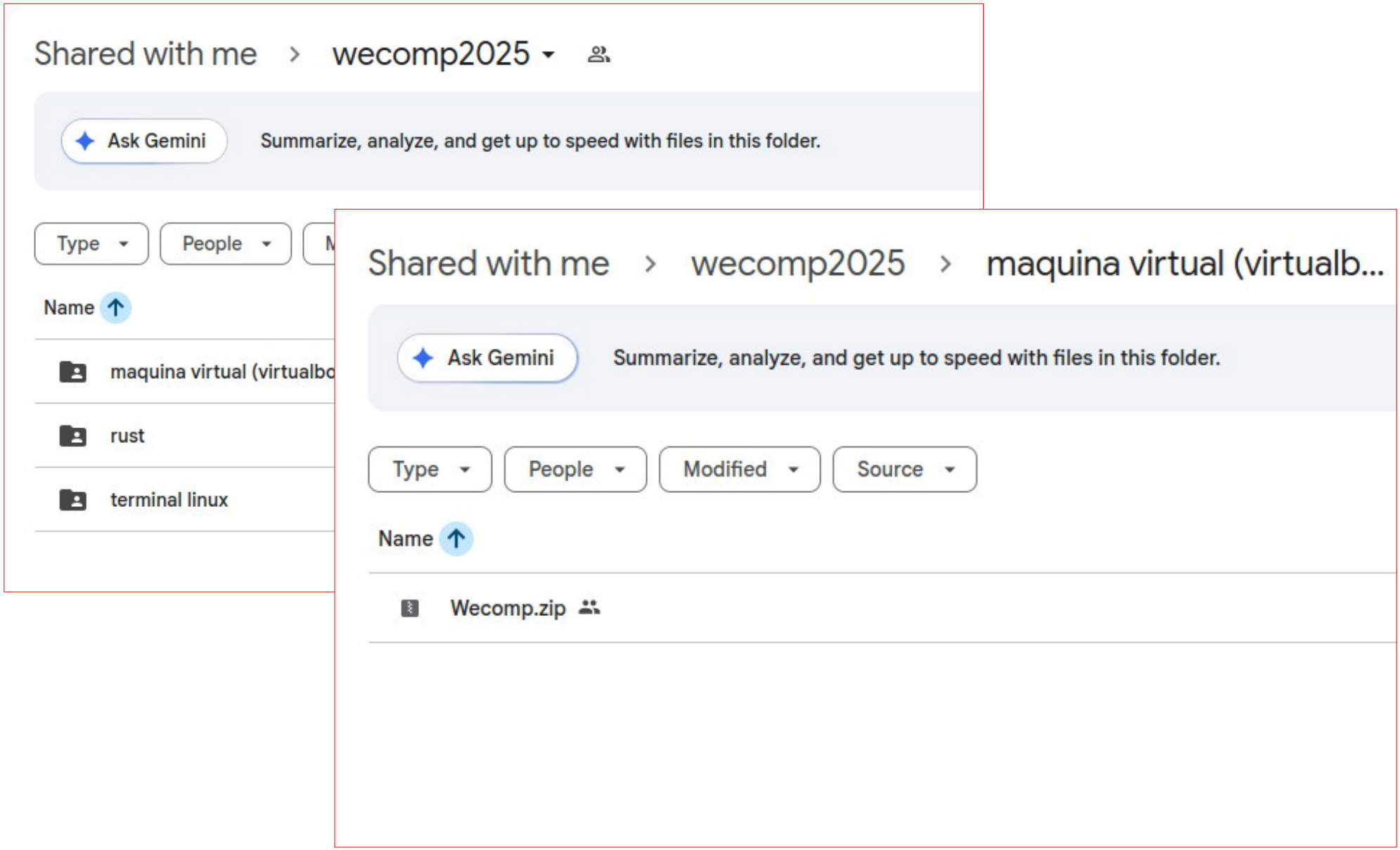
Experimentação: Máquina virtual

- Ubuntu, mais ambiente gráfico Lxde
- “Capada”, não tem quase nada, pra ficar leve
  - Terminal
  - Editor de textos simples (porém gráfico)
  - Navegador web velho...
  - **Compilador RUST !**
- Usuário **`wecomp`**
  - senha: **`wecomp`**
- Como já falei, a 2a parte desta aula é no lab 105.
  Quem estiver interessado, podemos instalar a máquina virtual lá para experimentação da linguagem.

Conteúdo disponível em

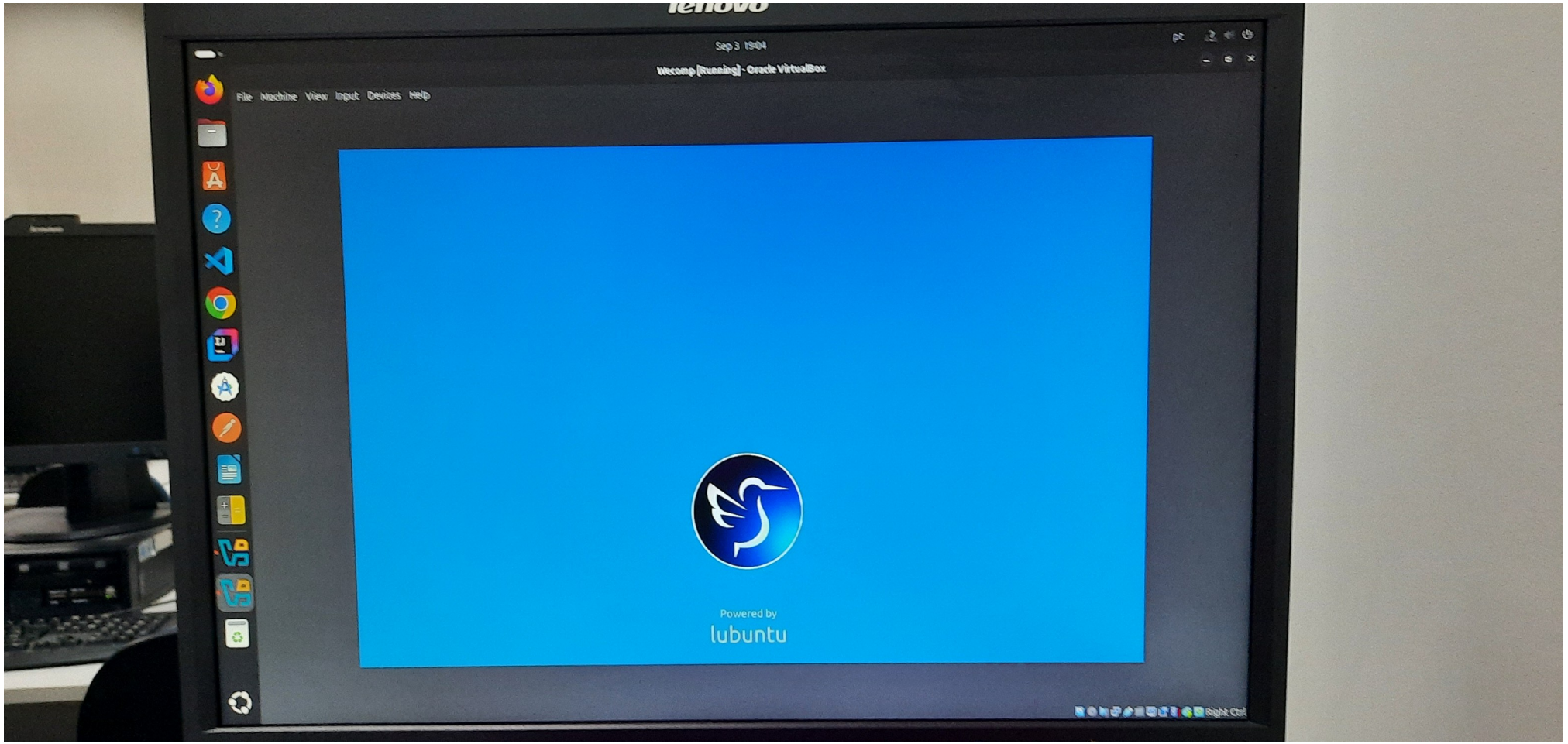
https://drive.google.com/drive/folders/1sP011P7y5tM1GmVZf1nx-p1Uc4Iw4e6t?usp=drive_link

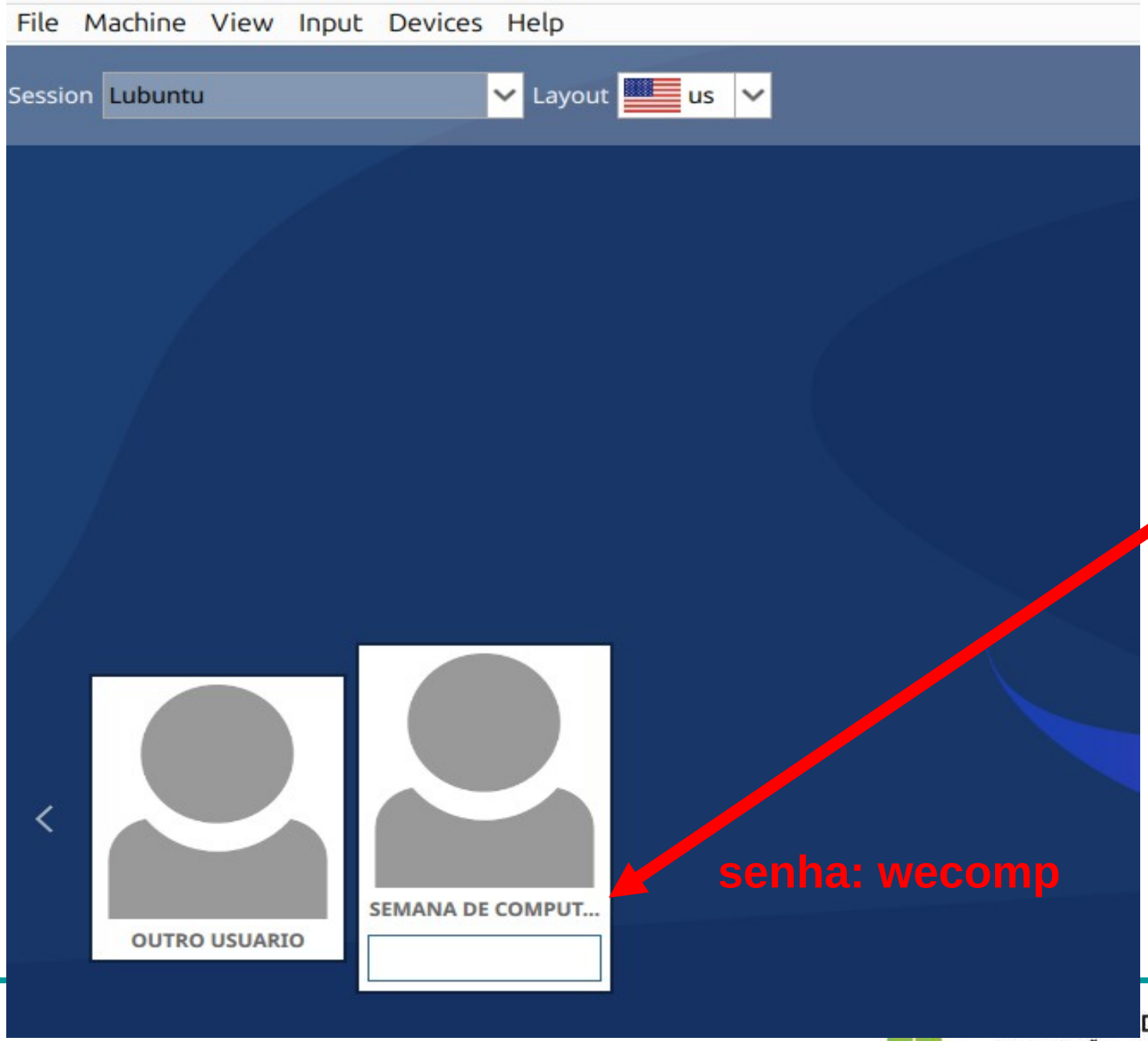

Shared with me
wecomp2025
Ask Gemini
Summarize, analyze, and get up to speed with files in this folder.
Type
People
Modified
Source
Name
maquina virtual (virtualbox)
rust
terminal linux

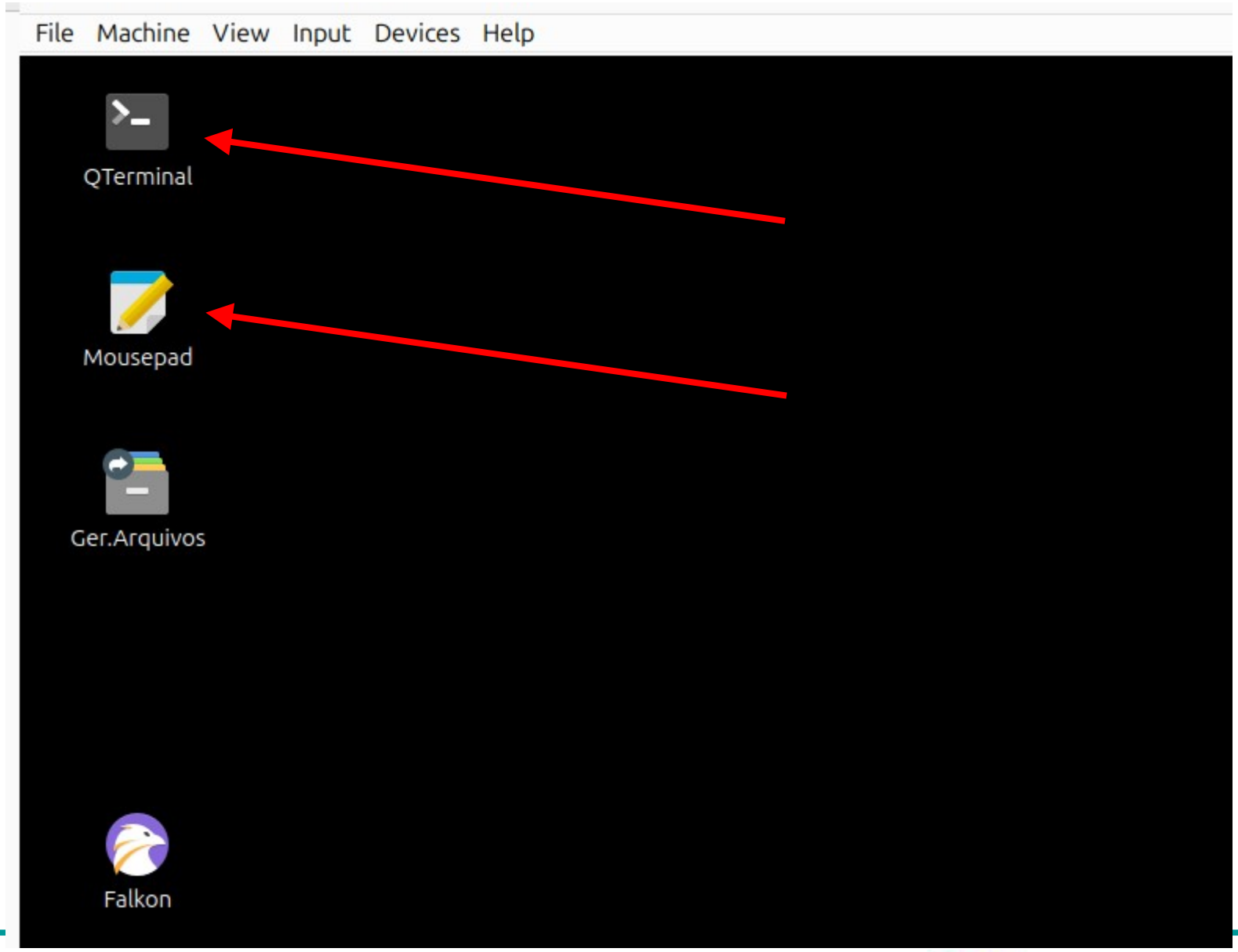

Shared with me
wecomp2025
Ask Gemini
Summarize, analyze, and get up to speed with files in this folder.
Type
People
Name
maquina virtual (virtualbo
rust
terminal linux
Shared with me
wecomp2025
maquina virtual (virtualb...
Ask Gemini
Summarize, analyze, and get up to speed with files in this folder.
Type
People
Modified
Source
Name
Wecomp.zip

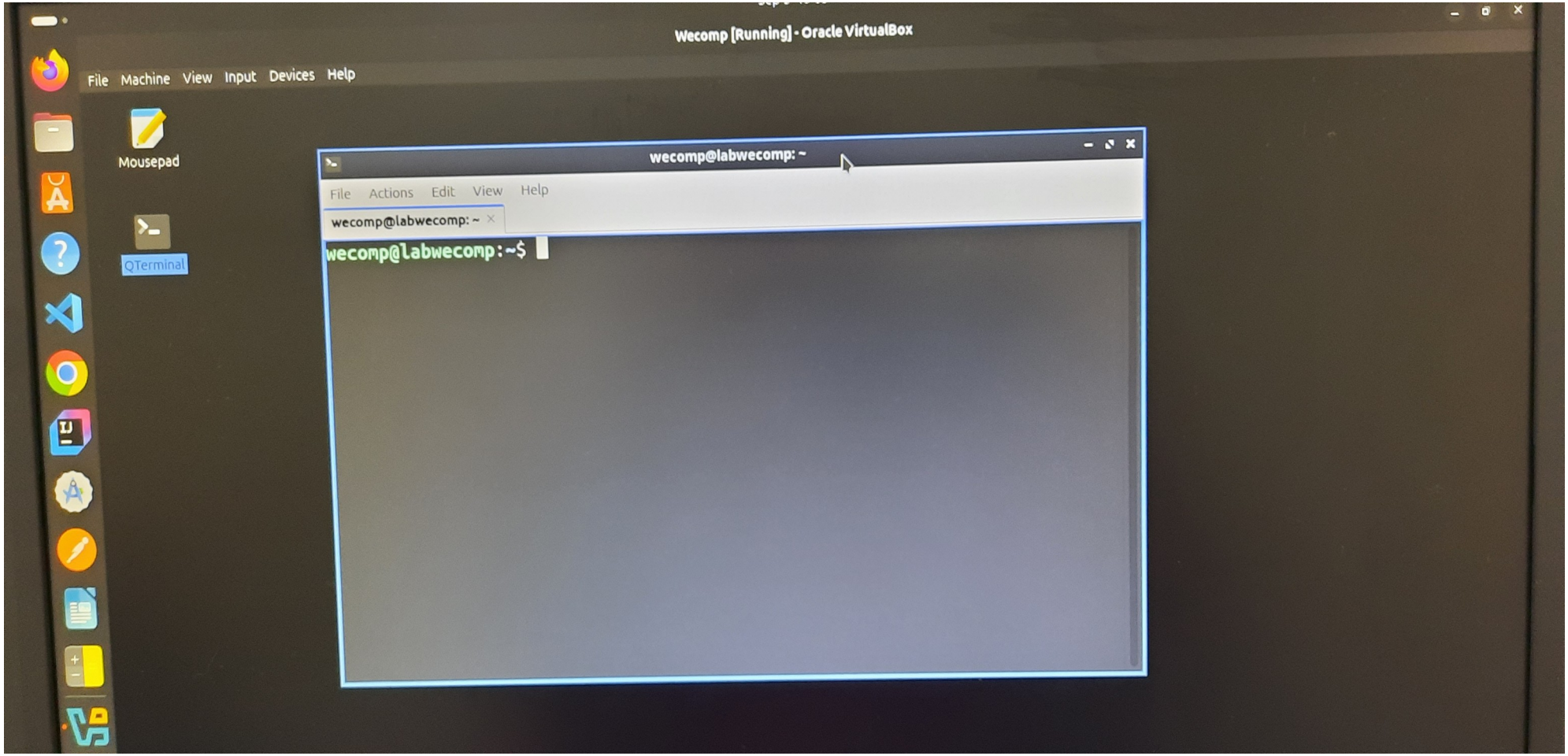
Não vamos usar nenhuma IDE.
A idéia aqui é mostrar uma introdução.
Vamos usar a linha de comando (terminal).

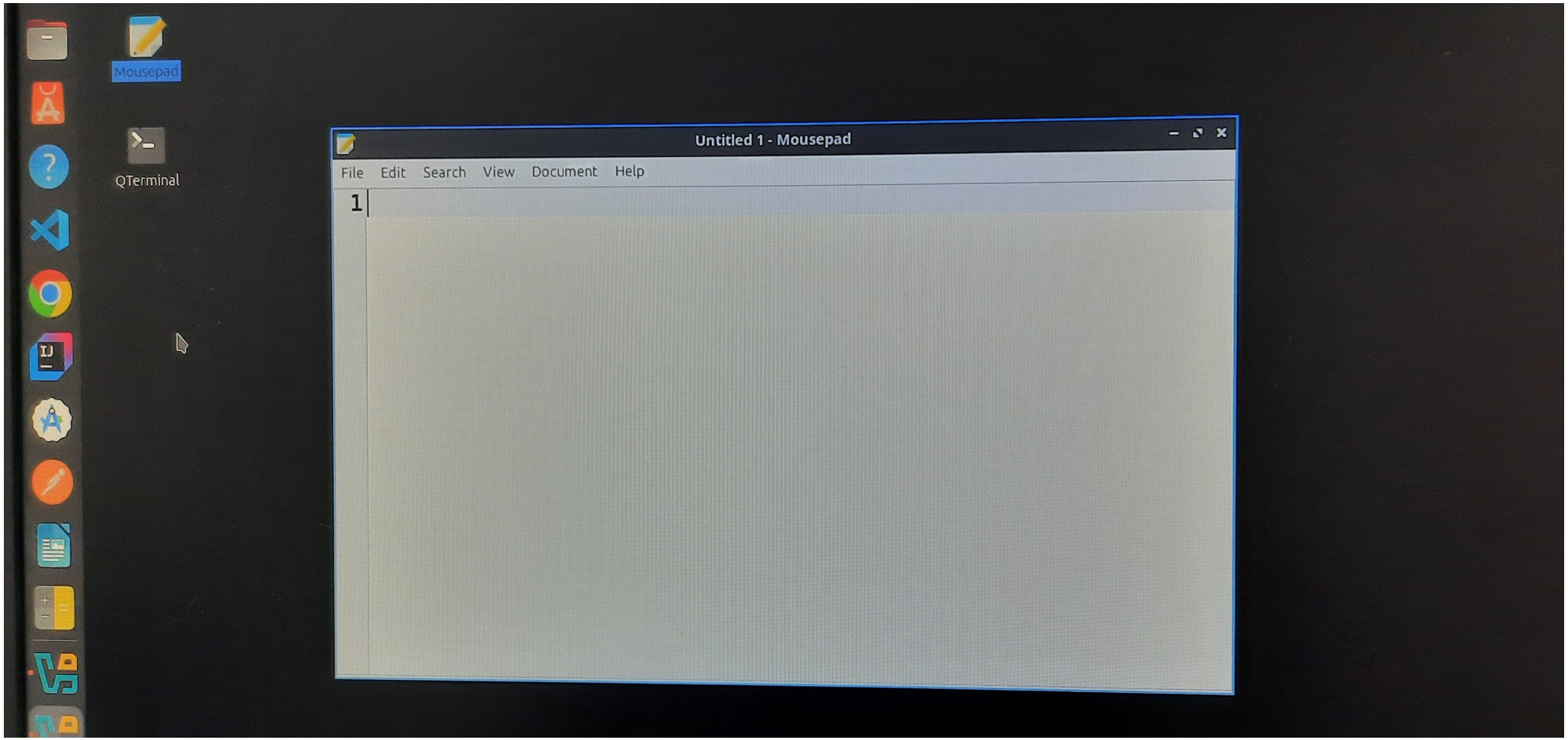
Máquina virtual:


Sep 3 19:04
Wecomp [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Powered by
lubuntu

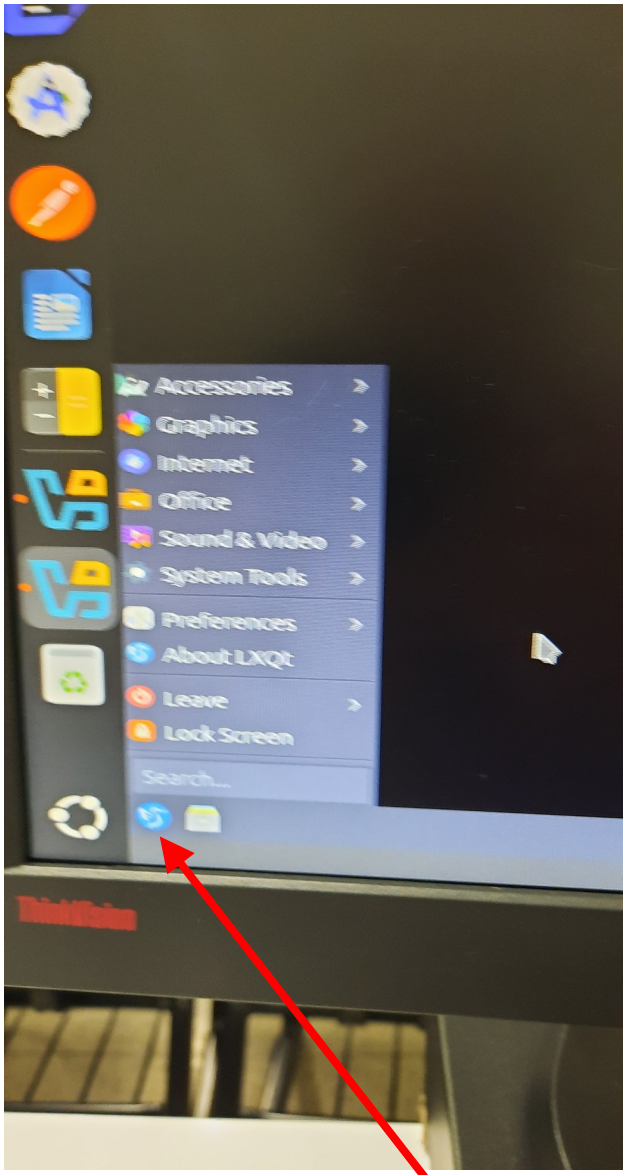

File Machine View Input Devices Help
Session Lubuntu
Layout us
OUTRO USUARIO
SEMANA DE COMPUT...
senha: wecomp

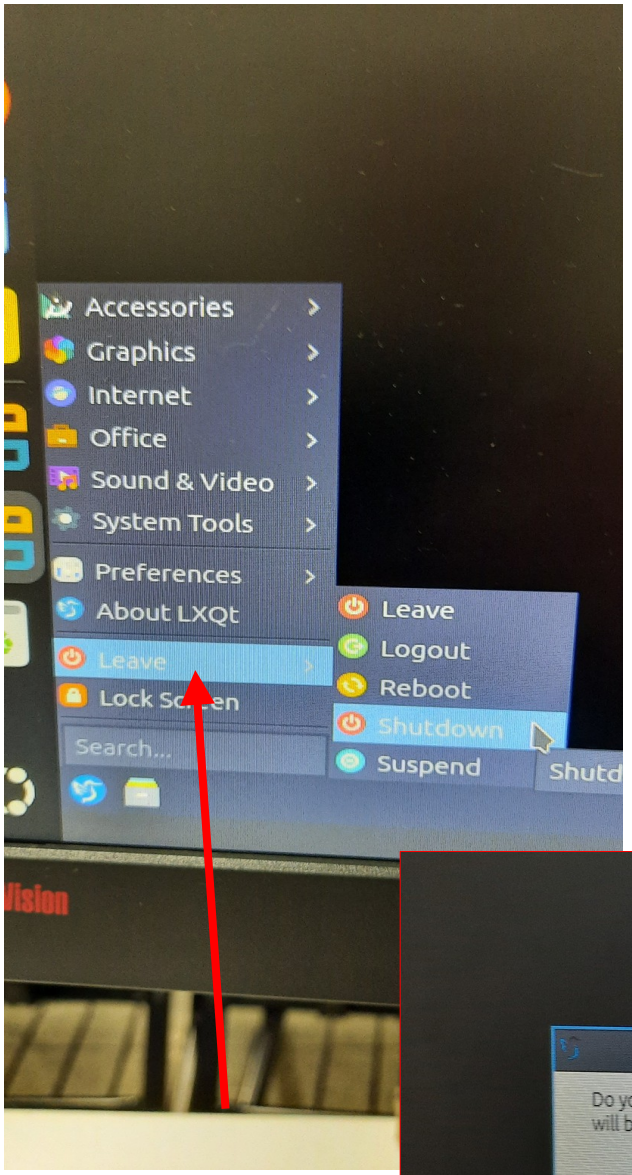

File Machine View Input Devices Help
QTerminal
Mousepad
Ger.Arquivos
Falkon

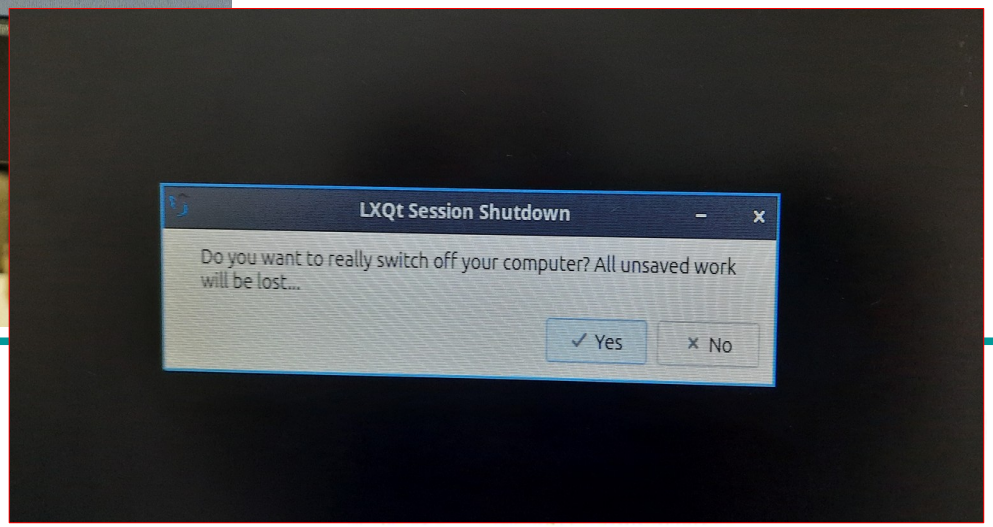

Wecomp [Running] - Oracle VirtualBox
File Machine View Input Devices Help
Mousepad
QTerminal
wecomp@labwecomp: ~
File Actions Edit View Help
wecomp@labwecomp: ~
wecomp@labwecomp:~$


Mousepad
QTerminal
Untitled 1 - Mousepad
File Edit Search View Document Help
1

E para desligar a máquina virtual...

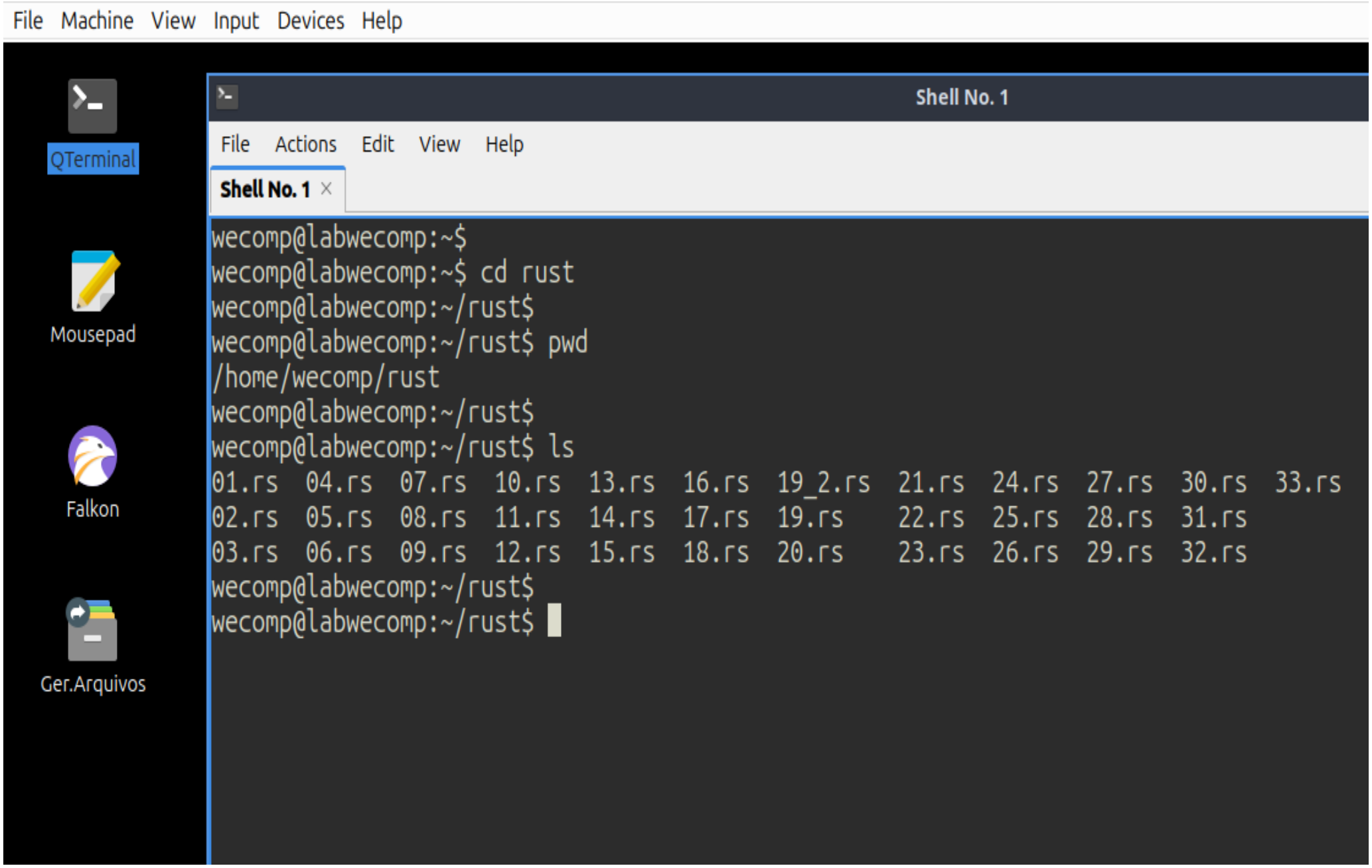






No Linux, a pasta do usuário fica em
`/home/wecomp`

Os códigos-fonte estão numa subpasta,
`/home/wecomp/rust`

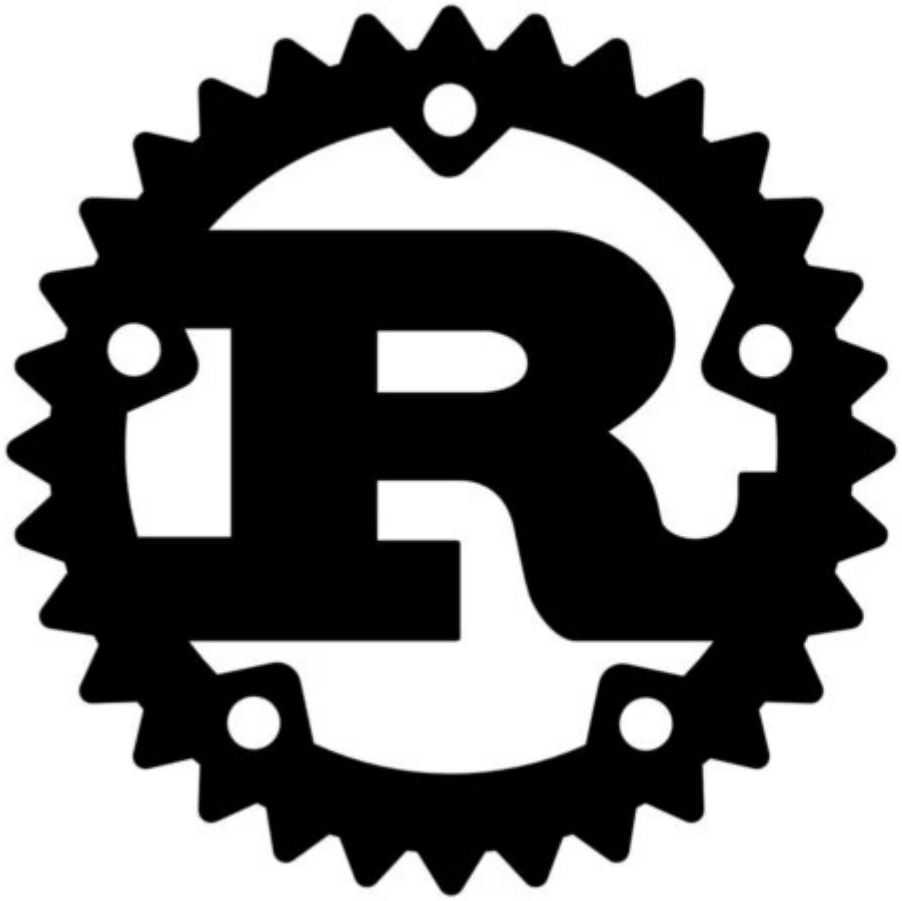

File Machine View Input Devices Help
QTerminal
Mousepad
Falkon
Ger.Arquivos
Shell No. 1
File Actions Edit View Help
Shell No. 1
wecomp@labwecomp:~$
wecomp@labwecomp:~$ cd rust
wecomp@labwecomp:~/rust$
wecomp@labwecomp:~/rust$ pwd
/home/wecomp/rust
wecomp@labwecomp:~/rust$
wecomp@labwecomp:~/rust$ ls
01.rs 04.rs 07.rs 10.rs 13.rs 16.rs 19_2.rs 21.rs 24.rs 27.rs 30.rs 33.rs
02.rs 05.rs 08.rs 11.rs 14.rs 17.rs 19.rs 22.rs 25.rs 28.rs 31.rs
03.rs 06.rs 09.rs 12.rs 15.rs 18.rs 20.rs 23.rs 26.rs 29.rs 32.rs
wecomp@labwecomp:~/rust$
wecomp@labwecomp:~/rust$

Nos slides a seguir,
os comandos foram dados
a partir do diretório home do usuário
(**~**).

Para realizar a recompilação
precisa estar dentro da pasta rust,
como mostrada.

Agora sim... vamos lá!





The Rust Programming Language

INSTITUTO FEDERAL DE
EDUCAÇÃO, CIÊNCIA E TECNOLOGIA
SÃO PAULO
Câmpus São Carlos

Um pouquinho de história...

- Um funcionário da Mozilla, Graydon Hoare, começou a trabalhar em uma linguagem como um projeto pessoal.
- A Mozilla patrocinou o projeto em 2009 e o anunciou em 2010.
- A primeira versão estável foi lançada em 15 de maio de 2015.



Algumas características

- Compilada!
- Foco em “Systems Programming”
  - Software que interage mais diretamente com o hardware e o sistema operacional.
    - Exemplos clássicos incluem:
      - sistemas operacionais,
      - drivers de dispositivos,
      - softwares embarcados, etc....
    - Esses programas geralmente precisam de:
      - alta performance,
      - baixo consumo de memória,
      - controle fino sobre recursos (como alocação e liberação de memória),
      - acesso direto a hardware ou APIs de sistema.

Algumas características

- Desempenho comparável com C e C++, mas com...
  - Memory Safety
    - Dizer que Rust é memory safe (ou que oferece memory safety) significa que ela evita automaticamente, em tempo de compilação, uma série de erros relacionados ao uso incorreto da memória, como:
      - Acesso a memória liberada (uso após free),
      - Acesso fora dos limites de um array (out of bounds),
      - Referências nulas ou inválidas (null pointers), etc...
    - Rust garante isso **sem** coletor de lixo (GC), usando um sistema de posse de memória (ownership), empréstimos (borrowing) e tempo de vida (lifetimes).

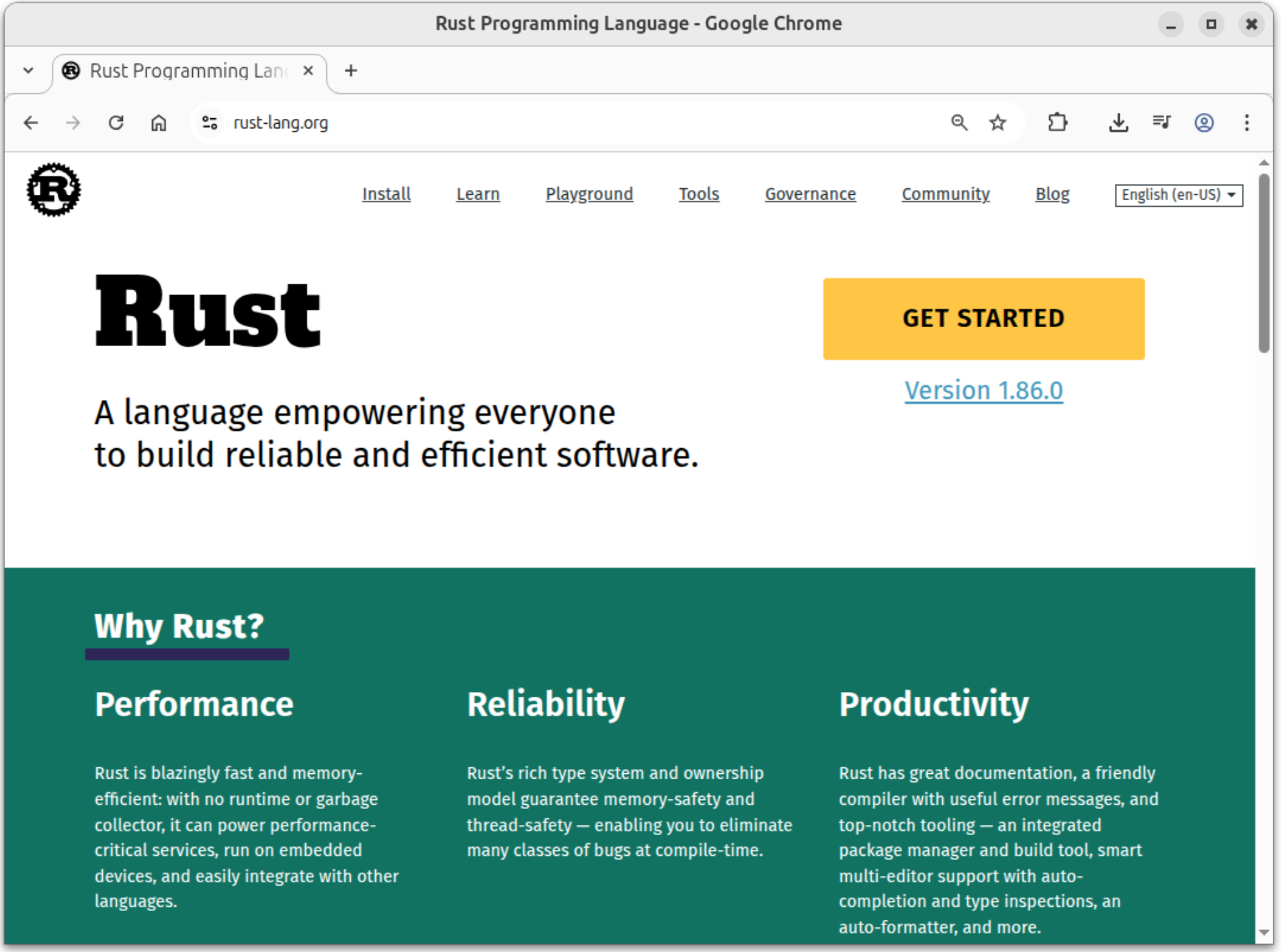
Algumas características

- Fortemente tipada:
  - Rust é uma linguagem strongly typed, ou seja, fortemente tipada. Isso significa que:
    - Cada valor tem um tipo bem definido, e esse tipo não muda automaticamente.
    - Conversões entre tipos (casting) precisam ser explícitas.
    - O compilador verifica os tipos em tempo de compilação, ajudando a detectar erros antes de rodar o programa.
    - Operações inválidas entre tipos diferentes não são permitidas automaticamente.

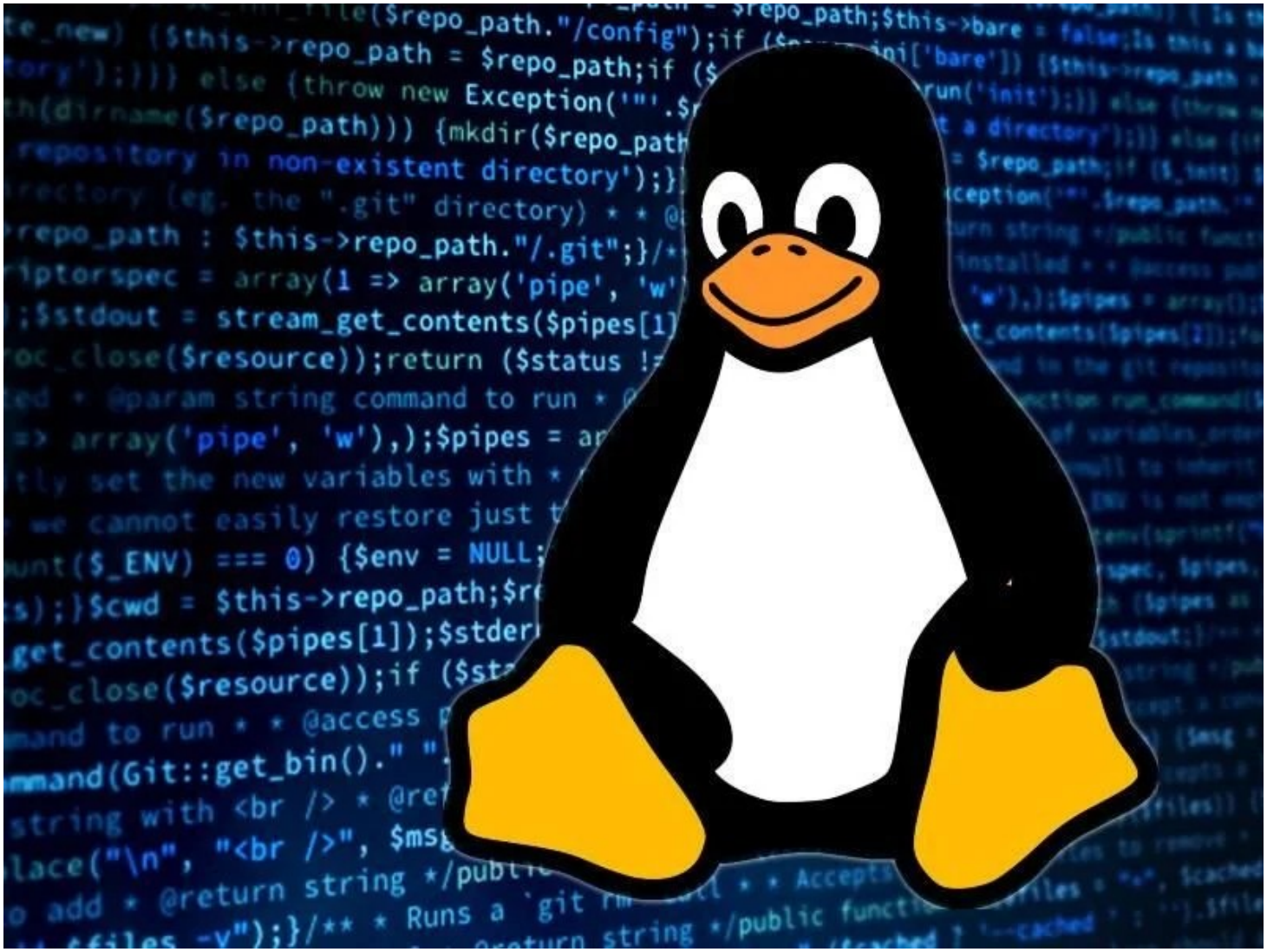
Algumas características

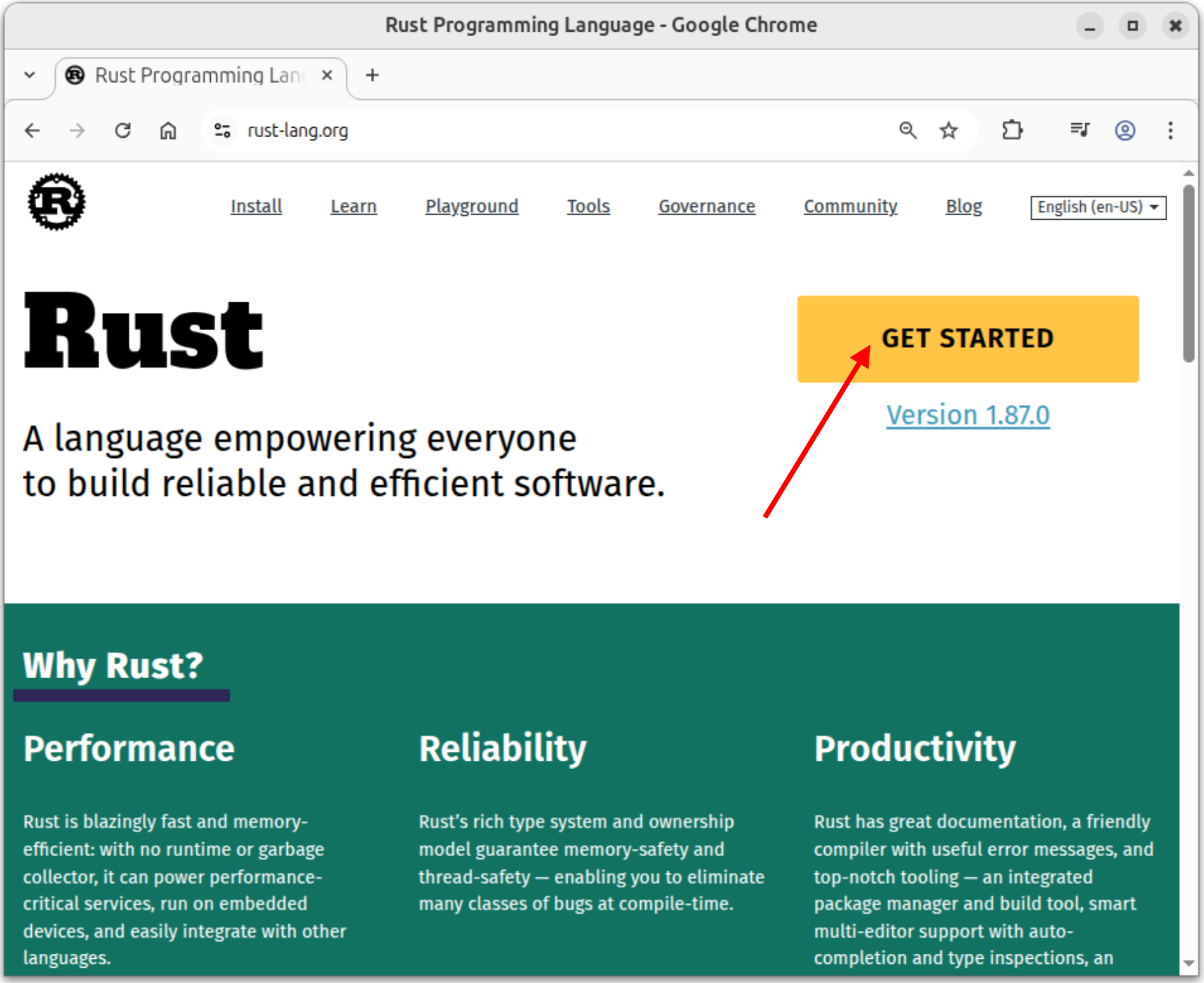
- Cargo Manager
  - Linguagens de sistema como C e C++ nunca tiveram um gerenciador de pacotes padrão.
  - O Rust oferece o gerenciador Cargo,
    que reúne todas as bibliotecas e frameworks da linguagem,
    não apenas para ajudar os desenvolvedores a criar aplicações fantásticas, mas também para distribuir código aos usuários finais.

Site oficial

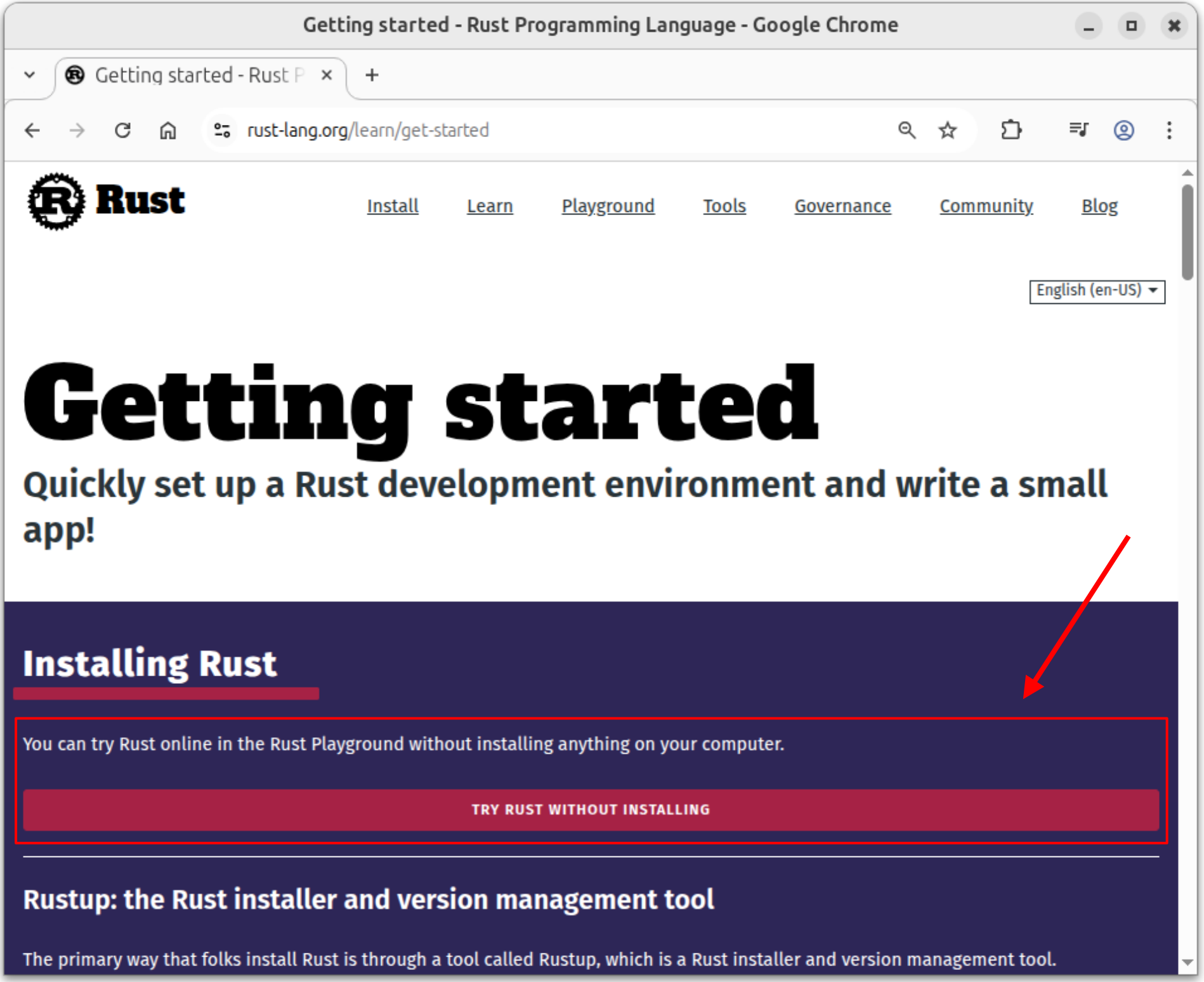


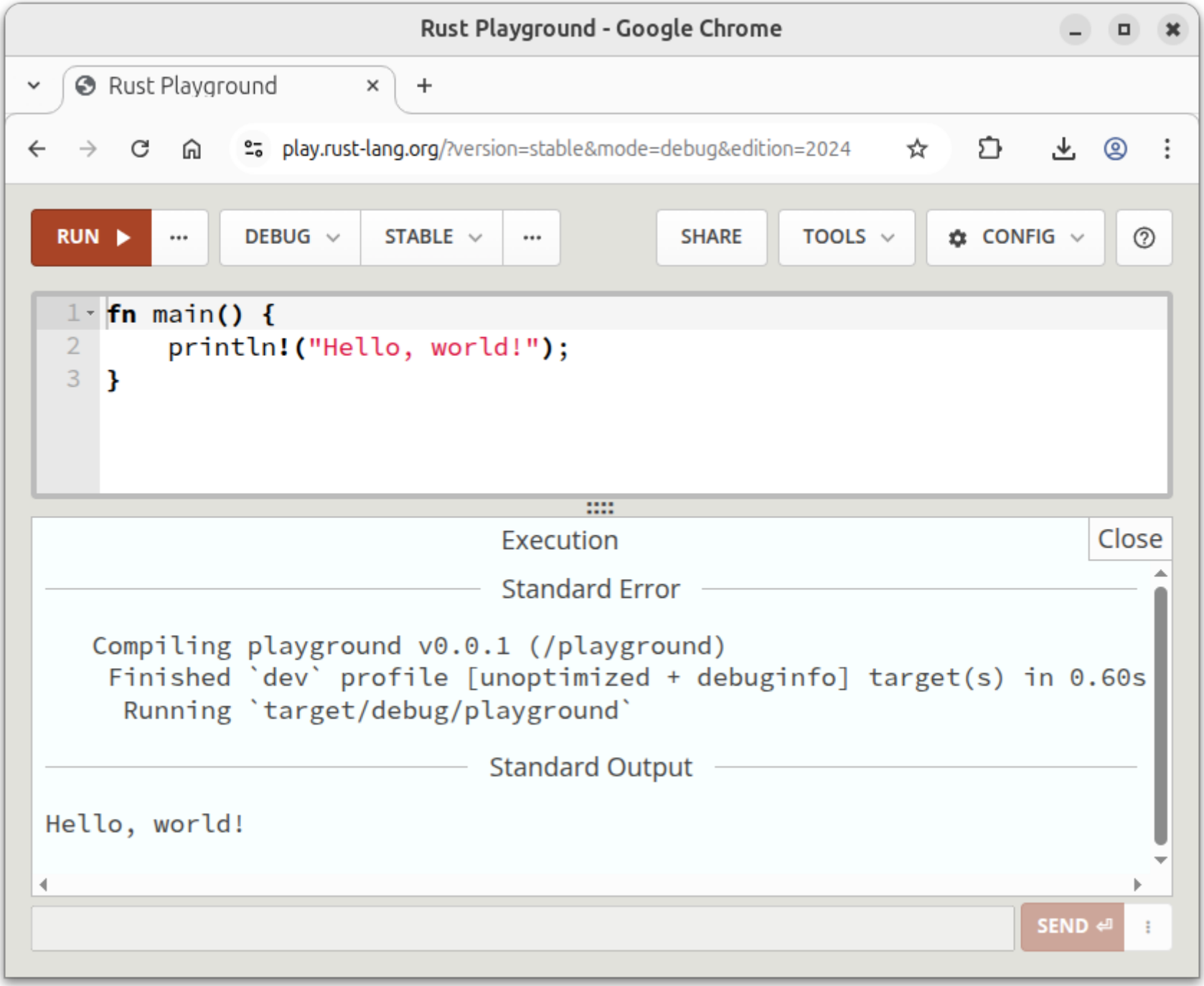
Instalação

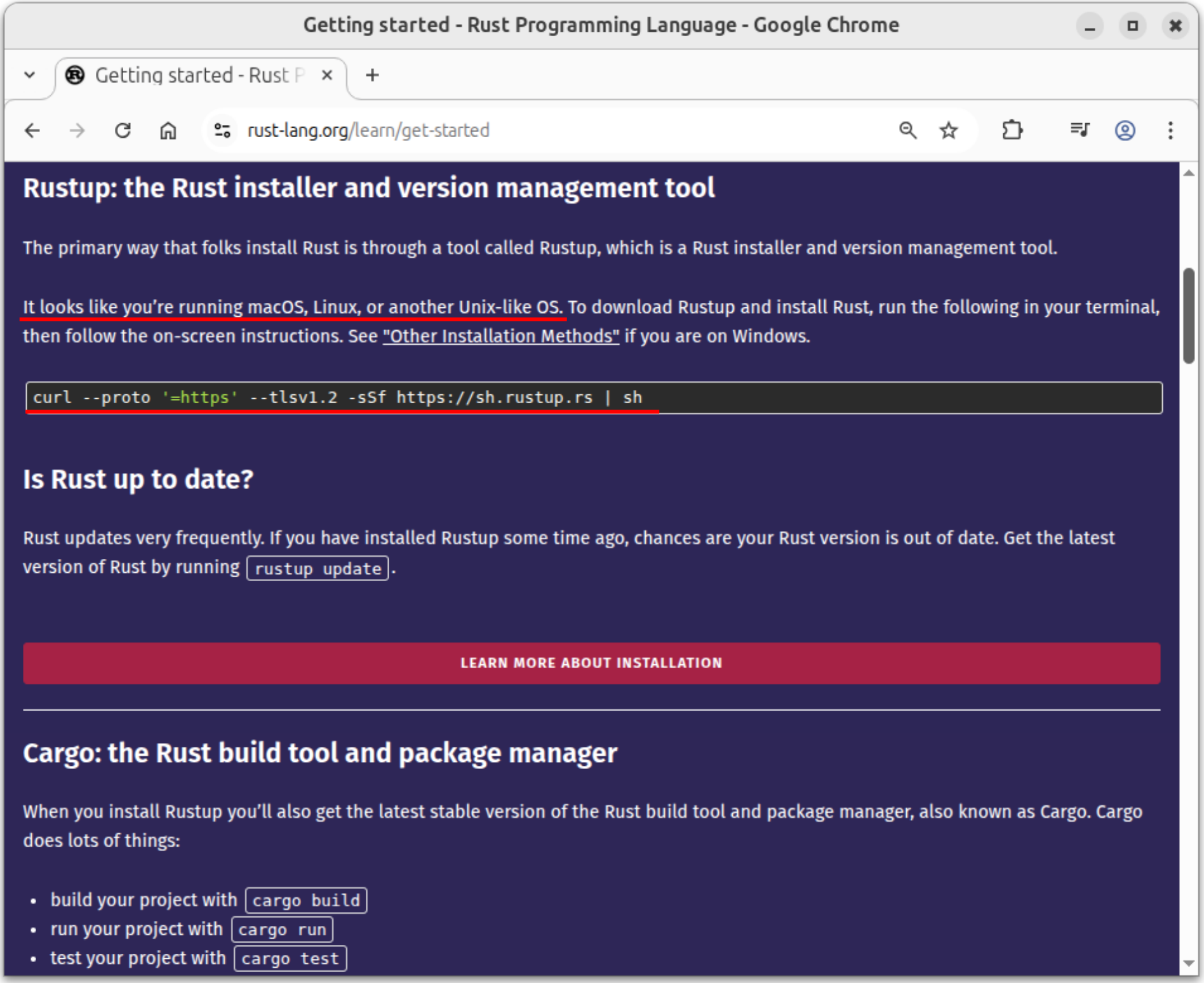


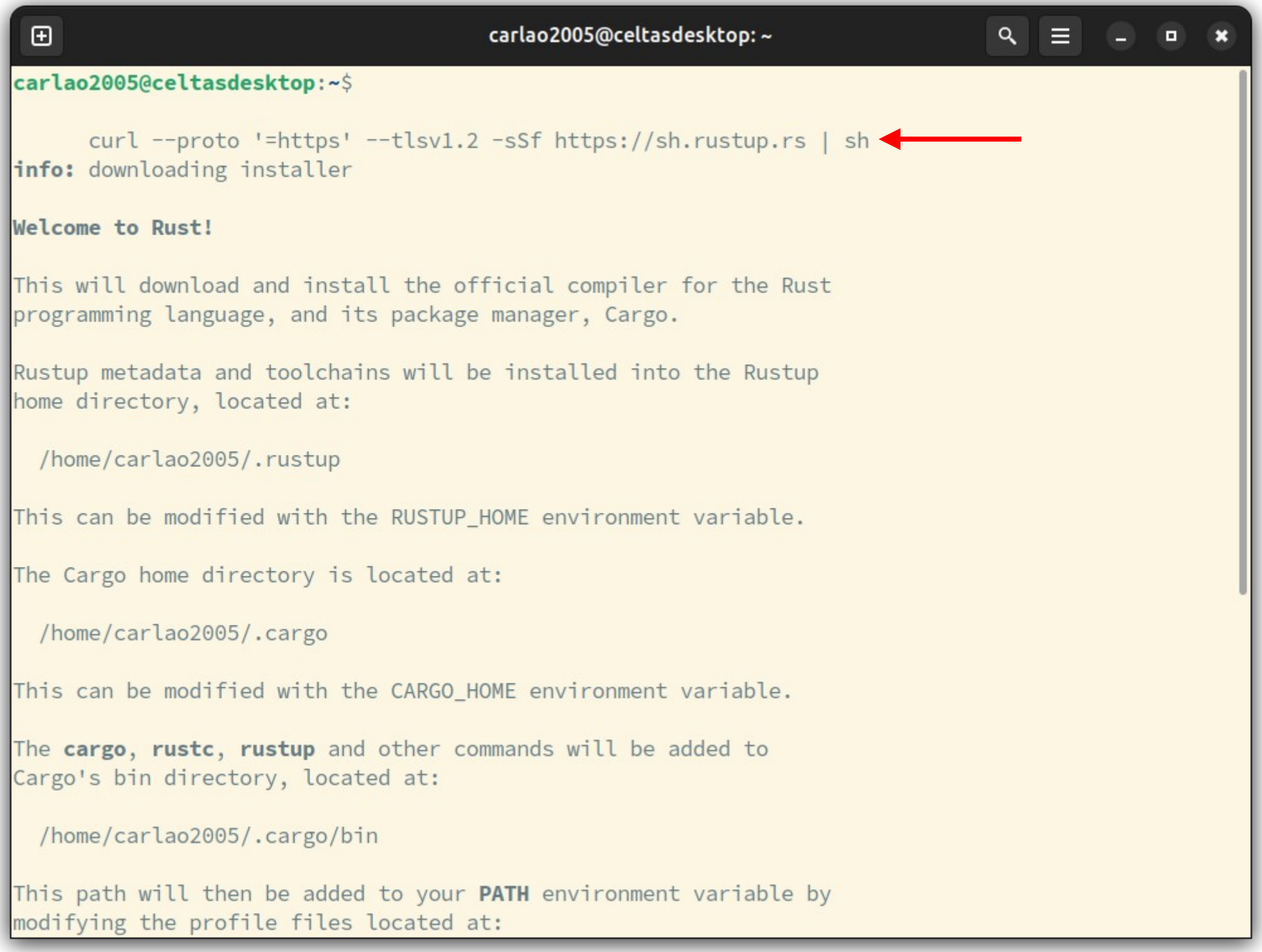

Rust Programming Language - Google Chrome
Rust Programming Lan
rust-lang.org
Install
Learn
Playground
Tools
Governance
Community
Blog
English (en-US)
Rust
A language empowering everyone
to build reliable and efficient software.
GET STARTED
Version 1.87.0
Why Rust?
Performance
Rust is blazingly fast and memory-efficient: with no runtime or garbage collector, it can power performance-critical services, run on embedded devices, and easily integrate with other languages.
Reliability
Rust's rich type system and ownership model guarantee memory-safety and thread-safety — enabling you to eliminate many classes of bugs at compile-time.
Productivity
Rust has great documentation, a friendly compiler with useful error messages, and top-notch tooling — an integrated package manager and build tool, smart multi-editor support with auto-completion and type inspections, an

Antes...


Getting started - Rust Programming Language - Google Chrome
Getting started - Rust P
rust-lang.org/learn/get-started
Rust
Install
Learn
Playground
Tools
Governance
Community
Blog
English (en-US)
Getting started
Quickly set up a Rust development environment and write a small app!
Installing Rust
You can try Rust online in the Rust Playground without installing anything on your computer.
TRY RUST WITHOUT INSTALLING
Rustup: the Rust installer and version management tool
The primary way that folks install Rust is through a tool called Rustup, which is a Rust installer and version management tool.


Rust Playground - Google Chrome
Rust Playground
play.rust-lang.org/?version=stable&mode=debug&edition=2024
RUN
DEBUG
STABLE
SHARE
TOOLS
CONFIG
1 fn main() {
2 println!("Hello, world!");
3 }
Execution
Close
Standard Error
Compiling playground v0.0.1 (/playground)
Finished `dev` profile [unoptimized + debuginfo] target(s) in 0.60s
Running `target/debug/playground`
Standard Output
Hello, world!
SEND


Getting started - Rust Programming Language - Google Chrome
Getting started - Rust P
rust-lang.org/learn/get-started
Rustup: the Rust installer and version management tool
The primary way that folks install Rust is through a tool called Rustup, which is a Rust installer and version management tool.
It looks like you're running macOS, Linux, or another Unix-like OS. To download Rustup and install Rust, run the following in your terminal, then follow the on-screen instructions. See "Other Installation Methods" if you are on Windows.
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
Is Rust up to date?
Rust updates very frequently. If you have installed Rustup some time ago, chances are your Rust version is out of date. Get the latest version of Rust by running rustup update.
LEARN MORE ABOUT INSTALLATION
Cargo: the Rust build tool and package manager
When you install Rustup you'll also get the latest stable version of the Rust build tool and package manager, also known as Cargo. Cargo does lots of things:
build your project with cargo build
run your project with cargo run
test your project with cargo test


carlao2005@celtasdesktop: ~
carlao2005@celtasdesktop:~$
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
info: downloading installer
Welcome to Rust!
This will download and install the official compiler for the Rust
programming language, and its package manager, Cargo.
Rustup metadata and toolchains will be installed into the Rustup
home directory, located at:
/home/carlao2005/.rustup
This can be modified with the RUSTUP_HOME environment variable.
The Cargo home directory is located at:
/home/carlao2005/.cargo
This can be modified with the CARGO_HOME environment variable.
The cargo, rustc, rustup and other commands will be added to
Cargo's bin directory, located at:
/home/carlao2005/.cargo/bin
This path will then be added to your PATH environment variable by
modifying the profile files located at:

```
  /home/carlao2005/.cargo

This can be modified with the CARGO_HOME environment variable.

The cargo, rustc, rustup and other commands will be added to
Cargo's bin directory, located at:

  /home/carlao2005/.cargo/bin

This path will then be added to your PATH environment variable by
modifying the profile files located at:

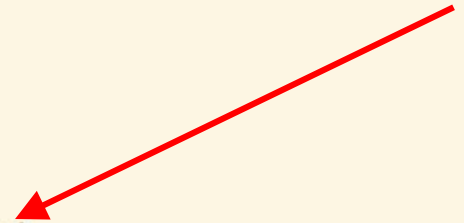
  /home/carlao2005/.profile
  /home/carlao2005/.bashrc

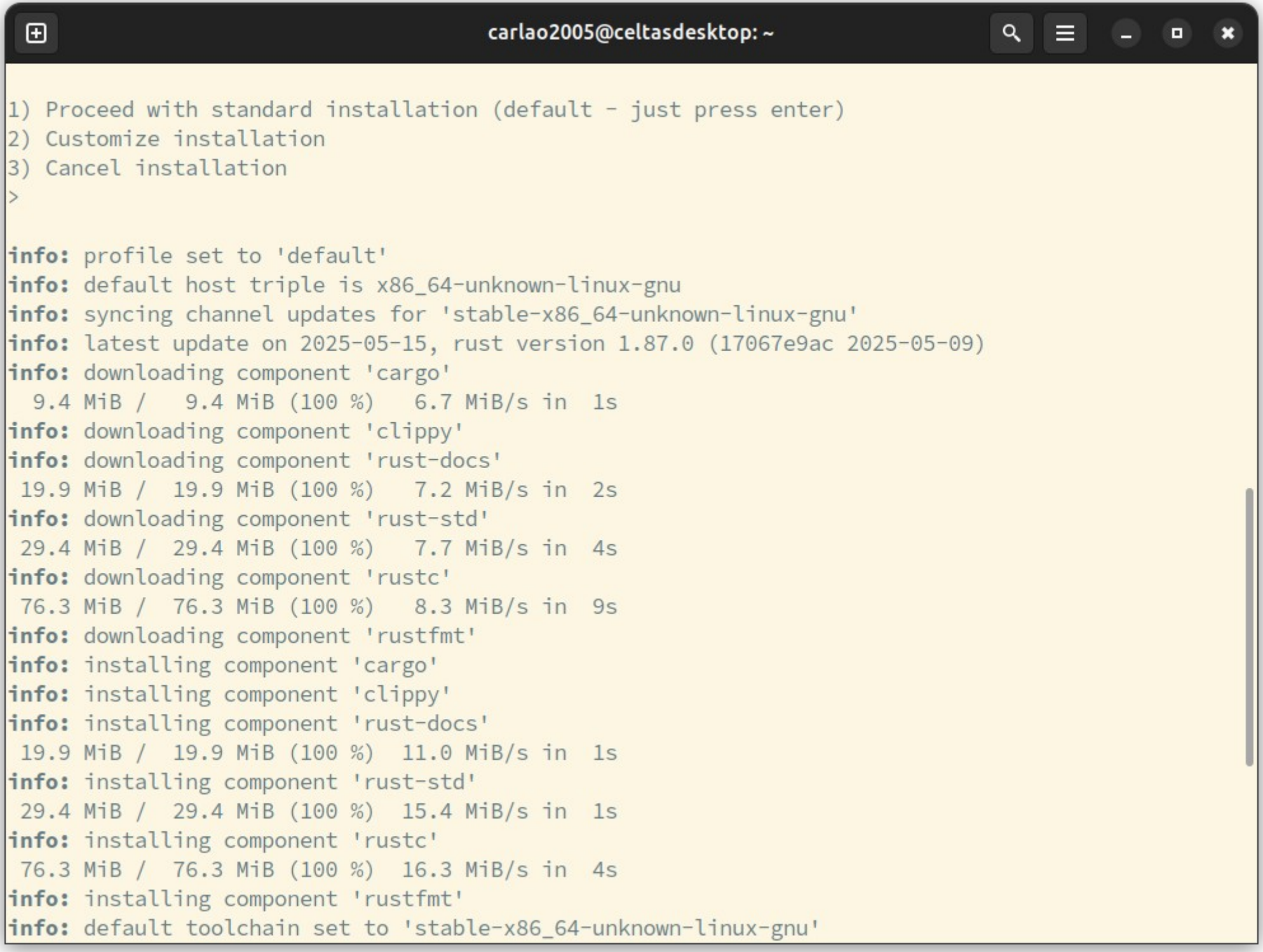
You can uninstall at any time with rustup self uninstall and
these changes will be reverted.

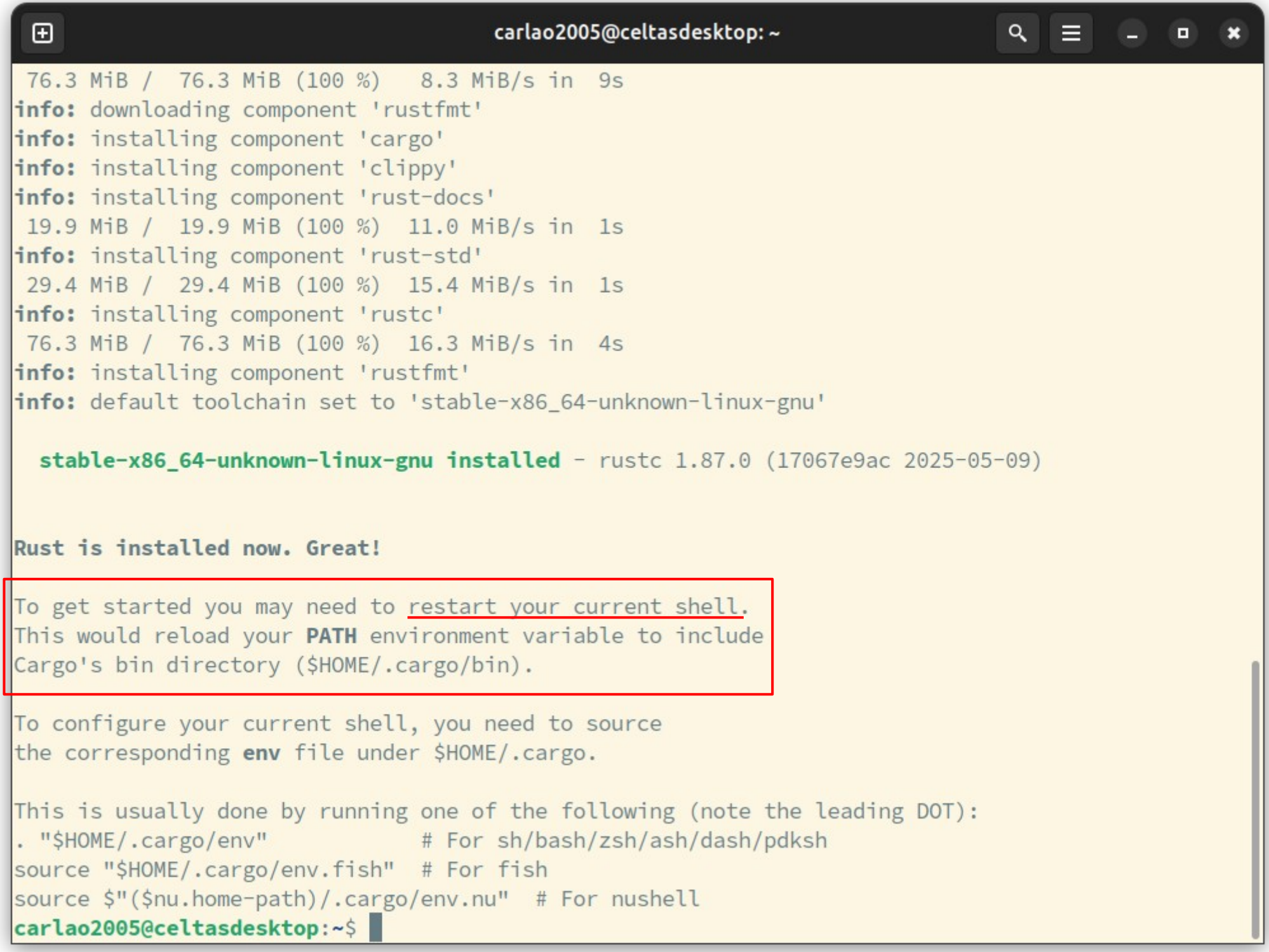
Current installation options:


   default host triple: x86_64-unknown-linux-gnu
     default toolchain: stable (default)
               profile: default
  modify PATH variable: yes

1) Proceed with standard installation (default - just press enter)
2) Customize installation
3) Cancel installation
>
```




carlao2005@celtasdesktop: ~
1) Proceed with standard installation (default - just press enter)
2) Customize installation
3) Cancel installation
>
info: profile set to 'default'
info: default host triple is x86_64-unknown-linux-gnu
info: syncing channel updates for 'stable-x86_64-unknown-linux-gnu'
info: latest update on 2025-05-15, rust version 1.87.0 (17067e9ac 2025-05-09)
info: downloading component 'cargo'
9.4 MiB / 9.4 MiB (100 %) 6.7 MiB/s in 1s
info: downloading component 'clippy'
info: downloading component 'rust-docs'
19.9 MiB / 19.9 MiB (100 %) 7.2 MiB/s in 2s
info: downloading component 'rust-std'
29.4 MiB / 29.4 MiB (100 %) 7.7 MiB/s in 4s
info: downloading component 'rustc'
76.3 MiB / 76.3 MiB (100 %) 8.3 MiB/s in 9s
info: downloading component 'rustfmt'
info: installing component 'cargo'
info: installing component 'clippy'
info: installing component 'rust-docs'
19.9 MiB / 19.9 MiB (100 %) 11.0 MiB/s in 1s
info: installing component 'rust-std'
29.4 MiB / 29.4 MiB (100 %) 15.4 MiB/s in 1s
info: installing component 'rustc'
76.3 MiB / 76.3 MiB (100 %) 16.3 MiB/s in 4s
info: installing component 'rustfmt'
info: default toolchain set to 'stable-x86_64-unknown-linux-gnu'


carlao2005@celtasdesktop: ~
76.3 MiB / 76.3 MiB (100 %) 8.3 MiB/s in 9s
info: downloading component 'rustfmt'
info: installing component 'cargo'
info: installing component 'clippy'
info: installing component 'rust-docs'
19.9 MiB / 19.9 MiB (100 %) 11.0 MiB/s in 1s
info: installing component 'rust-std'
29.4 MiB / 29.4 MiB (100 %) 15.4 MiB/s in 1s
info: installing component 'rustc'
76.3 MiB / 76.3 MiB (100 %) 16.3 MiB/s in 4s
info: installing component 'rustfmt'
info: default toolchain set to 'stable-x86_64-unknown-linux-gnu'
stable-x86_64-unknown-linux-gnu installed - rustc 1.87.0 (17067e9ac 2025-05-09)
Rust is installed now. Great!
To get started you may need to restart your current shell.
This would reload your PATH environment variable to include
Cargo's bin directory ($HOME/.cargo/bin).
To configure your current shell, you need to source
the corresponding env file under $HOME/.cargo.
This is usually done by running one of the following (note the leading DOT):
. "$HOME/.cargo/env" # For sh/bash/zsh/ash/dash/pdksh
source "$HOME/.cargo/env.fish" # For fish
source $"($nu.home-path)/.cargo/env.nu" # For nushell
carlao2005@celtasdesktop:~$

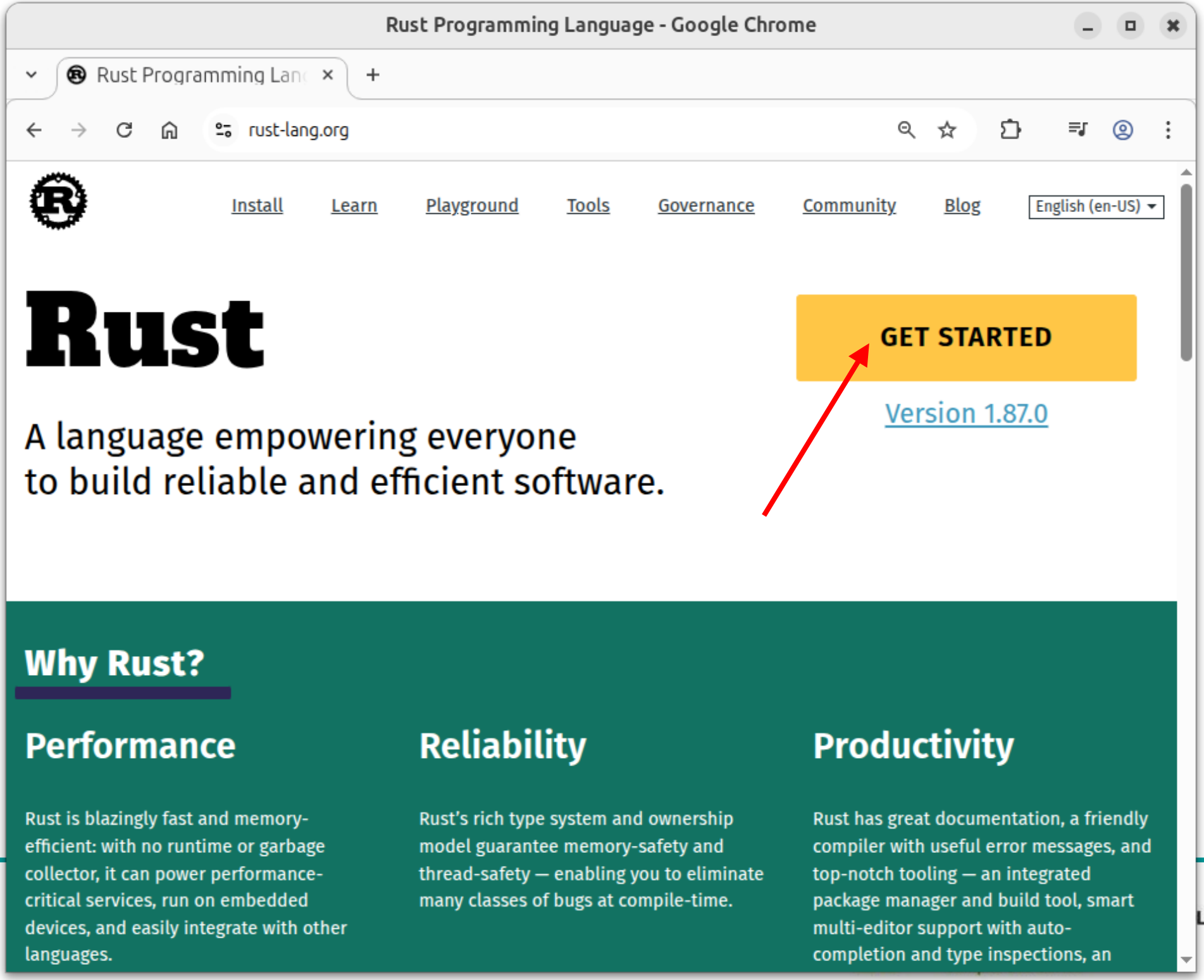
Fechei e reabri a janela do terminal...

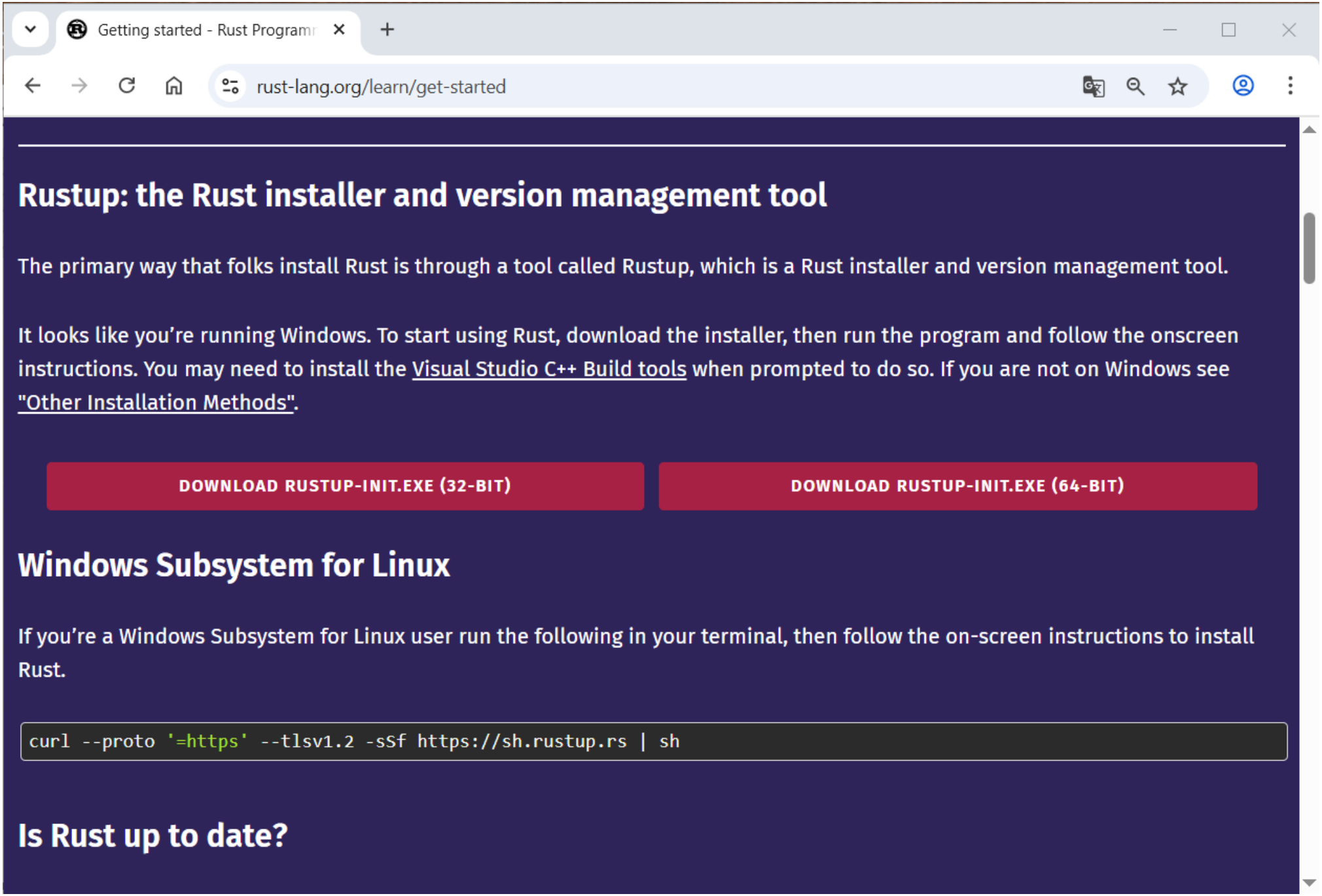

carlao2005@celtasdesktop: ~
carlao2005@celtasdesktop:~$ rustc -V
rustc 1.87.0 (17067e9ac 2025-05-09)
carlao2005@celtasdesktop:~$
carlao2005@celtasdesktop:~$ cargo -V
cargo 1.87.0 (99624be96 2025-05-06)
carlao2005@celtasdesktop:~$
carlao2005@celtasdesktop:~$

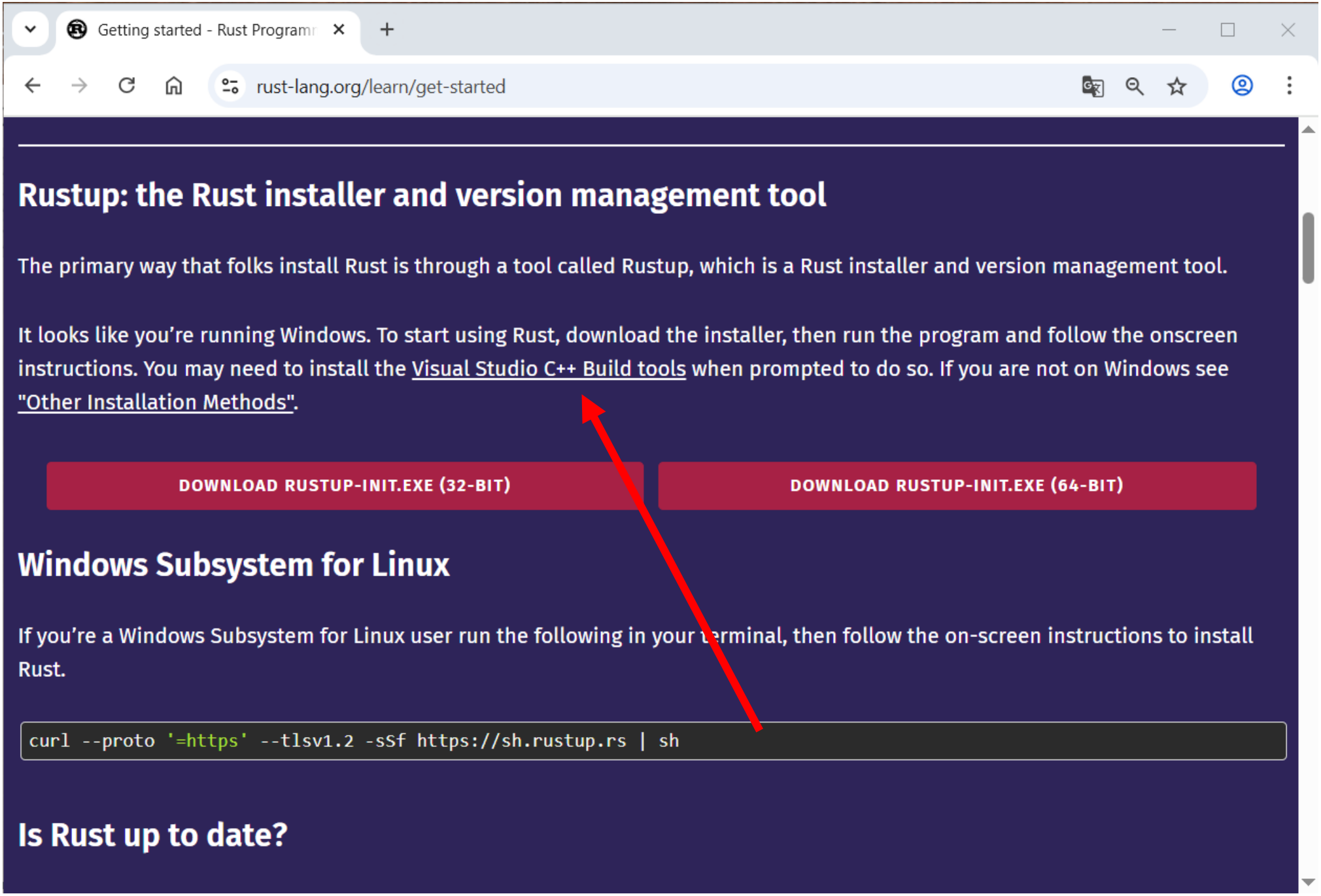
```
carlao2005@celtasdesktop:~$ rustc -V
rustc 1.87.0 (17067e9ac 2025-05-09)
carlao2005@celtasdesktop:~$
carlao2005@celtasdesktop:~$ cargo -V
cargo 1.87.0 (99624be96 2025-05-06)
carlao2005@celtasdesktop:~$
carlao2005@celtasdesktop:~$
```

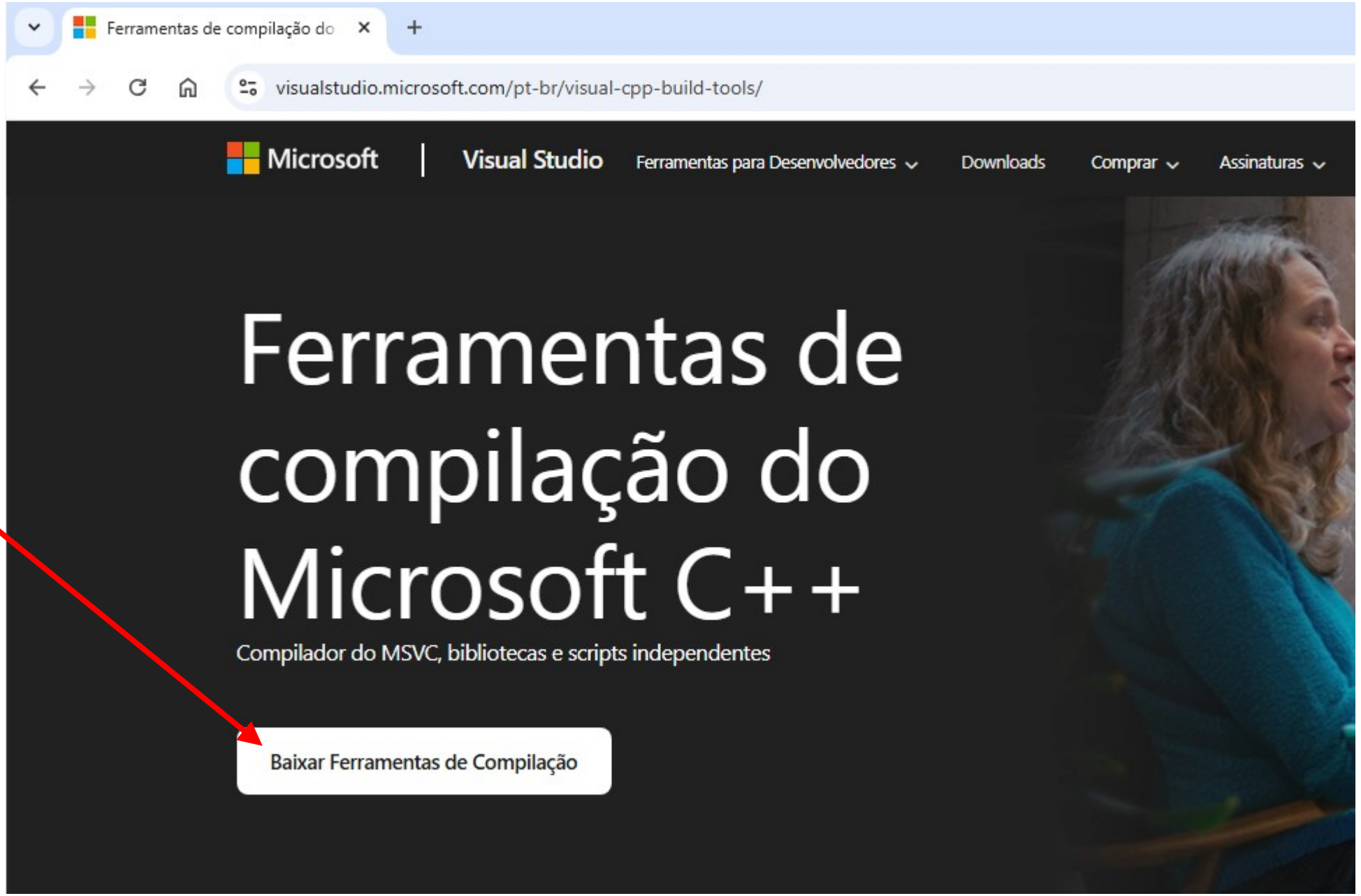


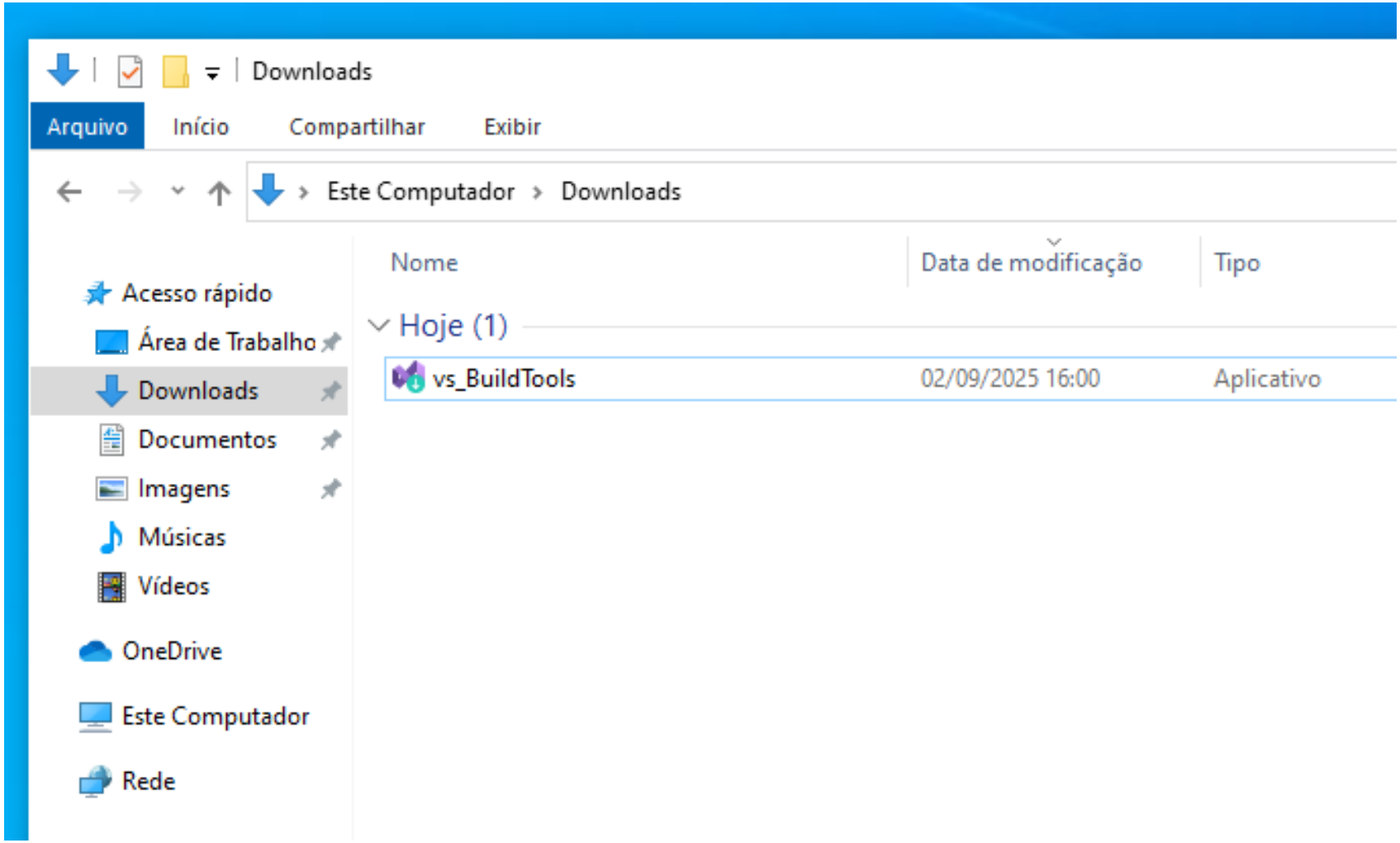
Instalação Windows

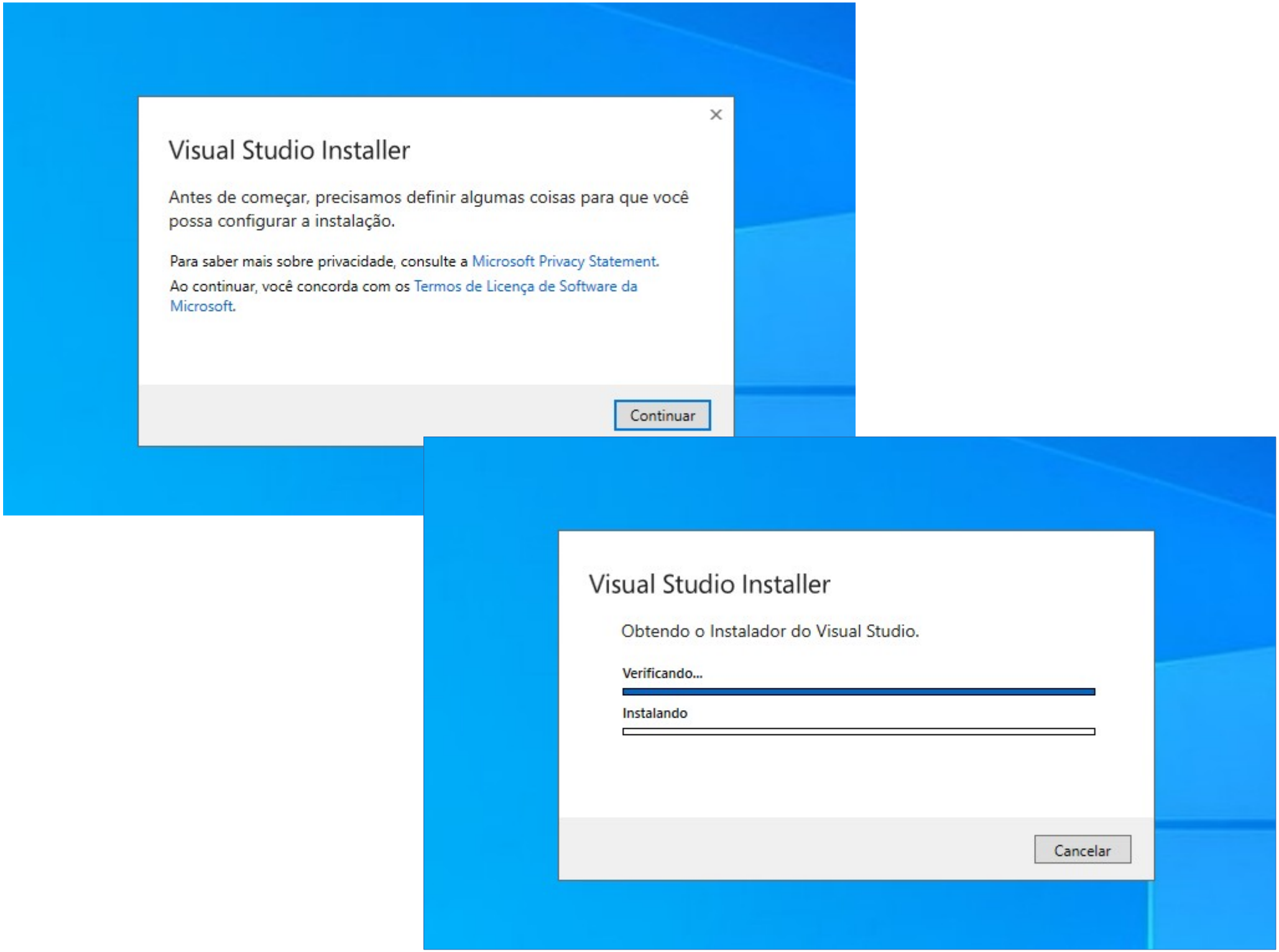

Rust Programming Language - Google Chrome
Rust Programming Lan
rust-lang.org
Install
Learn
Playground
Tools
Governance
Community
Blog
English (en-US)
Rust
A language empowering everyone
to build reliable and efficient software.
GET STARTED
Version 1.87.0
Why Rust?
Performance
Rust is blazingly fast and memory-efficient: with no runtime or garbage collector, it can power performance-critical services, run on embedded devices, and easily integrate with other languages.
Reliability
Rust's rich type system and ownership model guarantee memory-safety and thread-safety — enabling you to eliminate many classes of bugs at compile-time.
Productivity
Rust has great documentation, a friendly compiler with useful error messages, and top-notch tooling — an integrated package manager and build tool, smart multi-editor support with auto-completion and type inspections, an

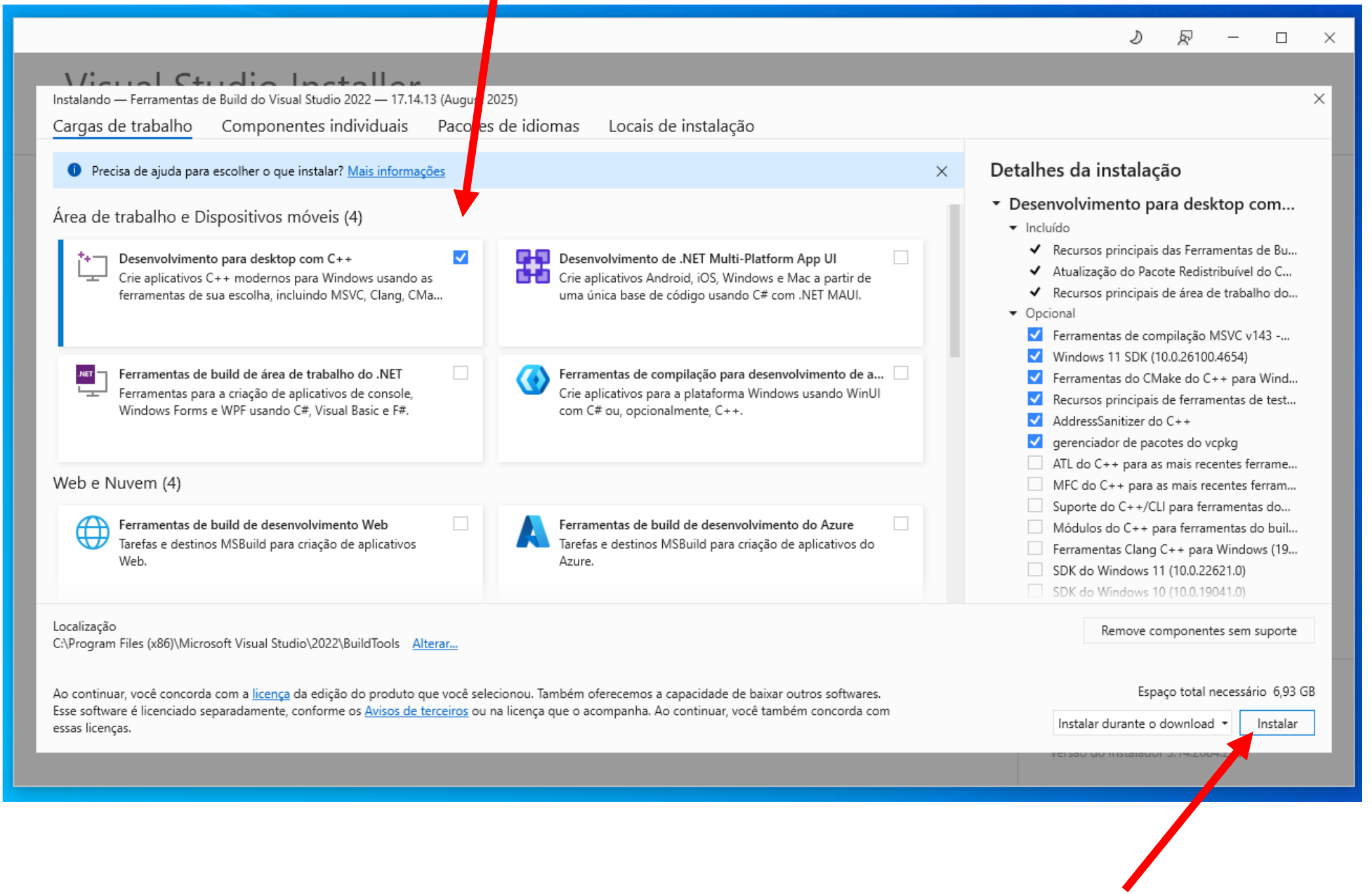

Getting started - Rust Program
rust-lang.org/learn/get-started
Rustup: the Rust installer and version management tool
The primary way that folks install Rust is through a tool called Rustup, which is a Rust installer and version management tool.
It looks like you're running Windows. To start using Rust, download the installer, then run the program and follow the onscreen instructions. You may need to install the Visual Studio C++ Build tools when prompted to do so. If you are not on Windows see "Other Installation Methods".
DOWNLOAD RUSTUP-INIT.EXE (32-BIT)
DOWNLOAD RUSTUP-INIT.EXE (64-BIT)
Windows Subsystem for Linux
If you're a Windows Subsystem for Linux user run the following in your terminal, then follow the on-screen instructions to install Rust.
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
Is Rust up to date?

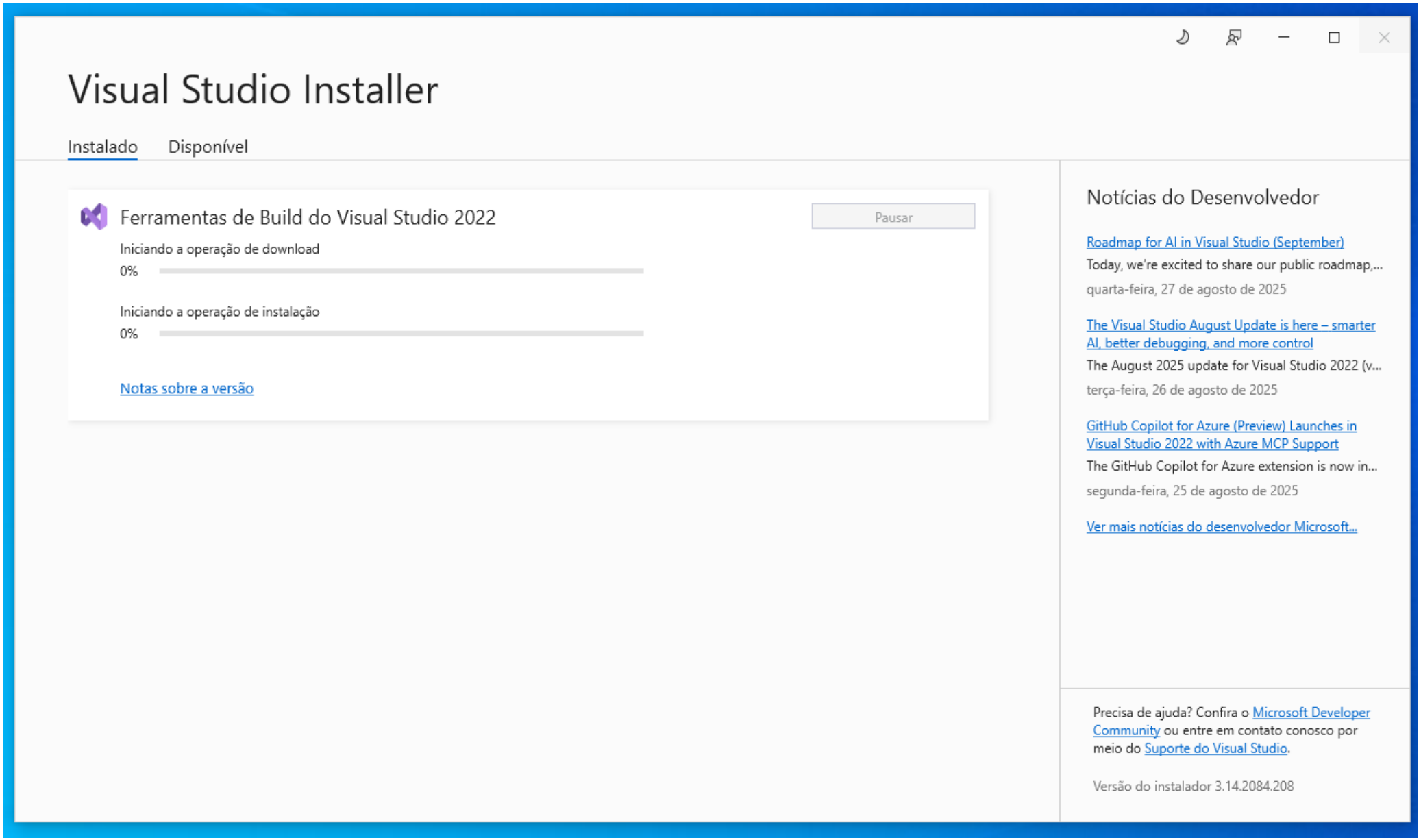

Getting started - Rust Programm
rust-lang.org/learn/get-started
Rustup: the Rust installer and version management tool
The primary way that folks install Rust is through a tool called Rustup, which is a Rust installer and version management tool.
It looks like you're running Windows. To start using Rust, download the installer, then run the program and follow the onscreen instructions. You may need to install the Visual Studio C++ Build tools when prompted to do so. If you are not on Windows see "Other Installation Methods".
DOWNLOAD RUSTUP-INIT.EXE (32-BIT)
DOWNLOAD RUSTUP-INIT.EXE (64-BIT)
Windows Subsystem for Linux
If you're a Windows Subsystem for Linux user run the following in your terminal, then follow the on-screen instructions to install Rust.
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
Is Rust up to date?

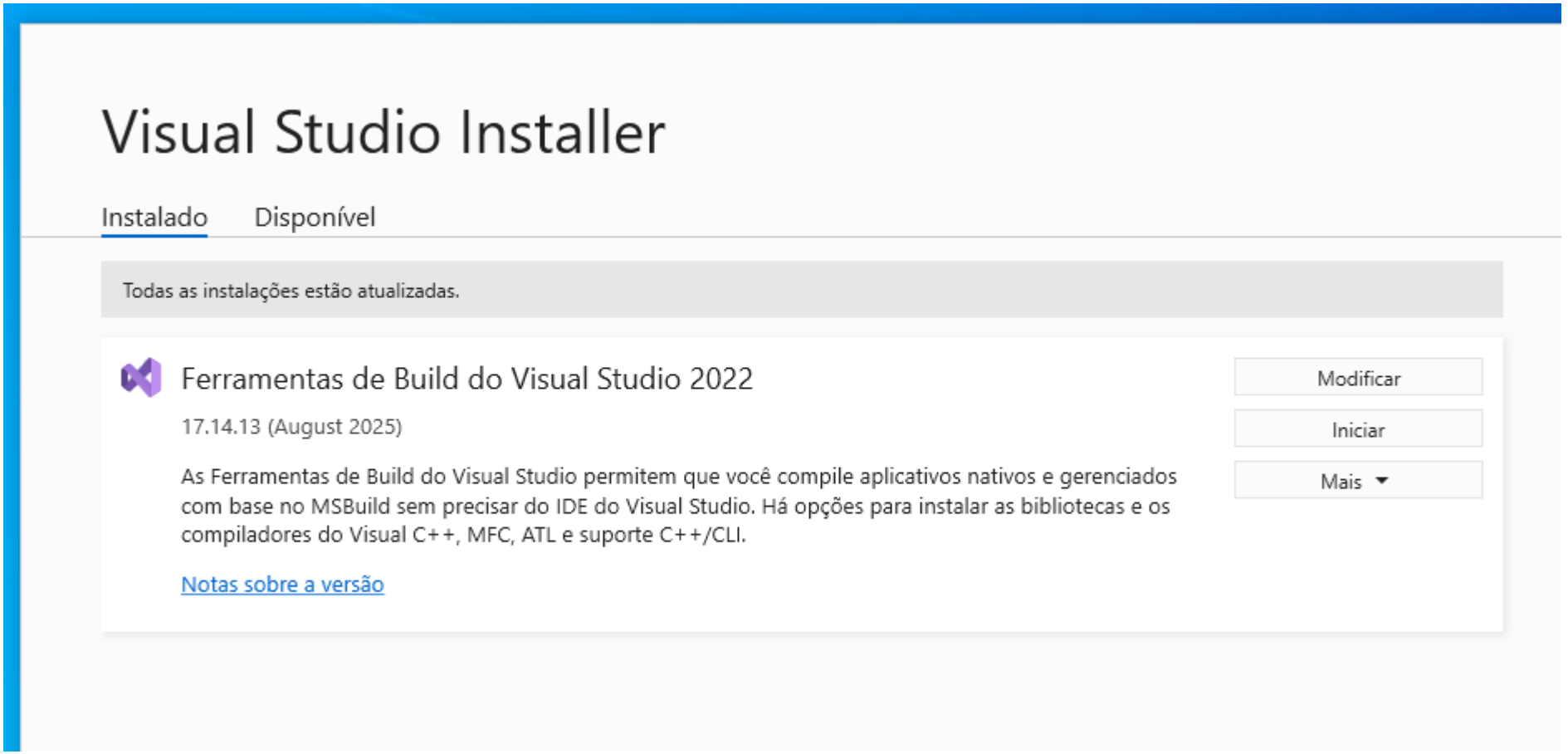

Ferramentas de compilação do
visualstudio.microsoft.com/pt-br/visual-cpp-build-tools/
Microsoft | Visual Studio
Ferramentas para Desenvolvedores
Downloads
Comprar
Assinaturas
Ferramentas de compilação do Microsoft C++
Compilador do MSVC, bibliotecas e scripts independentes
Baixar Ferramentas de Compilação

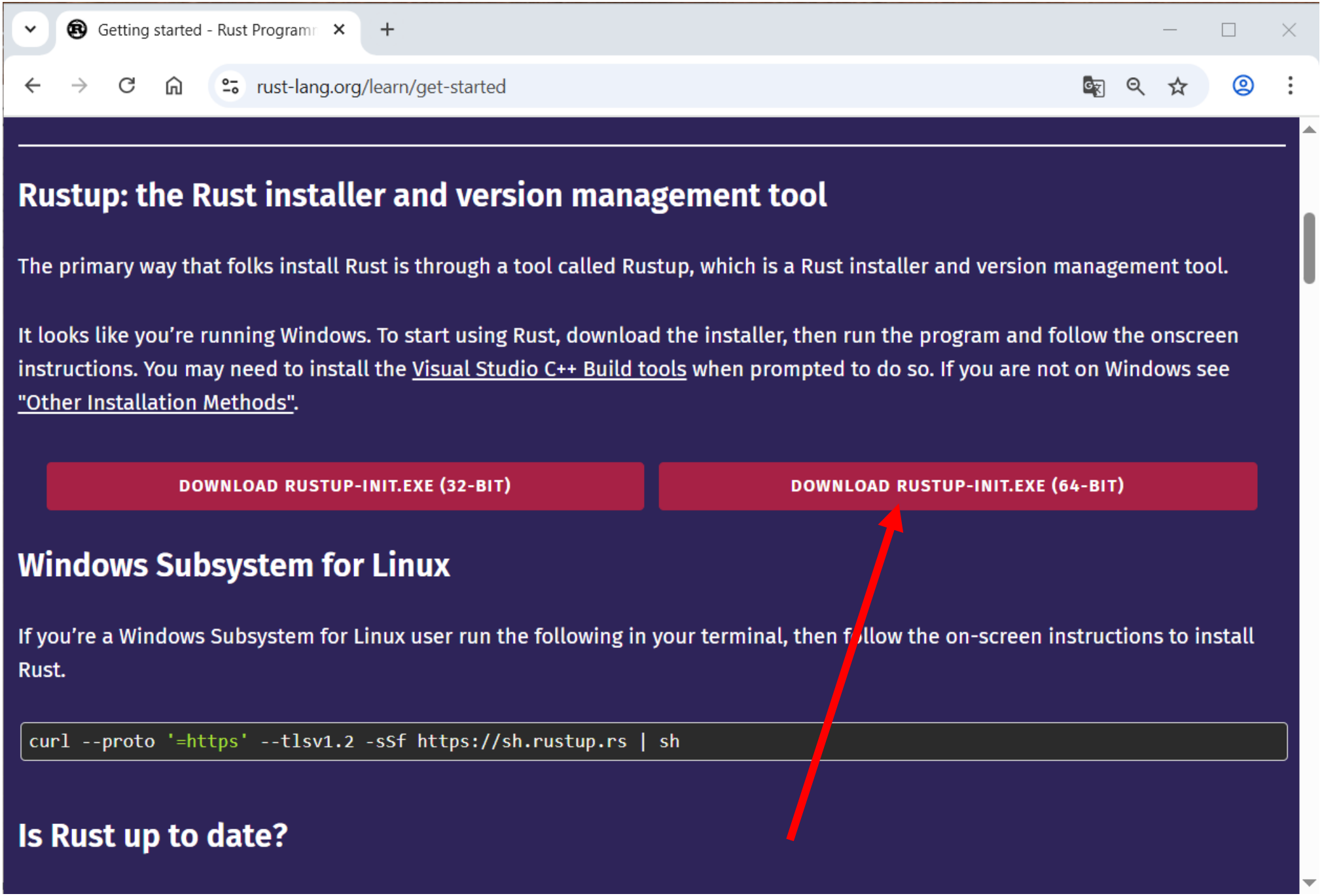

Downloads
Arquivo
Início
Compartilhar
Exibir
Este Computador
Downloads
Acesso rápido
Área de Trabalho
Downloads
Documentos
Imagens
Músicas
Vídeos
OneDrive
Este Computador
Rede
Nome
Data de modificação
Tipo
Hoje (1)
vs_BuildTools
02/09/2025 16:00
Aplicativo

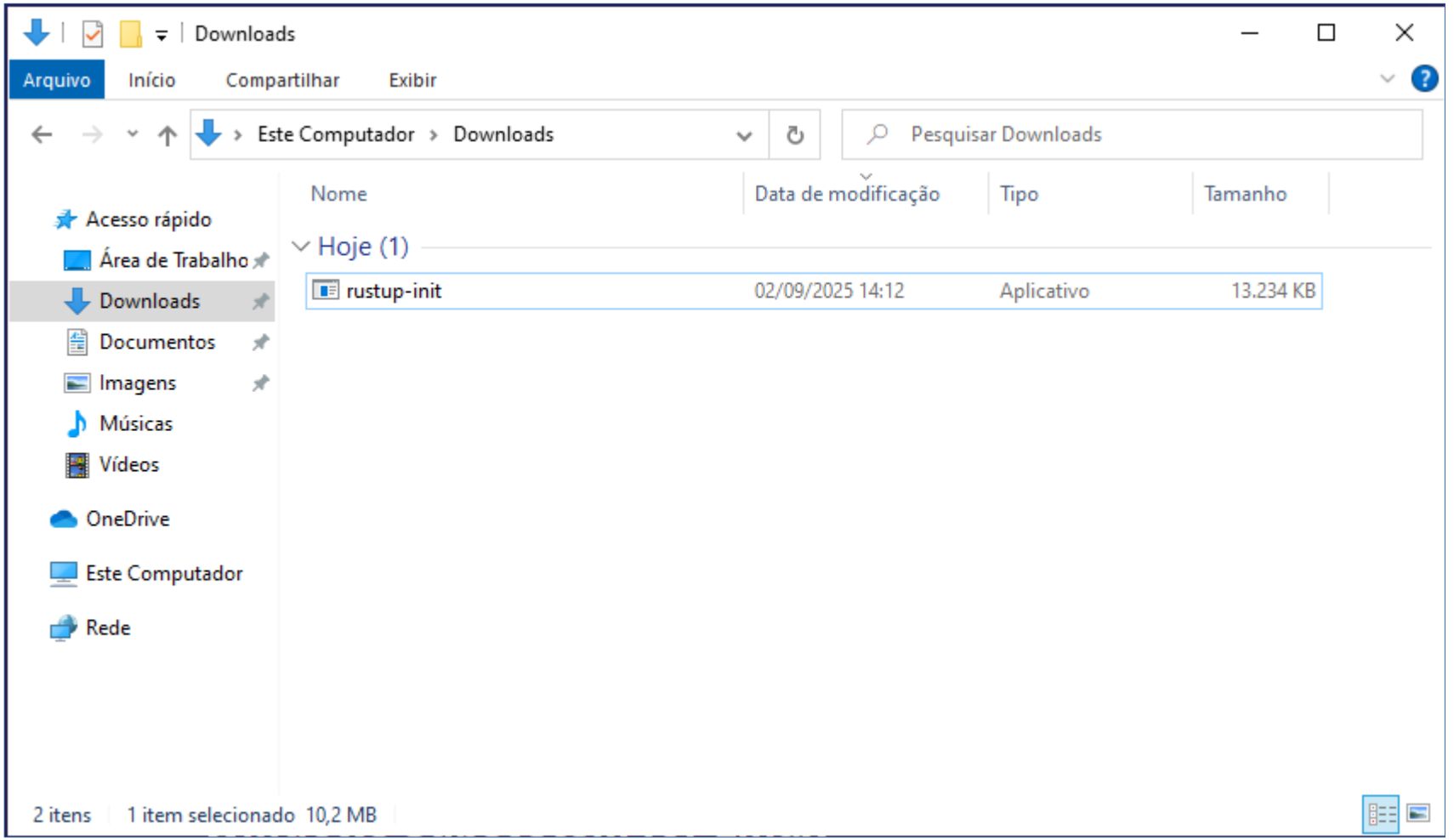

Visual Studio Installer
Antes de começar, precisamos definir algumas coisas para que você possa configurar a instalação.
Para saber mais sobre privacidade, consulte a Microsoft Privacy Statement.
Ao continuar, você concorda com os Termos de Licença de Software da Microsoft.
Continuar
Visual Studio Installer
Obtendo o Instalador do Visual Studio.
Verificando...
Instalando
Cancelar

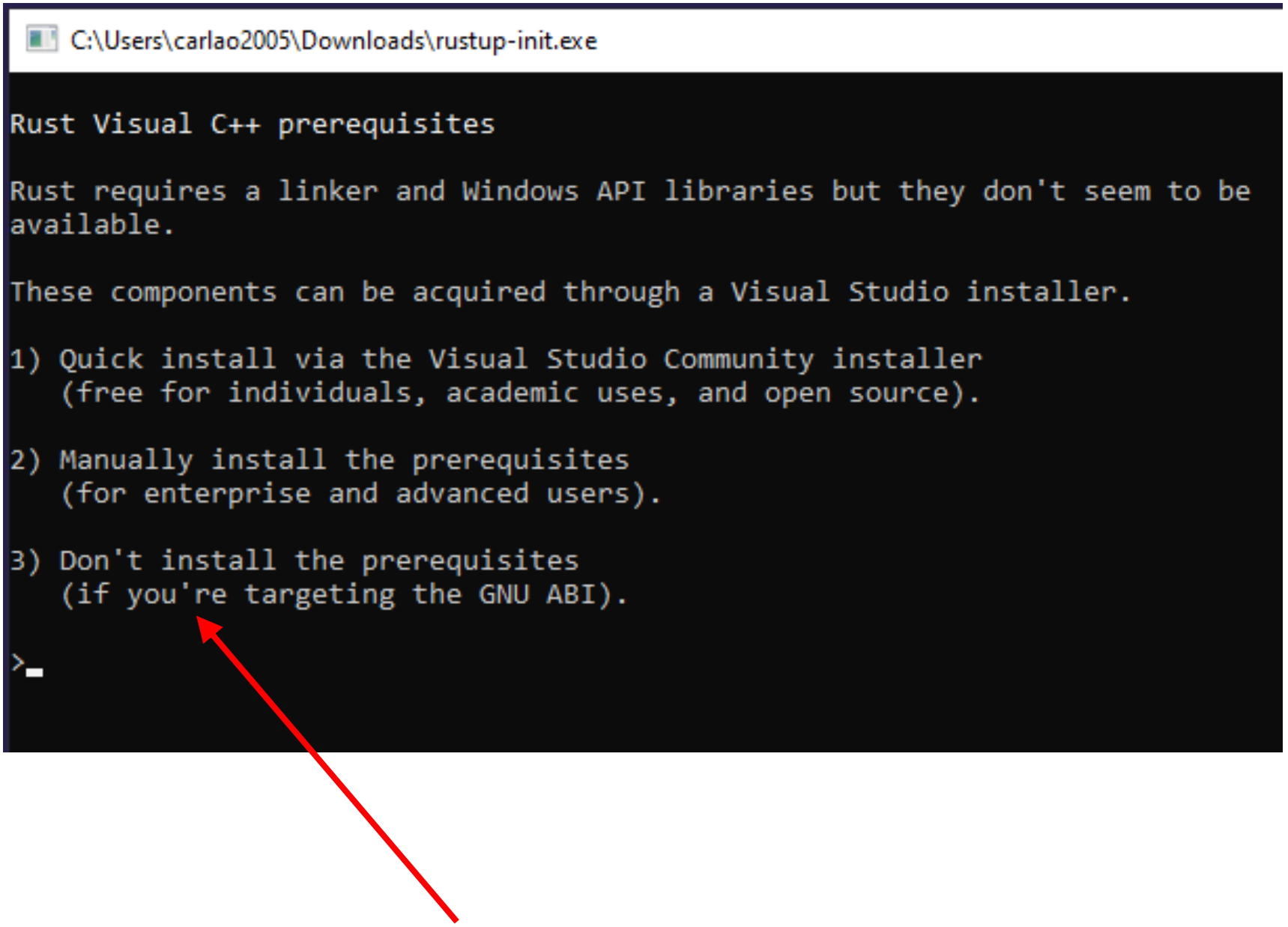

Instalando — Ferramentas de Build do Visual Studio 2022 — 17.14.13 (Augu 2025)
Cargas de trabalho
Componentes individuais
Pac es de idiomas
Locais de instalação
Precisa de ajuda para escolher o que instalar? Mais informações
Área de trabalho e Dispositivos móveis (4)
Desenvolvimento para desktop com C++
Crie aplicativos C++ modernos para Windows usando as ferramentas de sua escolha, incluindo MSVC, Clang, CMa...
Desenvolvimento de .NET Multi-Platform App UI
Crie aplicativos Android, iOS, Windows e Mac a partir de uma única base de código usando C# com .NET MAUI.
Ferramentas de build de área de trabalho do .NET
Ferramentas para a criação de aplicativos de console, Windows Forms e WPF usando C#, Visual Basic e F#.
Ferramentas de compilação para desenvolvimento de a...
Crie aplicativos para a plataforma Windows usando WinUI com C# ou, opcionalmente, C++.
Web e Nuvem (4)
Ferramentas de build de desenvolvimento Web
Tarefas e destinos MSBuild para criação de aplicativos Web.
Ferramentas de build de desenvolvimento do Azure
Tarefas e destinos MSBuild para criação de aplicativos do Azure.
Detalhes da instalação
Desenvolvimento para desktop com...
Incluído
Recursos principais das Ferramentas de Bu...
Atualização do Pacote Redistribuível do C...
Recursos principais de área de trabalho do...
Opcional
Ferramentas de compilação MSVC v143 -...
Windows 11 SDK (10.0.26100.4654)
Ferramentas do CMake do C++ para Wind...
Recursos principais de ferramentas de test...
AddressSanitizer do C++
gerenciador de pacotes do vcpkg
ATL do C++ para as mais recentes ferrame...
MFC do C++ para as mais recentes ferram...
Suporte do C++/CLI para ferramentas do...
Módulos do C++ para ferramentas do buil...
Ferramentas Clang C++ para Windows (19...
SDK do Windows 11 (10.0.22621.0)
SDK do Windows 10 (10.0.19041.0)
Localização
C:\Program Files (x86)\Microsoft Visual Studio\2022\BuildTools Alterar...
Remove componentes sem suporte
Ao continuar, você concorda com a licença da edição do produto que você selecionou. Também oferecemos a capacidade de baixar outros softwares. Esse software é licenciado separadamente, conforme os Avisos de terceiros ou na licença que o acompanha. Ao continuar, você também concorda com essas licenças.
Espaço total necessário 6,93 GB
Instalar durante o download
Instalar

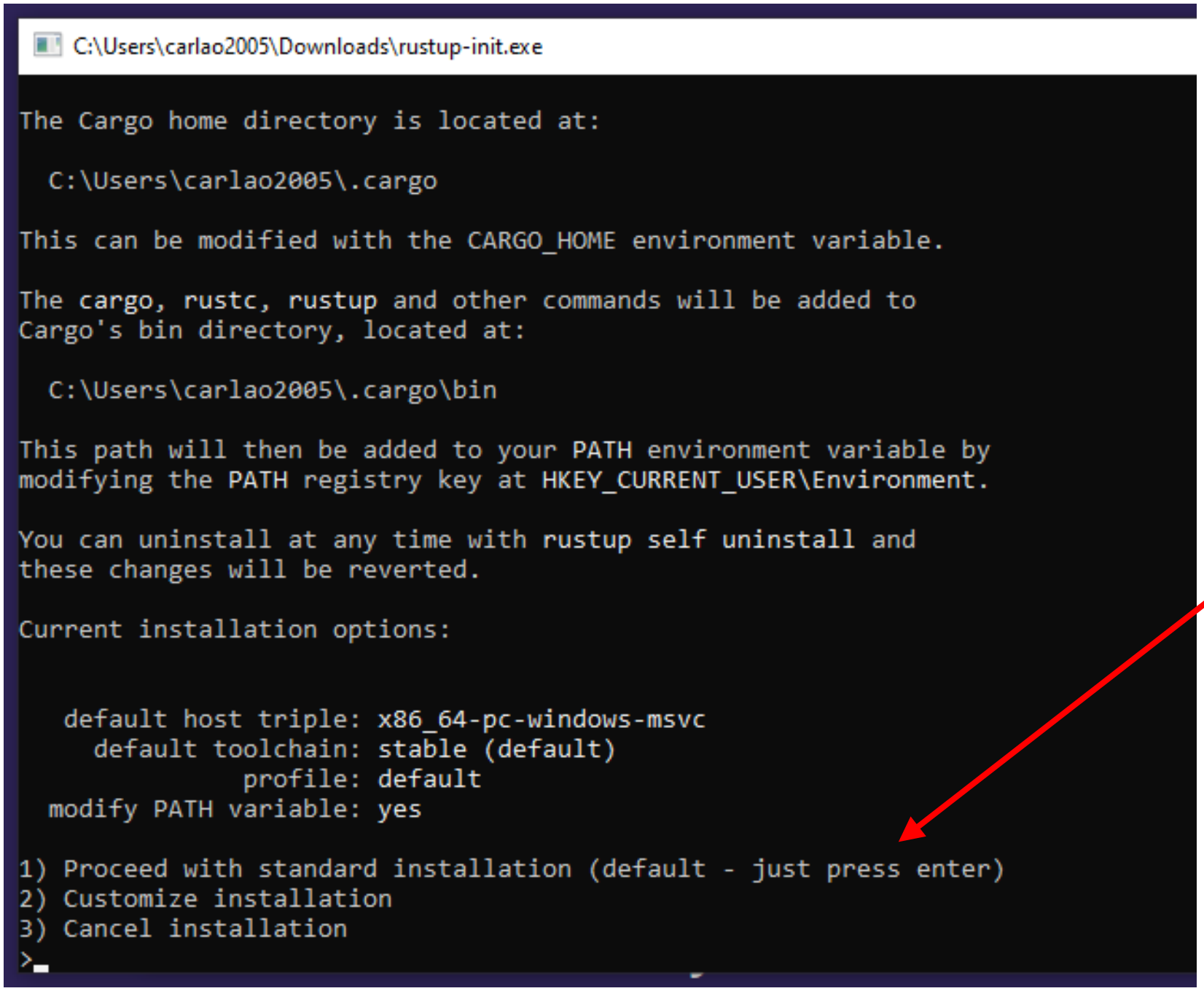

Visual Studio Installer
Instalado
Disponível
Ferramentas de Build do Visual Studio 2022
Pausar
Iniciando a operação de download
0%
Iniciando a operação de instalação
0%
Notas sobre a versão
Notícias do Desenvolvedor
Roadmap for AI in Visual Studio (September)
Today, we're excited to share our public roadmap,...
quarta-feira, 27 de agosto de 2025
The Visual Studio August Update is here – smarter AI, better debugging, and more control
The August 2025 update for Visual Studio 2022 (v...
terça-feira, 26 de agosto de 2025
GitHub Copilot for Azure (Preview) Launches in Visual Studio 2022 with Azure MCP Support
The GitHub Copilot for Azure extension is now in...
segunda-feira, 25 de agosto de 2025
Ver mais notícias do desenvolvedor Microsoft...
Precisa de ajuda? Confira o Microsoft Developer Community ou entre em contato conosco por meio do Suporte do Visual Studio.
Versão do instalador 3.14.2084.208

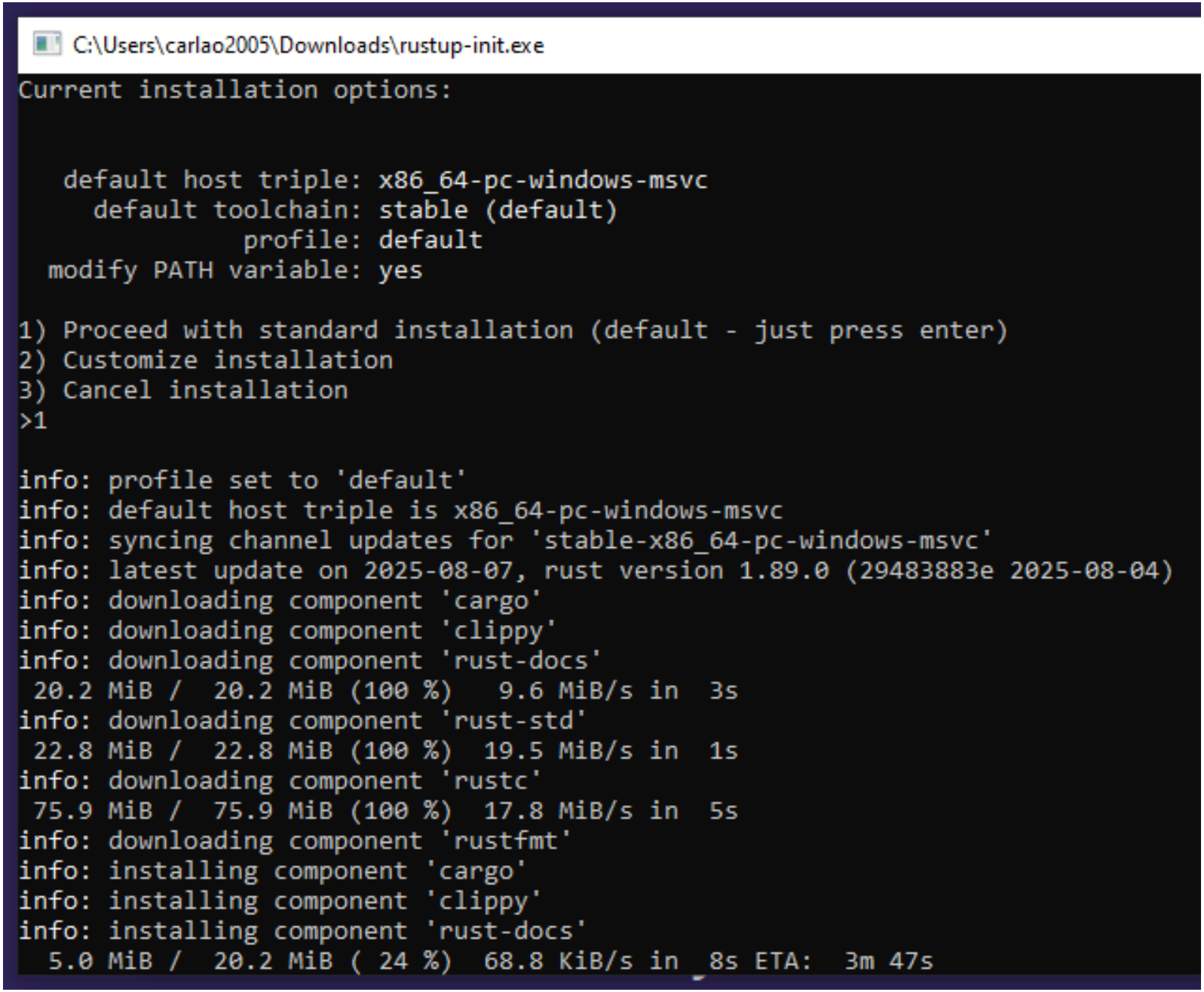

Visual Studio Installer
Instalado
Disponível
Todas as instalações estão atualizadas.
Ferramentas de Build do Visual Studio 2022
17.14.13 (August 2025)
As Ferramentas de Build do Visual Studio permitem que você compile aplicativos nativos e gerenciados com base no MSBuild sem precisar do IDE do Visual Studio. Há opções para instalar as bibliotecas e os compiladores do Visual C++, MFC, ATL e suporte C++/CLI.
Notas sobre a versão
Modificar
Iniciar
Mais

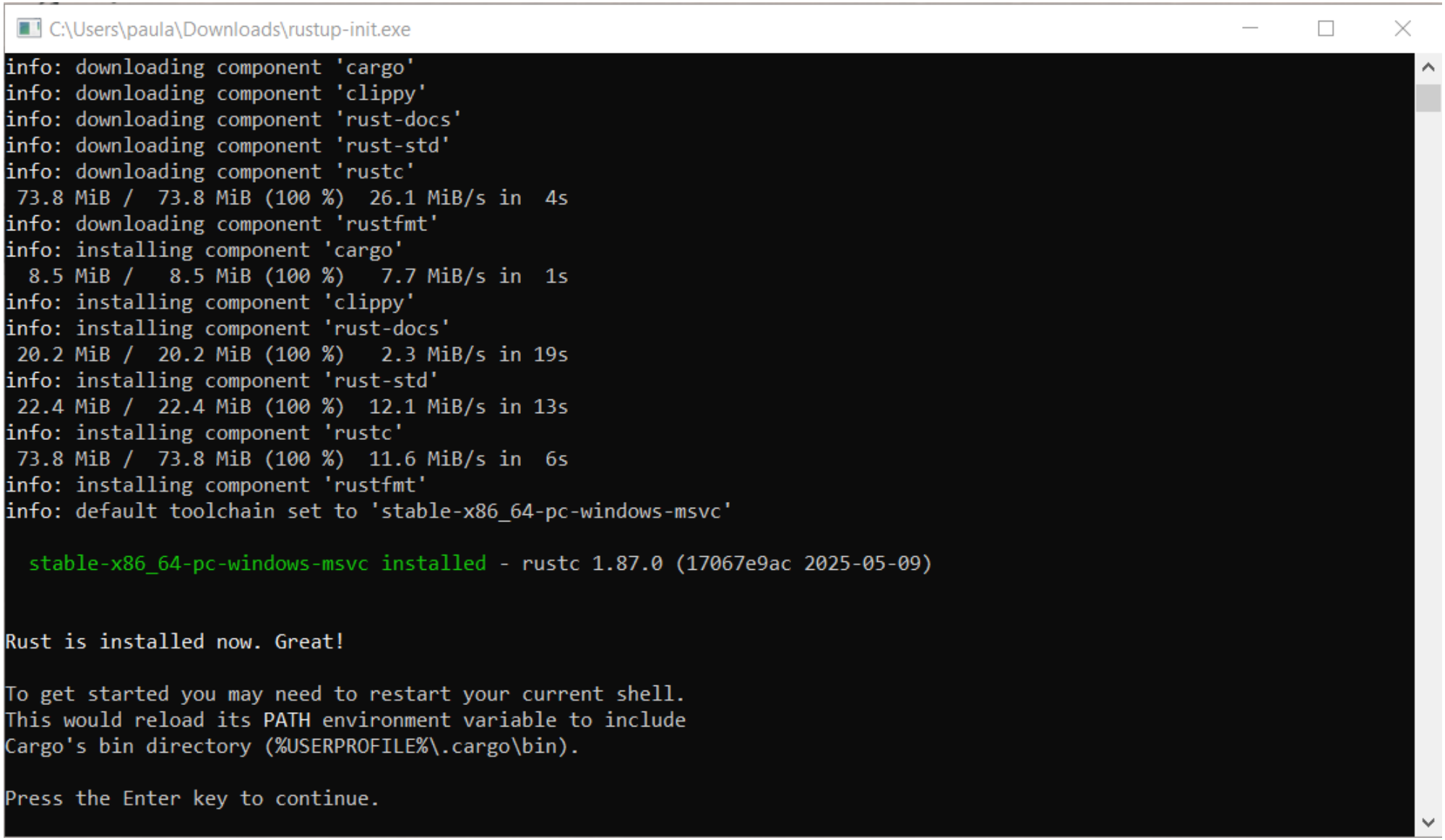

Getting started - Rust Programm
rust-lang.org/learn/get-started
Rustup: the Rust installer and version management tool
The primary way that folks install Rust is through a tool called Rustup, which is a Rust installer and version management tool.
It looks like you're running Windows. To start using Rust, download the installer, then run the program and follow the onscreen instructions. You may need to install the Visual Studio C++ Build tools when prompted to do so. If you are not on Windows see "Other Installation Methods".
DOWNLOAD RUSTUP-INIT.EXE (32-BIT)
DOWNLOAD RUSTUP-INIT.EXE (64-BIT)
Windows Subsystem for Linux
If you're a Windows Subsystem for Linux user run the following in your terminal, then follow the on-screen instructions to install Rust.
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
Is Rust up to date?

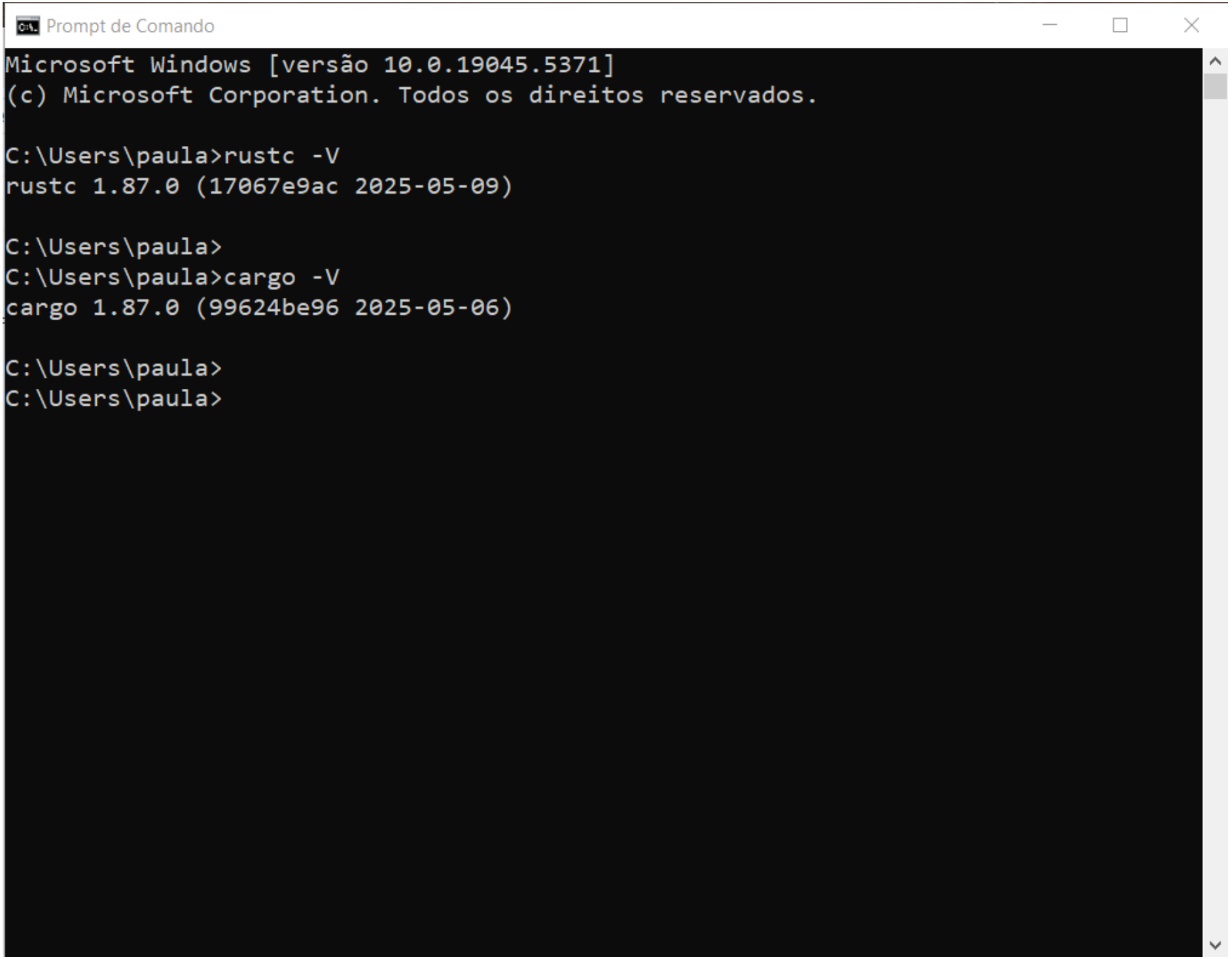

Downloads
Arquivo
Início
Compartilhar
Exibir
Este Computador
Downloads
Pesquisar Downloads
Acesso rápido
Área de Trabalho
Downloads
Documentos
Imagens
Músicas
Vídeos
OneDrive
Este Computador
Rede
Nome
Data de modificação
Tipo
Tamanho
Hoje (1)
rustup-init
02/09/2025 14:12
Aplicativo
13.234 KB
2 itens
1 item selecionado 10,2 MB


C:\Users\carlao2005\Downloads\rustup-init.exe
Rust Visual C++ prerequisites
Rust requires a linker and Windows API libraries but they don't seem to be available.
These components can be acquired through a Visual Studio installer.
1) Quick install via the Visual Studio Community installer
(free for individuals, academic uses, and open source).
2) Manually install the prerequisites
(for enterprise and advanced users).
3) Don't install the prerequisites
(if you're targeting the GNU ABI).
>


C:\Users\carlao2005\Downloads\rustup-init.exe
The Cargo home directory is located at:
C:\Users\carlao2005\.cargo
This can be modified with the CARGO_HOME environment variable.
The cargo, rustc, rustup and other commands will be added to
Cargo's bin directory, located at:
C:\Users\carlao2005\.cargo\bin
This path will then be added to your PATH environment variable by
modifying the PATH registry key at HKEY_CURRENT_USER\Environment.
You can uninstall at any time with rustup self uninstall and
these changes will be reverted.
Current installation options:
default host triple: x86_64-pc-windows-msvc
default toolchain: stable (default)
profile: default
modify PATH variable: yes
1) Proceed with standard installation (default - just press enter)
2) Customize installation
3) Cancel installation
>

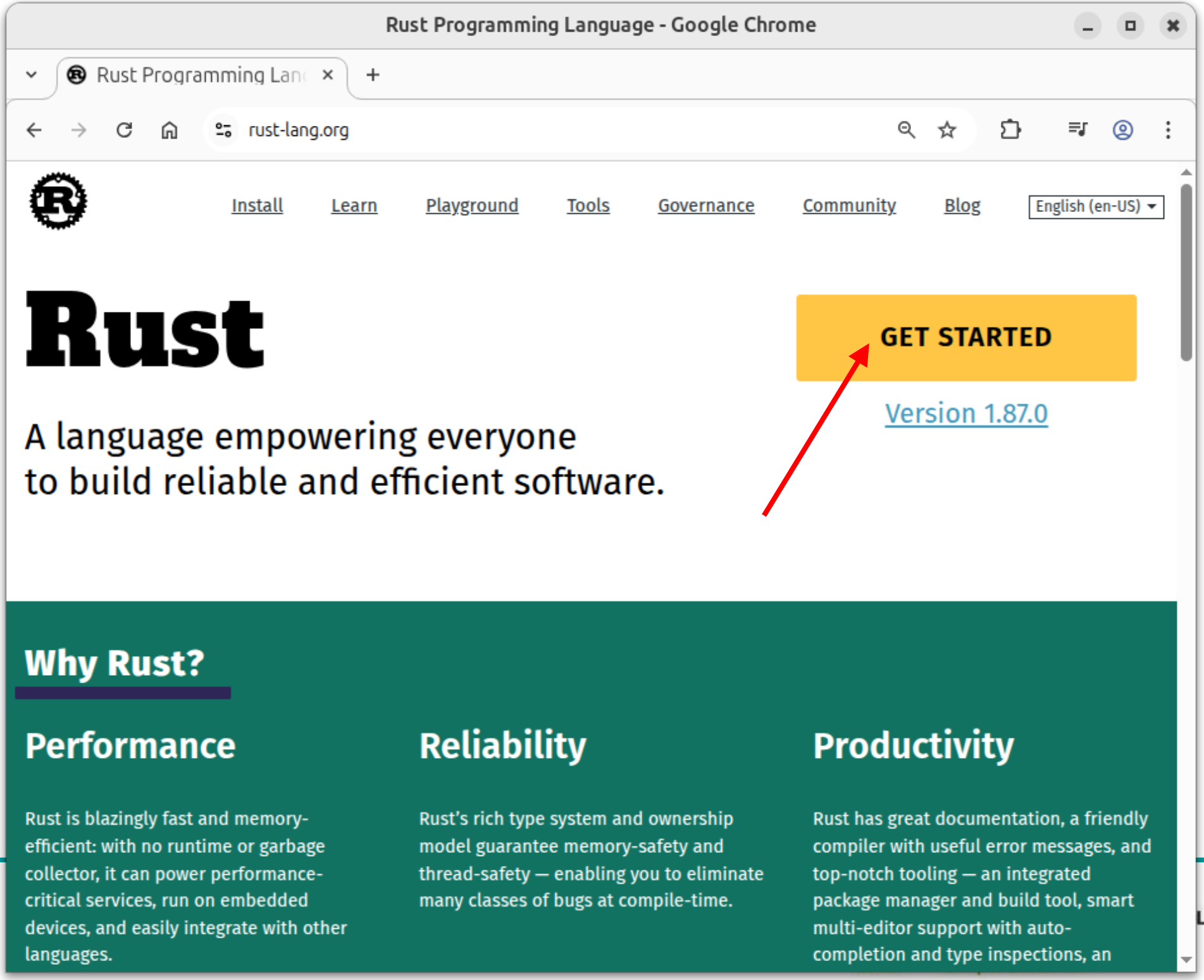

C:\Users\carlao2005\Downloads\rustup-init.exe
Current installation options:
default host triple: x86_64-pc-windows-msvc
default toolchain: stable (default)
profile: default
modify PATH variable: yes
1) Proceed with standard installation (default - just press enter)
2) Customize installation
3) Cancel installation
>1
info: profile set to 'default'
info: default host triple is x86_64-pc-windows-msvc
info: syncing channel updates for 'stable-x86_64-pc-windows-msvc'
info: latest update on 2025-08-07, rust version 1.89.0 (29483883e 2025-08-04)
info: downloading component 'cargo'
info: downloading component 'clippy'
info: downloading component 'rust-docs'
20.2 MiB / 20.2 MiB (100 %) 9.6 MiB/s in 3s
info: downloading component 'rust-std'
22.8 MiB / 22.8 MiB (100 %) 19.5 MiB/s in 1s
info: downloading component 'rustc'
75.9 MiB / 75.9 MiB (100 %) 17.8 MiB/s in 5s
info: downloading component 'rustfmt'
info: installing component 'cargo'
info: installing component 'clippy'
info: installing component 'rust-docs'
5.0 MiB / 20.2 MiB (24 %) 68.8 KiB/s in 8s ETA: 3m 47s

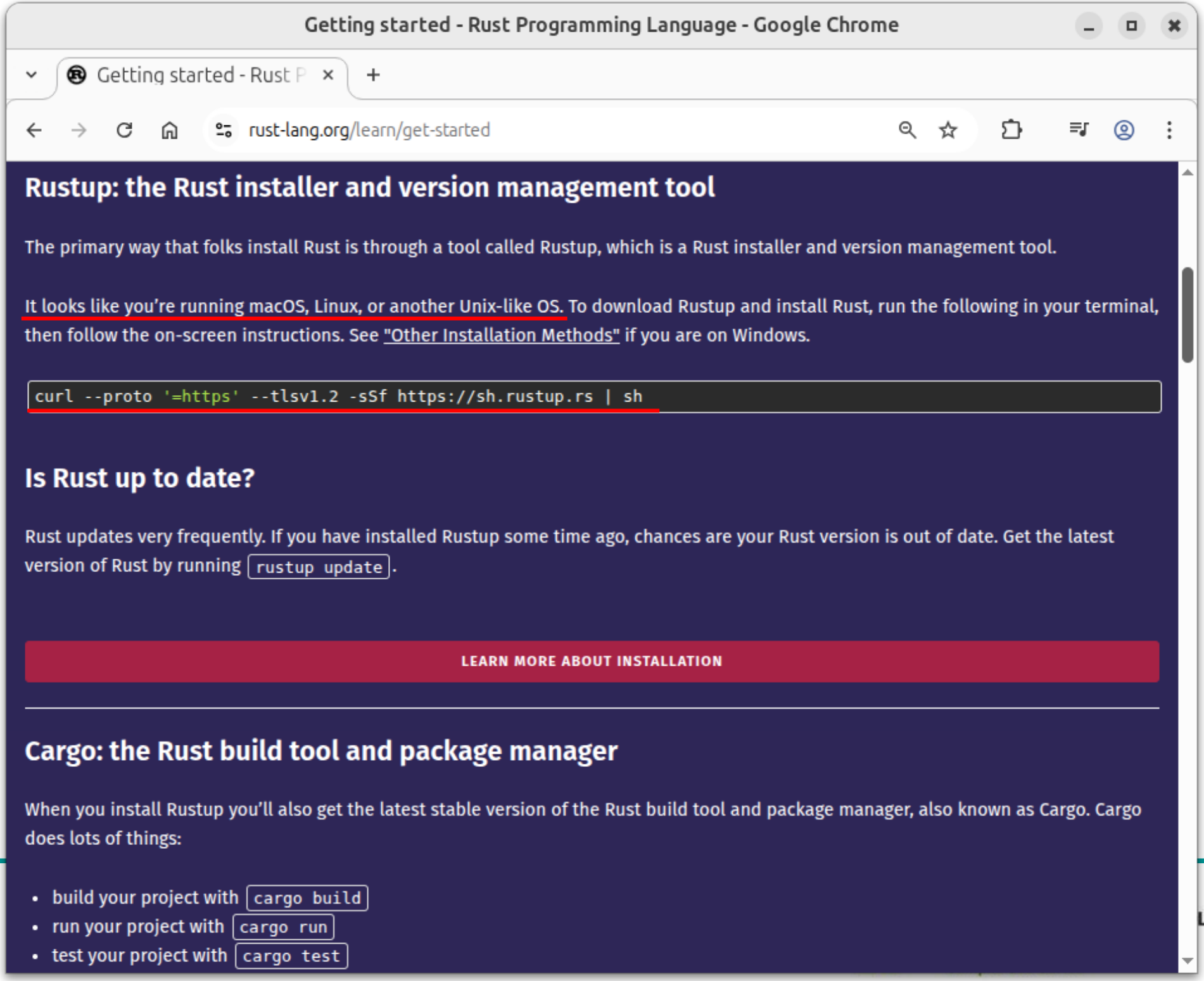

C:\Users\paula\Downloads\rustup-init.exe
info: downloading component 'cargo'
info: downloading component 'clippy'
info: downloading component 'rust-docs'
info: downloading component 'rust-std'
info: downloading component 'rustc'
73.8 MiB / 73.8 MiB (100 %) 26.1 MiB/s in 4s
info: downloading component 'rustfmt'
info: installing component 'cargo'
8.5 MiB / 8.5 MiB (100 %) 7.7 MiB/s in 1s
info: installing component 'clippy'
info: installing component 'rust-docs'
20.2 MiB / 20.2 MiB (100 %) 2.3 MiB/s in 19s
info: installing component 'rust-std'
22.4 MiB / 22.4 MiB (100 %) 12.1 MiB/s in 13s
info: installing component 'rustc'
73.8 MiB / 73.8 MiB (100 %) 11.6 MiB/s in 6s
info: installing component 'rustfmt'
info: default toolchain set to 'stable-x86_64-pc-windows-msvc'
stable-x86_64-pc-windows-msvc installed - rustc 1.87.0 (17067e9ac 2025-05-09)
Rust is installed now. Great!
To get started you may need to restart your current shell.
This would reload its PATH environment variable to include
Cargo's bin directory (%USERPROFILE%\.cargo\bin).
Press the Enter key to continue.


Prompt de Comando
Microsoft Windows [versão 10.0.19045.5371]
(c) Microsoft Corporation. Todos os direitos reservados.
C:\Users\paula>rustc -V
rustc 1.87.0 (17067e9ac 2025-05-09)
C:\Users\paula>
C:\Users\paula>cargo -V
cargo 1.87.0 (99624be96 2025-05-06)
C:\Users\paula>
C:\Users\paula>

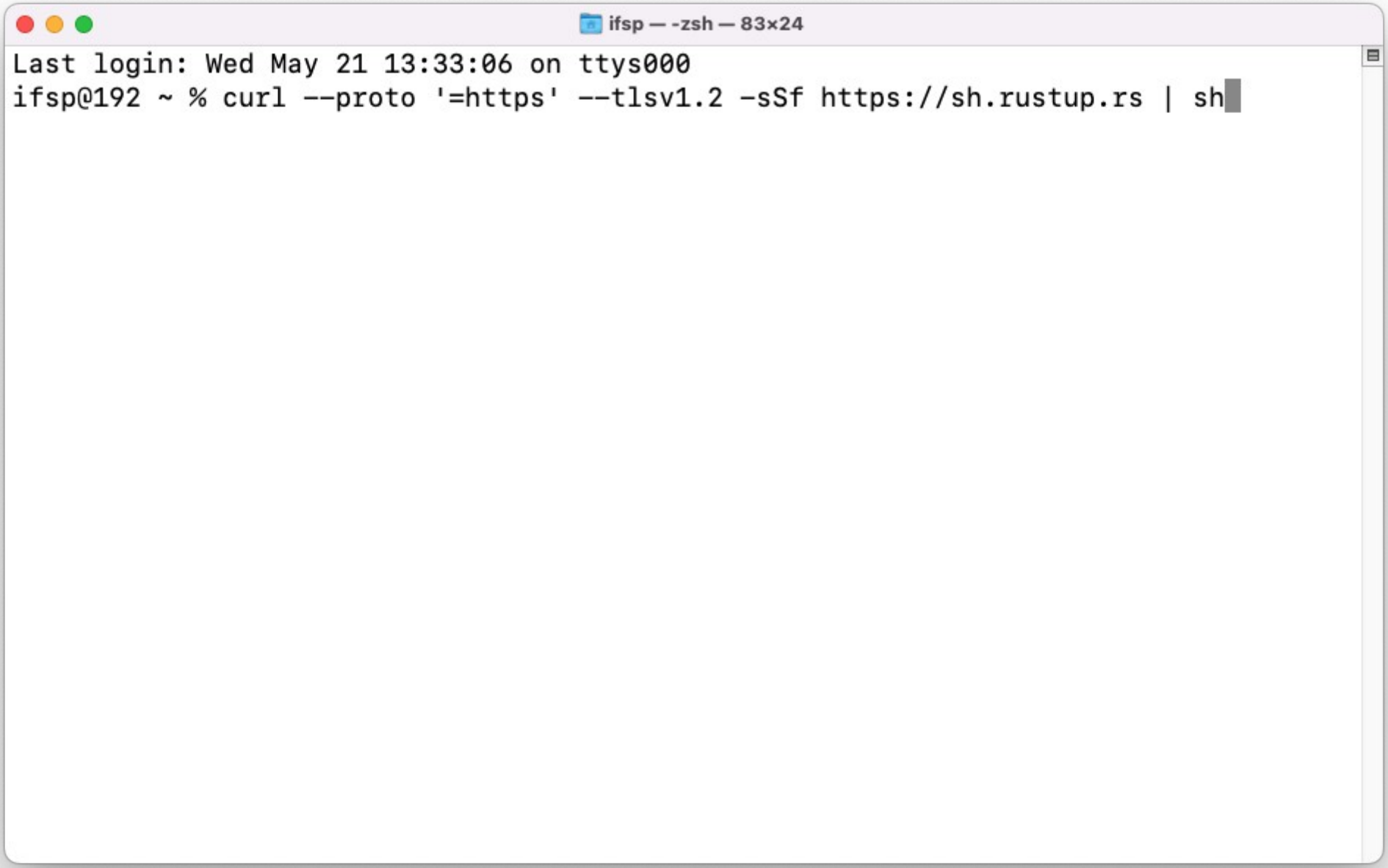
Instalação no Mac

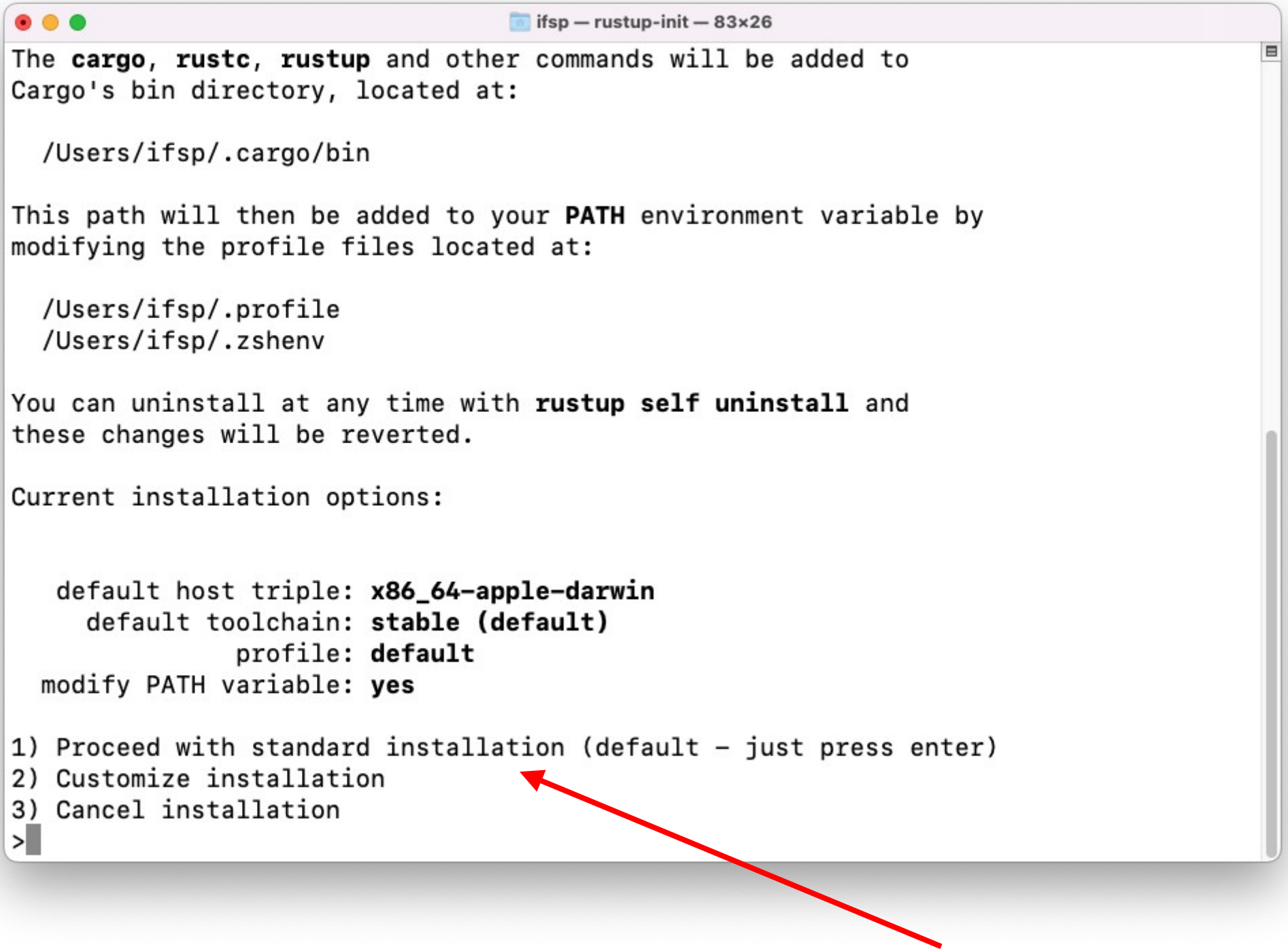


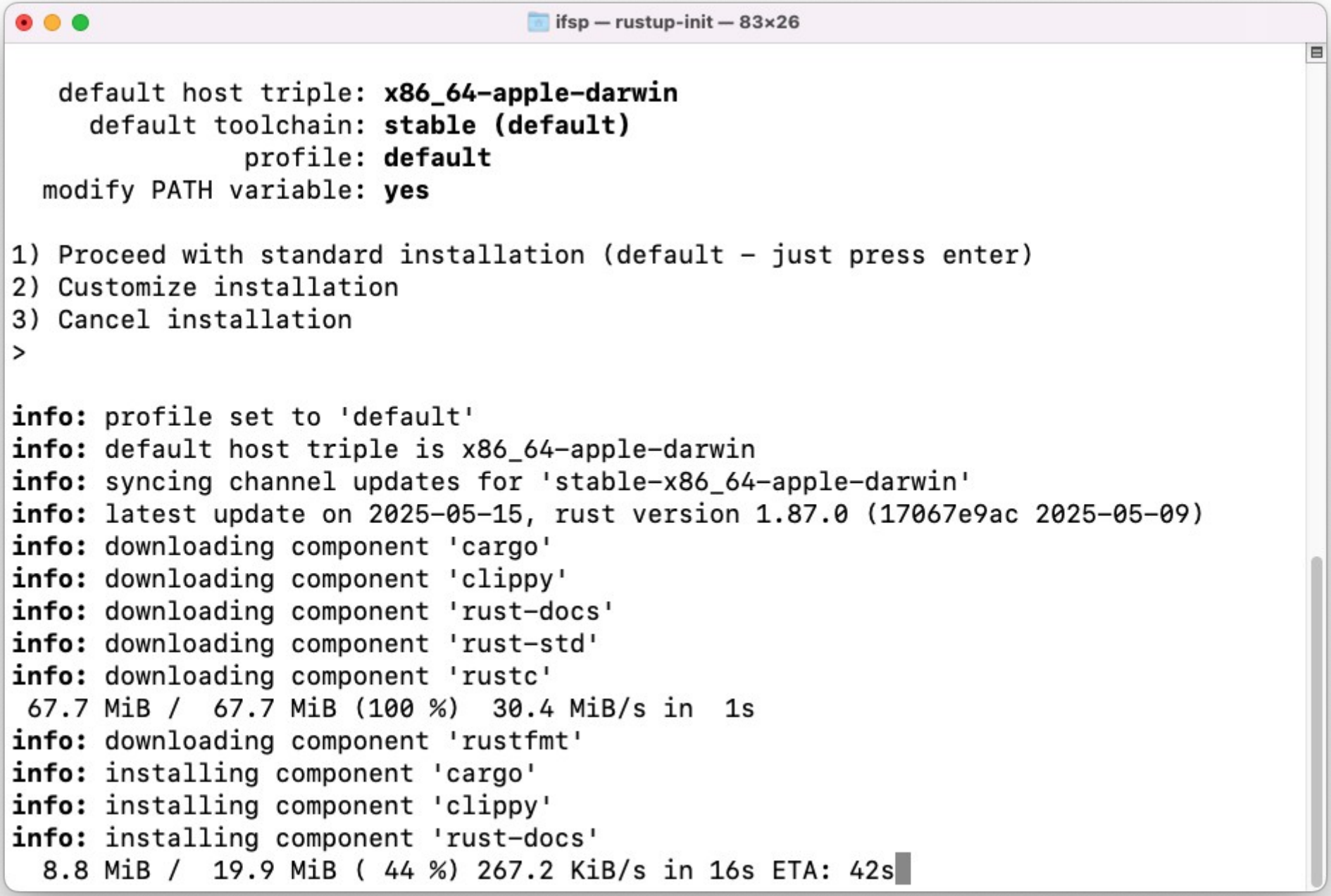

Rust Programming Language - Google Chrome
Rust Programming Lan
rust-lang.org
Install
Learn
Playground
Tools
Governance
Community
Blog
English (en-US)
Rust
A language empowering everyone
to build reliable and efficient software.
GET STARTED
Version 1.87.0
Why Rust?
Performance
Rust is blazingly fast and memory-efficient: with no runtime or garbage collector, it can power performance-critical services, run on embedded devices, and easily integrate with other languages.
Reliability
Rust's rich type system and ownership model guarantee memory-safety and thread-safety — enabling you to eliminate many classes of bugs at compile-time.
Productivity
Rust has great documentation, a friendly compiler with useful error messages, and top-notch tooling — an integrated package manager and build tool, smart multi-editor support with auto-completion and type inspections, an

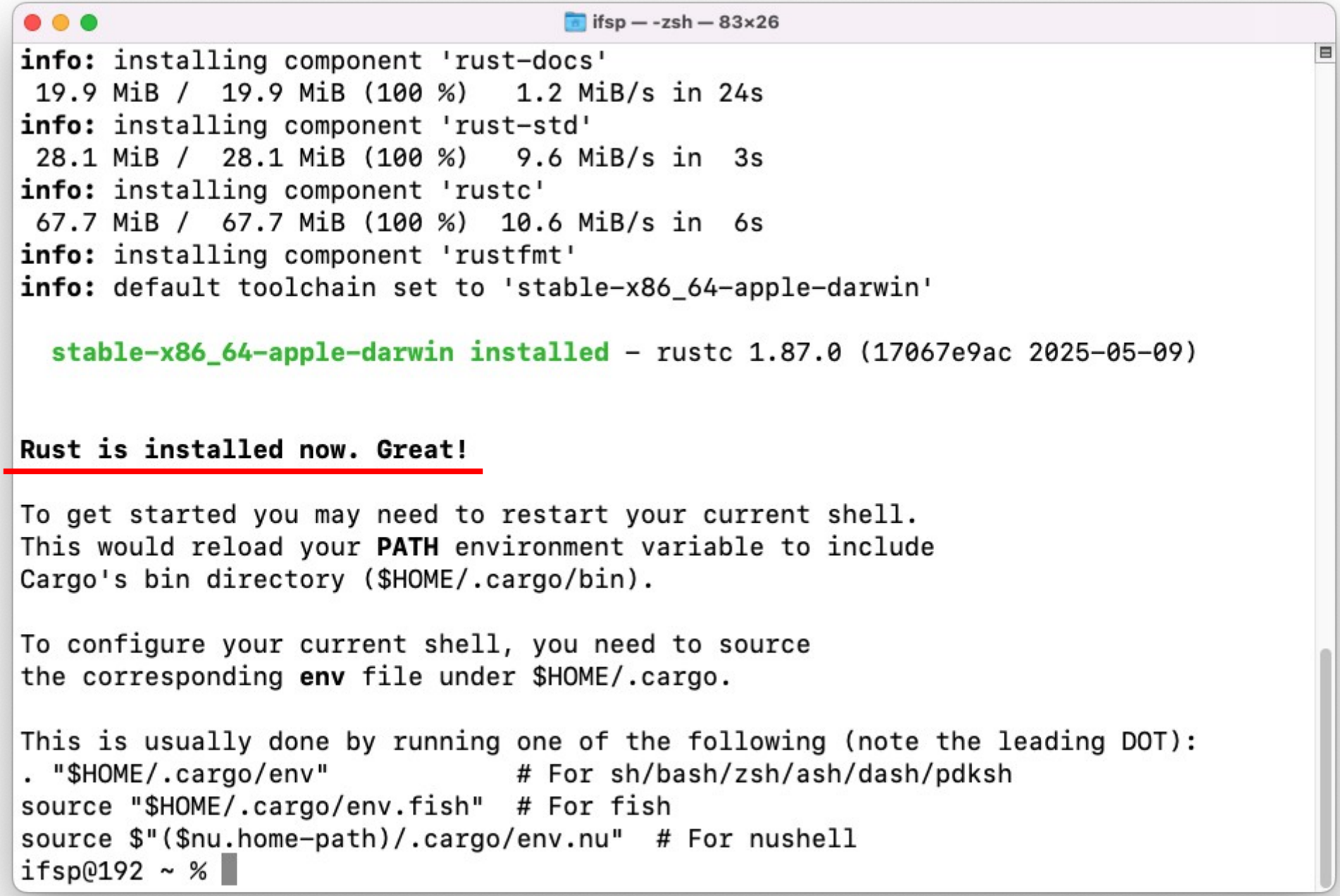

Getting started - Rust Programming Language - Google Chrome
Getting started - Rust P
rust-lang.org/learn/get-started
Rustup: the Rust installer and version management tool
The primary way that folks install Rust is through a tool called Rustup, which is a Rust installer and version management tool.
It looks like you're running macOS, Linux, or another Unix-like OS. To download Rustup and install Rust, run the following in your terminal, then follow the on-screen instructions. See "Other Installation Methods" if you are on Windows.
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
Is Rust up to date?
Rust updates very frequently. If you have installed Rustup some time ago, chances are your Rust version is out of date. Get the latest version of Rust by running rustup update.
LEARN MORE ABOUT INSTALLATION
Cargo: the Rust build tool and package manager
When you install Rustup you'll also get the latest stable version of the Rust build tool and package manager, also known as Cargo. Cargo does lots of things:
build your project with cargo build
run your project with cargo run
test your project with cargo test

LOGIA

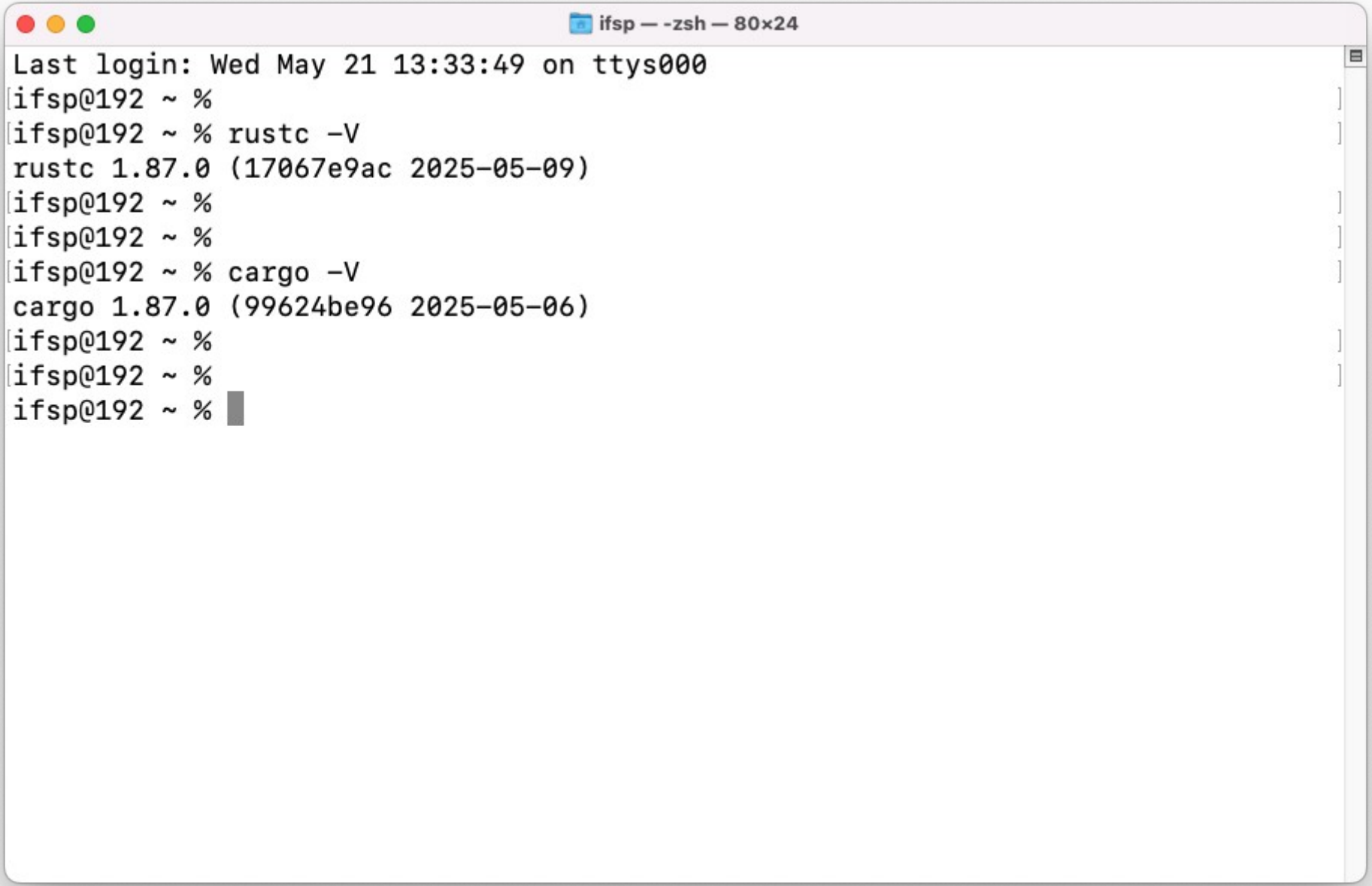

ifsp — -zsh — 83×24
Last login: Wed May 21 13:33:06 on ttys000
ifsp@192 ~ % curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh


ifsp — rustup-init — 83×26
The cargo, rustc, rustup and other commands will be added to
Cargo's bin directory, located at:
/Users/ifsp/.cargo/bin
This path will then be added to your PATH environment variable by
modifying the profile files located at:
/Users/ifsp/.profile
/Users/ifsp/.zshenv
You can uninstall at any time with rustup self uninstall and
these changes will be reverted.
Current installation options:
default host triple: x86_64-apple-darwin
default toolchain: stable (default)
profile: default
modify PATH variable: yes
1) Proceed with standard installation (default - just press enter)
2) Customize installation
3) Cancel installation
>


ifsp — rustup-init — 83×26
default host triple: x86_64-apple-darwin
default toolchain: stable (default)
profile: default
modify PATH variable: yes
1) Proceed with standard installation (default - just press enter)
2) Customize installation
3) Cancel installation
>
info: profile set to 'default'
info: default host triple is x86_64-apple-darwin
info: syncing channel updates for 'stable-x86_64-apple-darwin'
info: latest update on 2025-05-15, rust version 1.87.0 (17067e9ac 2025-05-09)
info: downloading component 'cargo'
info: downloading component 'clippy'
info: downloading component 'rust-docs'
info: downloading component 'rust-std'
info: downloading component 'rustc'
67.7 MiB / 67.7 MiB (100 %) 30.4 MiB/s in 1s
info: downloading component 'rustfmt'
info: installing component 'cargo'
info: installing component 'clippy'
info: installing component 'rust-docs'
8.8 MiB / 19.9 MiB (44 %) 267.2 KiB/s in 16s ETA: 42s


ifsp — -zsh — 83×26
info: installing component 'rust-docs'
19.9 MiB / 19.9 MiB (100 %) 1.2 MiB/s in 24s
info: installing component 'rust-std'
28.1 MiB / 28.1 MiB (100 %) 9.6 MiB/s in 3s
info: installing component 'rustc'
67.7 MiB / 67.7 MiB (100 %) 10.6 MiB/s in 6s
info: installing component 'rustfmt'
info: default toolchain set to 'stable-x86_64-apple-darwin'
stable-x86_64-apple-darwin installed - rustc 1.87.0 (17067e9ac 2025-05-09)
Rust is installed now. Great!
To get started you may need to restart your current shell.
This would reload your PATH environment variable to include
Cargo's bin directory ($HOME/.cargo/bin).
To configure your current shell, you need to source
the corresponding env file under $HOME/.cargo.
This is usually done by running one of the following (note the leading DOT):
. "$HOME/.cargo/env" # For sh/bash/zsh/ash/dash/pdksh
source "$HOME/.cargo/env.fish" # For fish
source $"($nu.home-path)/.cargo/env.nu" # For nushell
ifsp@192 ~ %

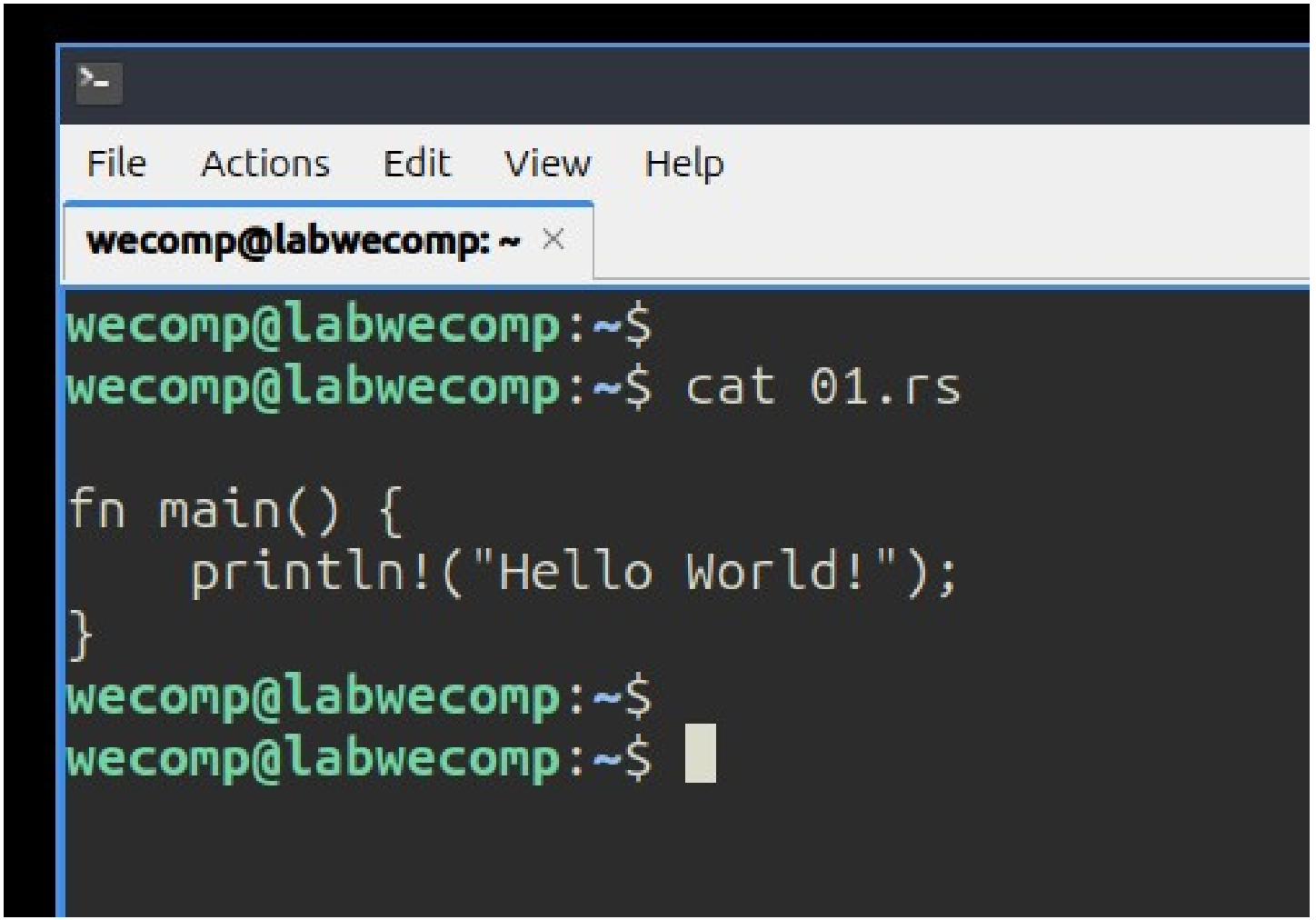
Fechei e reabri a janela do terminal...


ifsp — -zsh — 80×24
Last login: Wed May 21 13:33:49 on ttys000
ifsp@192 ~ %
ifsp@192 ~ % rustc -V
rustc 1.87.0 (17067e9ac 2025-05-09)
ifsp@192 ~ %
ifsp@192 ~ %
ifsp@192 ~ % cargo -V
cargo 1.87.0 (99624be96 2025-05-06)
ifsp@192 ~ %
ifsp@192 ~ %
ifsp@192 ~ %

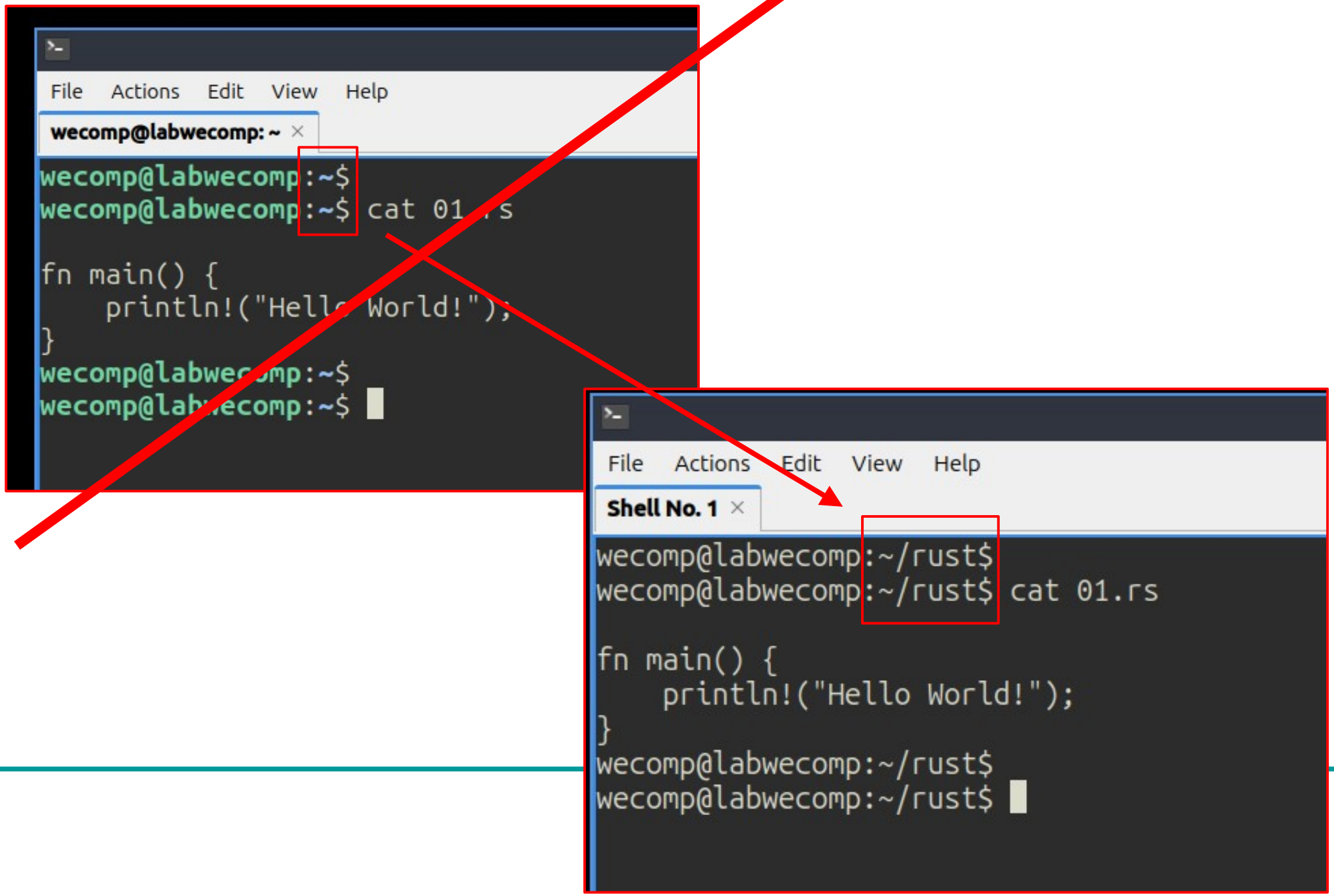
Algumas dicas para manipulação
dos arquivos de códigos...

Lembrando,
usaremos apenas o terminal de comandos.

Visualizar conteúdo de arquivo no terminal: cat



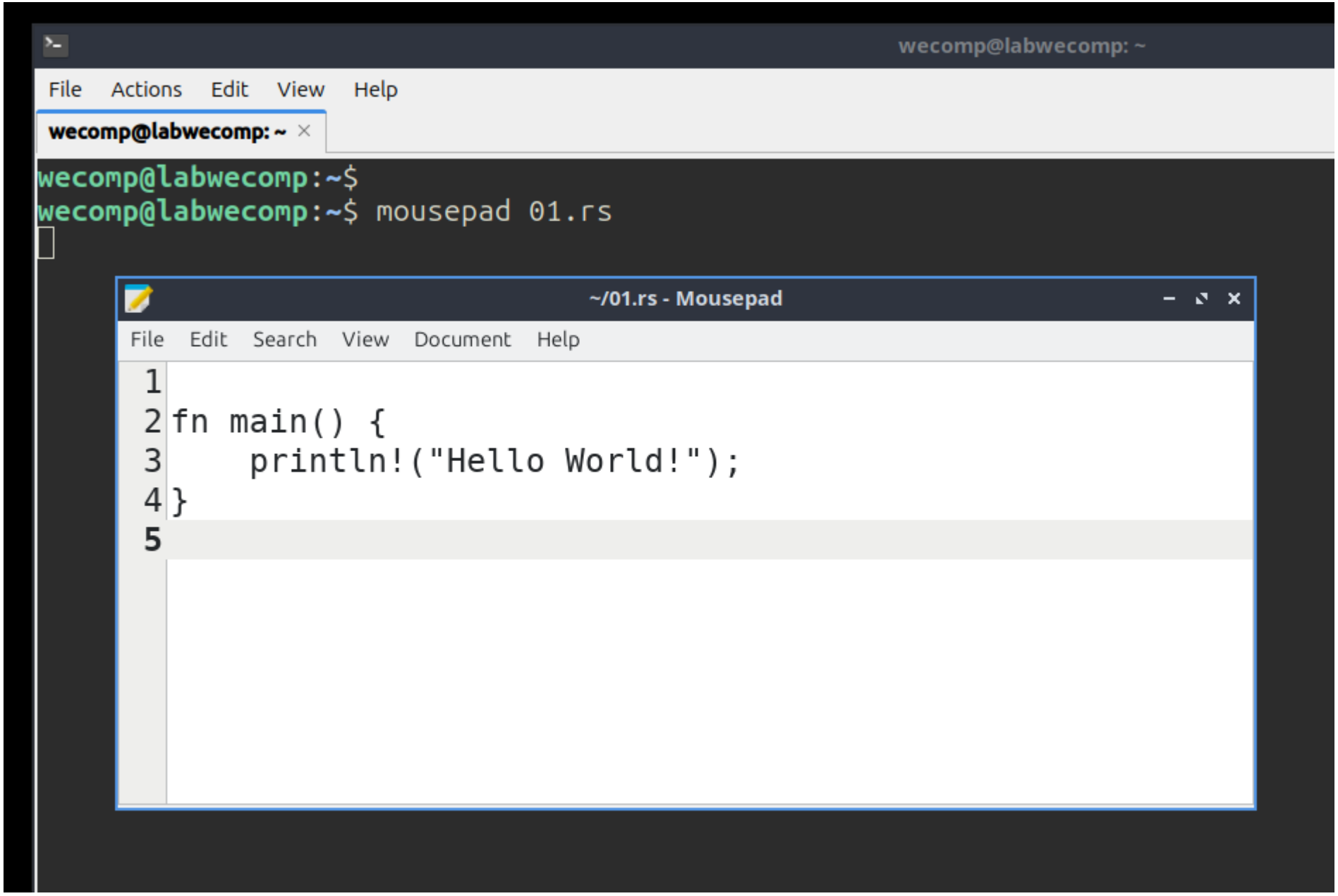
Lembrando.... pasta rust !!!!



Visualizar conteúdo de arquivo com editor “mousepad”

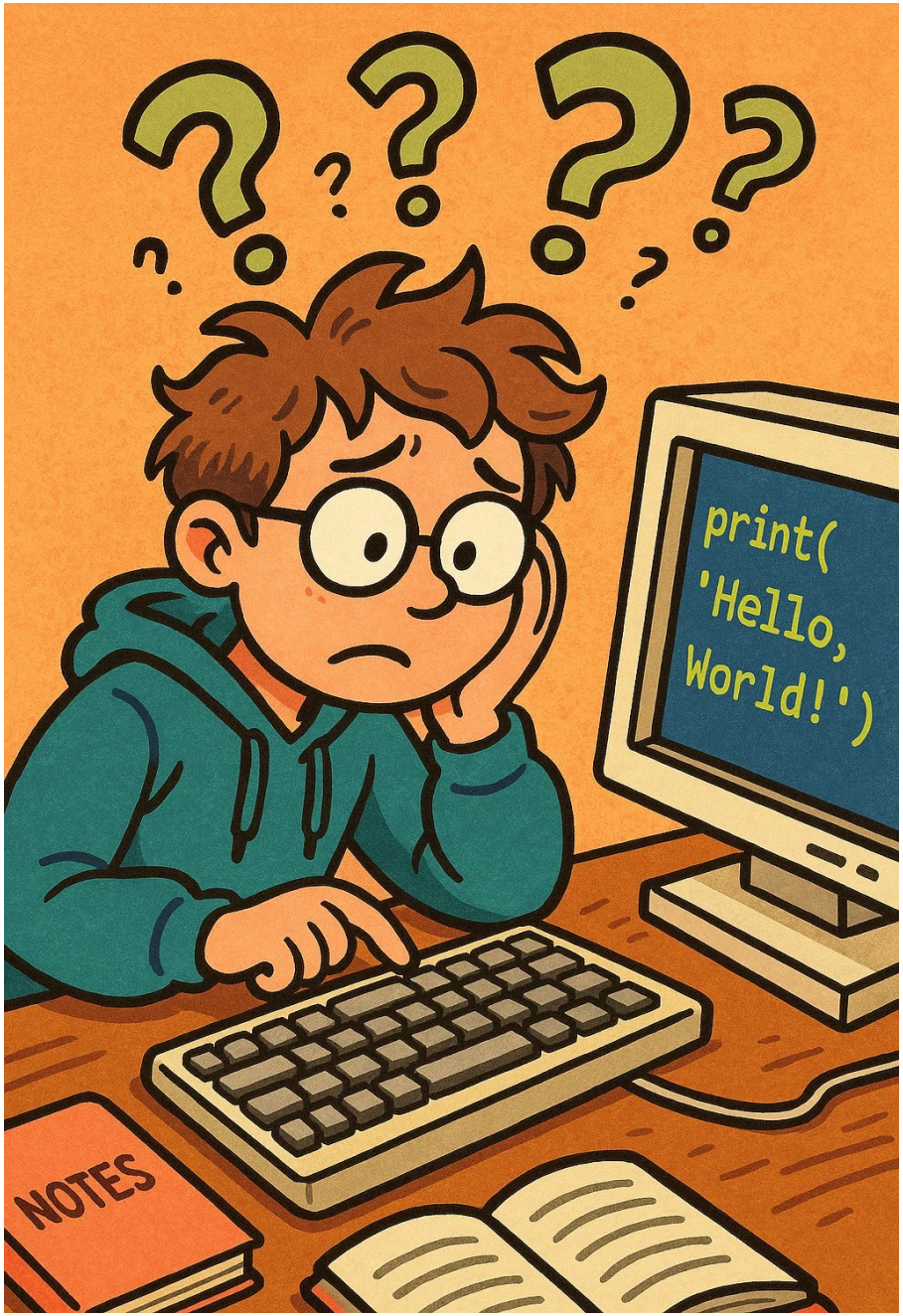
Pode abrir o editor, file|open, blablabla...

Ou abrir direto do terminal o arquivo desejado.



Para limpar (apagar)
a tela do terminal,
tecle
control + L

Hello World !



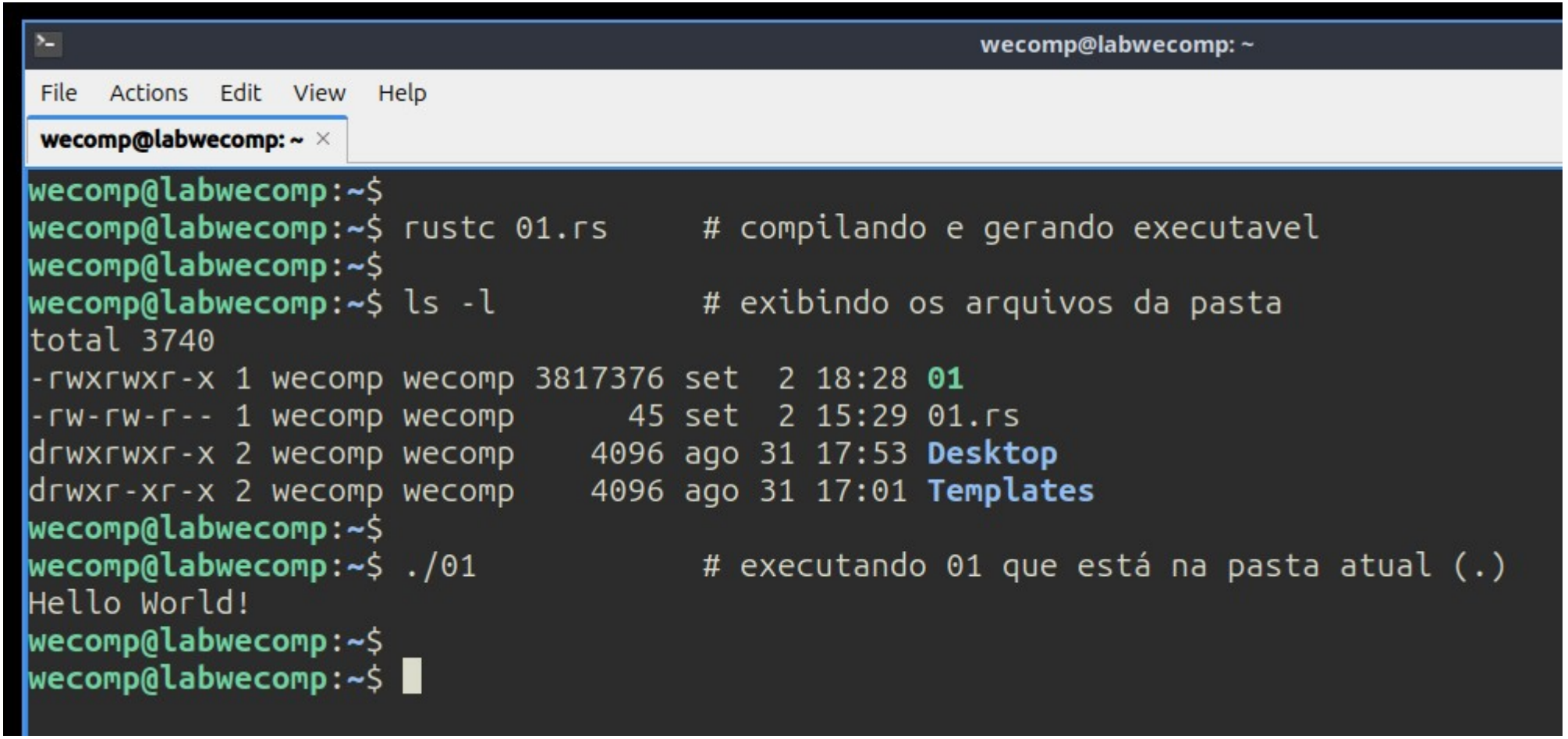
Arquivo 01.rs

```rust

fn main() {
    println!("Hello World!");
}
```

- `fn main()`
  - Define uma função chamada `main`.
  - Em Rust, o ponto de entrada do programa **sempre** é a função `main`.
  - É a primeira função chamada quando o programa inicia.
- `{ ... }`
  - As chaves delimitam o **bloco de código** da função.
  - Tudo que estiver dentro delas será executado quando a função for chamada.
- `println!("Hello World!");`
  - `println!` é uma **macro** do Rust (identificada pelo `!`).
  - Ela imprime texto no console, seguido de uma **quebra de linha**.
  - `"Hello World!"` é uma **string literal** que está sendo passada como argumento.
  - O ponto e vírgula (`;`) indica o fim da instrução.

Compilação



OBS: No windows seria “01.exe”

Variáveis

- Criação com **`let`**
- Tipos:
  - i8, i16, ...
  - u8, u16, ...
  - f32, f64
  - bool, char, &str, ()

Tipos de variáveis

🔢 Tipos inteiros

Inteiros podem ser **com sinal** (aceitam negativos e positivos) ou **sem sinal** (apenas positivos).

Com sinal (signed)

- `i8` → 8 bits → intervalo de **-128 a 127**
- `i16` → 16 bits → intervalo de **-32.768 a 32.767**
- `i32` → 32 bits → intervalo de **-2.147.483.648 a 2.147.483.647**
- `i64` → 64 bits → intervalo de **-9.223.372.036.854.775.808 a 9.223.372.036.854.775.807**
- `i128` → 128 bits → intervalo muito maior, raramente usado.
- `isize` → tamanho depende da arquitetura:
  - 32 bits em sistemas 32-bit
  - 64 bits em sistemas 64-bit

Sem sinal (unsigned)

- `u8` → 8 bits → **0 a 255**
- `u16` → 16 bits → **0 a 65.535**
- `u32` → 32 bits → **0 a 4.294.967.295**
- `u64` → 64 bits → **0 a 18 quintilhões e pouco**
- `u128` → 128 bits → ainda maior.
- `usize` → mesmo tamanho que `isize`, mas só aceita valores positivos (muito usado para índices em

🌊 Tipos de ponto flutuante

- `f32` → 32 bits (precisão simples, similar ao `float` em C/Java).
- `f64` → 64 bits (precisão dupla, similar ao `double`).

> Por padrão, Rust usa `f64` quando você escreve algo como `let x = 3.14;`.

std
1.87.0
(17067e9ac 2025-05-09)

All Items

Sections

The Rust Standard Libr...
How to read this docu...
What is in the standar...
Contributing changes t...
A Tour of The Rust Stan...
Containers and colle...
Platform abstraction...
Use before and after m...

Crate Items

Primitive Types
Modules
Macros
Keywords

Crates

alloc
core
proc_macro
std

Primitive Types

array	A fixed-size array, denoted `[T; N]`, for the element type, `T`, and the non-negative compile-time constant size, `N`.
bool	The boolean type.
char	A character type.
f32	A 32-bit floating-point type (specifically, the "binary32" type defined in IEEE 754-2008).
f64	A 64-bit floating-point type (specifically, the "binary64" type defined in IEEE 754-2008).
fn	Function pointers, like `fn(usize) -> bool`.
i8	The 8-bit signed integer type.
i16	The 16-bit signed integer type.
i32	The 32-bit signed integer type.
i64	The 64-bit signed integer type.
i128	The 128-bit signed integer type.
isize	The pointer-sized signed integer type.
pointer	Raw, unsafe pointers, `*const T`, and `*mut T`.
reference	References, `&T` and `&mut T`.
slice	A dynamically-sized view into a contiguous sequence, `[T]`.
str	String slices.
tuple	A finite heterogeneous sequence, `(T, U, ..)`.
u8	The 8-bit unsigned integer type.
u16	The 16-bit unsigned integer type.
u32	The 32-bit unsigned integer type.
u64	The 64-bit unsigned integer type.
u128	The 128-bit unsigned integer type.
unit	The `()` type, also called "unit".
usize	The pointer-sized unsigned integer type.
f16 Experimental	A 16-bit floating-point type (specifically, the "binary16" type defined in IEEE 754-2008).
f128 Experimental	A 128-bit floating-point type (specifically, the "binary128" type defined in IEEE 754-2008).
never Experimental	The `!` type, also called "never".

Arquivo 02.rs

```
1
2 fn main() {
3     let variavel:i8 = 128;
4     println!("variavel = {}", variavel);
5 }
6
```

- `fn main() { ... }`
  Define a função principal, ponto de entrada do programa.
- `let variavel:i8 = 128;`
  - `let` declara uma variável.
  - `variavel` é o nome dado a ela.
  - `:i8` indica explicitamente o **tipo**: `i8` é um inteiro **com sinal** de 8 bits.
    - Ele pode armazenar valores de **-128 até +127**.
  - O valor `128` foi atribuído.
- `println!("variavel = {}", variavel);`
  - Usa a macro `println!` para imprimir no console.
  - `"variavel = {}"` é uma string de formatação.
  - O `{}` será substituído pelo valor de `variavel`.

Arquivo 02.rs

```
1
2 fn main() {
3     let variavel:i8 = 128;
4     println!("variavel = {}", variavel);
5 }
6
```

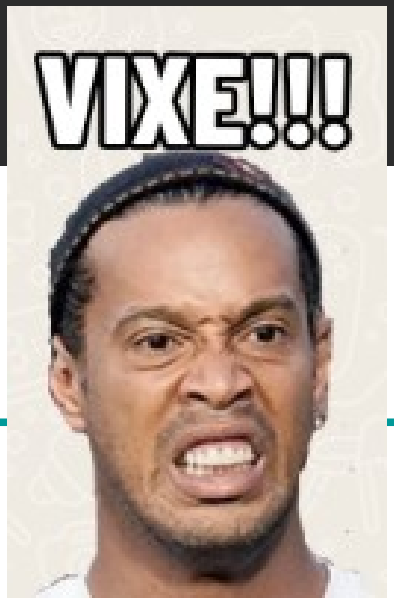
- `fn main() { ... }`
  Define a função principal, ponto de entrada do programa.
- `let variavel:i8 = 128;`
  - `let` declara uma variável.
  - `variavel` é o nome dado a ela.
  - `:i8` indica explicitamente o **tipo**: `i8` é um inteiro **com sinal** de 8 bits.
    - Ele pode armazenar valores de **-128 até +127**.
  - O valor `128` foi atribuído.
- `println!("variavel = {}", variave`
  - Usa a macro `println!` para impri
  - `"variavel = {}"` é uma string de
  - O `{}` será substituído pelo valor d

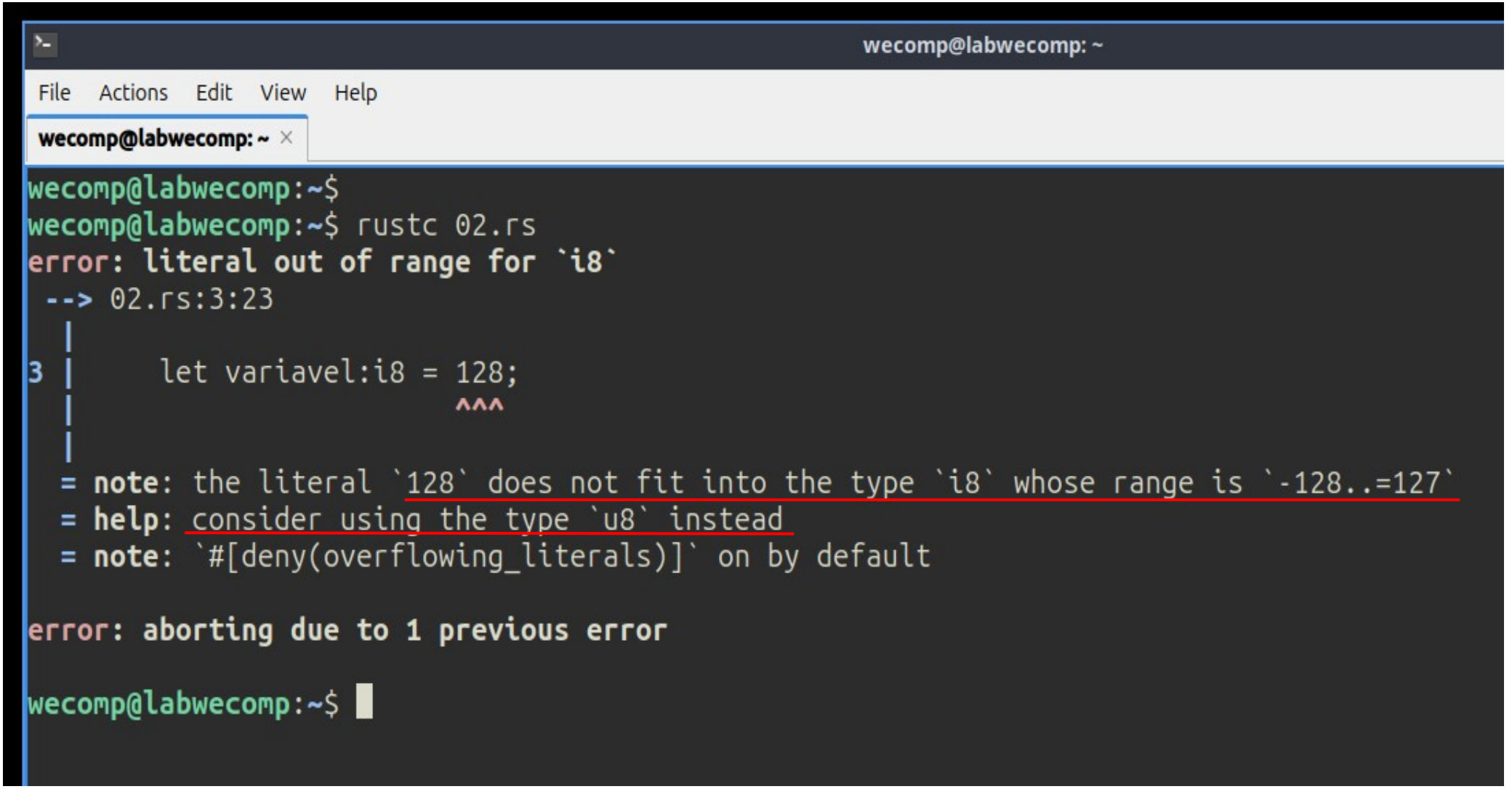


```
wecomp@labwecomp: ~
File  Actions  Edit  View  Help
wecomp@labwecomp: ~ ×
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 02.rs
error: literal out of range for `i8`
 --> 02.rs:3:23
  |
3 |     let variavel:i8 = 128;
  |                       ^^^
  |
  = note: the literal `128` does not fit into the type `i8` whose range is `-128..=127`
  = help: consider using the type `u8` instead
  = note: `#[deny(overflowing_literals)]` on by default

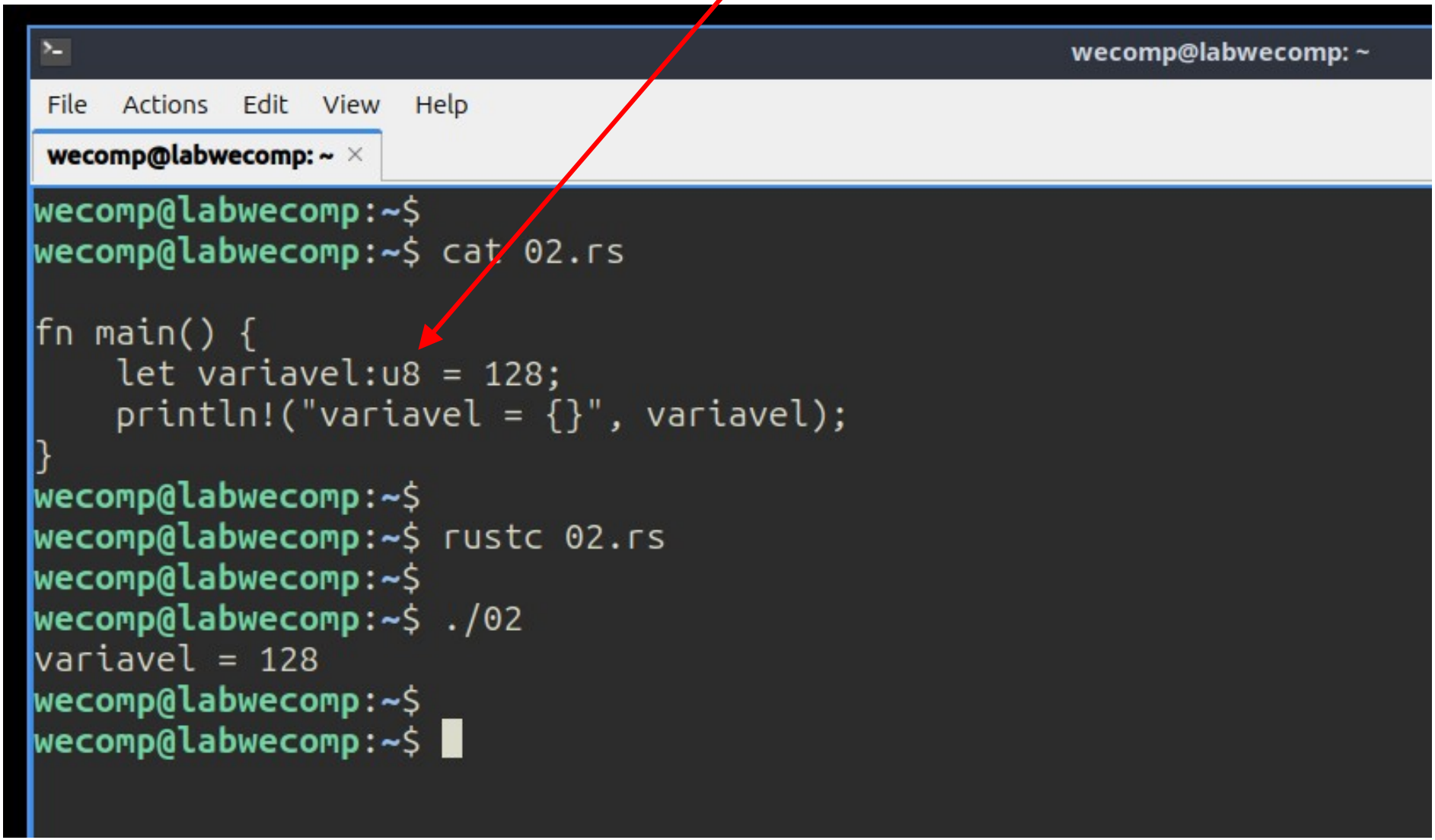
error: aborting due to 1 previous error

wecomp@labwecomp:~$
```




wecomp@labwecomp: ~
File Actions Edit View Help
wecomp@labwecomp: ~
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 02.rs
error: literal out of range for `i8`
--> 02.rs:3:23
3 | let variavel:i8 = 128;
^^^
= note: the literal `128` does not fit into the type `i8` whose range is `-128..=127`
= help: consider using the type `u8` instead
= note: `#[deny(overflowing_literals)]` on by default
error: aborting due to 1 previous error
wecomp@labwecomp:~$

Altere para u8, e recompile.


wecomp@labwecomp: ~
File Actions Edit View Help
wecomp@labwecomp: ~
wecomp@labwecomp:~$
wecomp@labwecomp:~$ cat 02.rs
fn main() {
let variavel:u8 = 128;
println!("variavel = {}", variavel);
}
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 02.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./02
variavel = 128
wecomp@labwecomp:~$
wecomp@labwecomp:~$

wecomp@labwecomp: ~

File Actions Edit View Help

wecomp@labwecomp: ~

```
wecomp@labwecomp:~$
wecomp@labwecomp:~$ cat 02.rs

fn main() {
    let variavel:u8 = 128;
    println!("variavel = {}", variavel);
}
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 02.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./02
variavel = 128
wecomp@labwecomp:~$
wecomp@labwecomp:~$
```



Se não colocar o tipo,
ele usa o tipo padrão i32
(4 bytes)

Arquivo 03.rs

```rust

fn main() {
    let variavel = 300;
    println!("variavel={}, tamanho={} bytes", variavel, std::mem::size_of_val(&variavel));
}
```

2️⃣ `let variavel = 300;`

- Declara uma variável chamada `variavel`.
- Rust **infere automaticamente o tipo** dela como `i32`, já que o literal `300` cabe nesse tipo.

3️⃣ `println!("variavel={}, tamanho={} bytes", variavel, std::mem::size_of_val(&variavel));`

- `println!` imprime no console.
- `"variavel={}, tamanho={} bytes"` é a string de formatação, com `{}` substituídos pelos valores.
- O primeiro `{}` recebe `variavel` → imprime `300`.
- O segundo `{}` recebe o resultado de `std::mem::size_of_val(&variavel)`.

`&variavel`

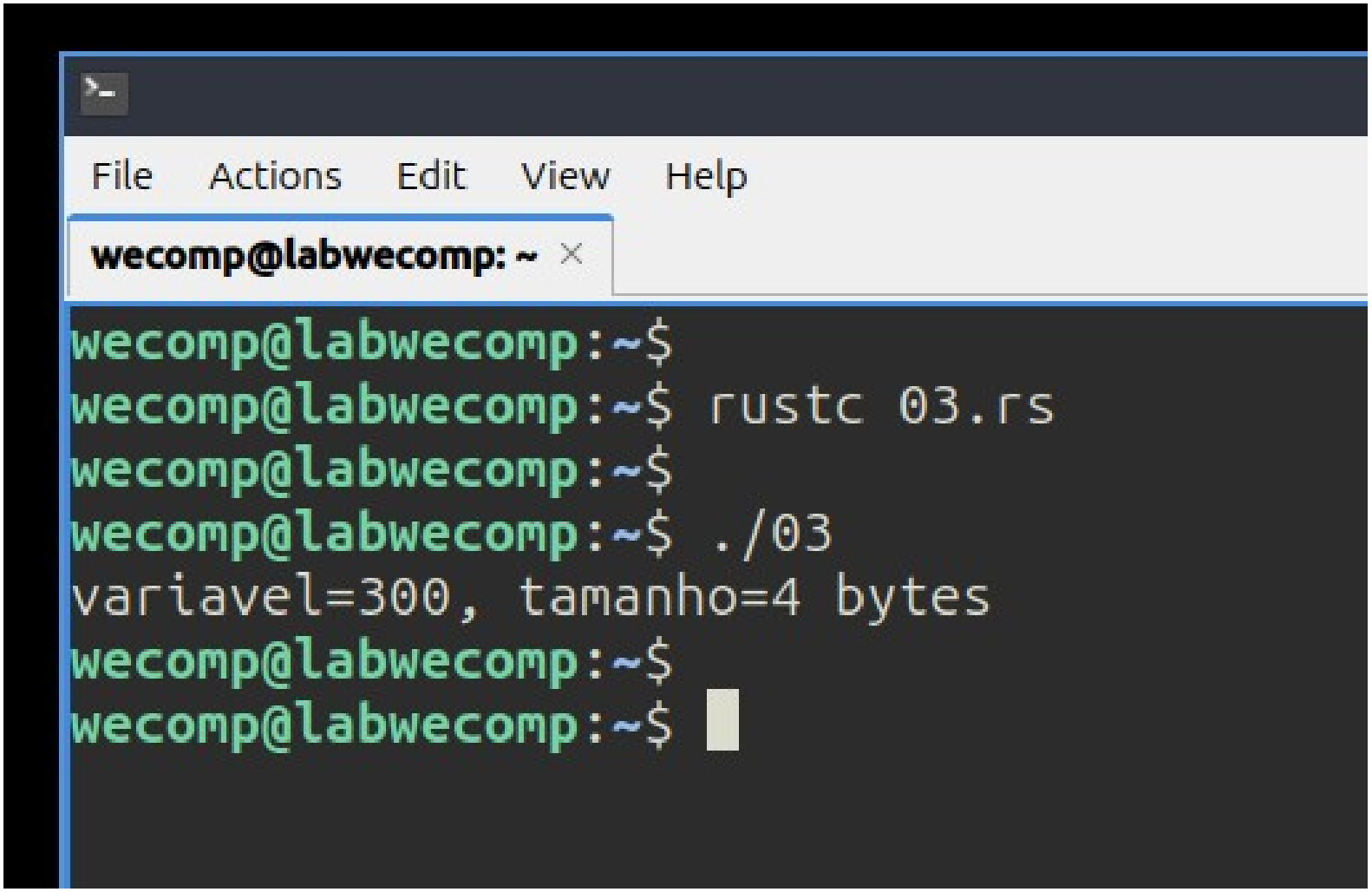
- O `&` pega **o endereço de memória da variável**, como em C.
- Em Rust, `&variavel` cria uma **referência** para `variavel`, sem copiar o valor.

`std::mem::size_of_val(&variavel)`

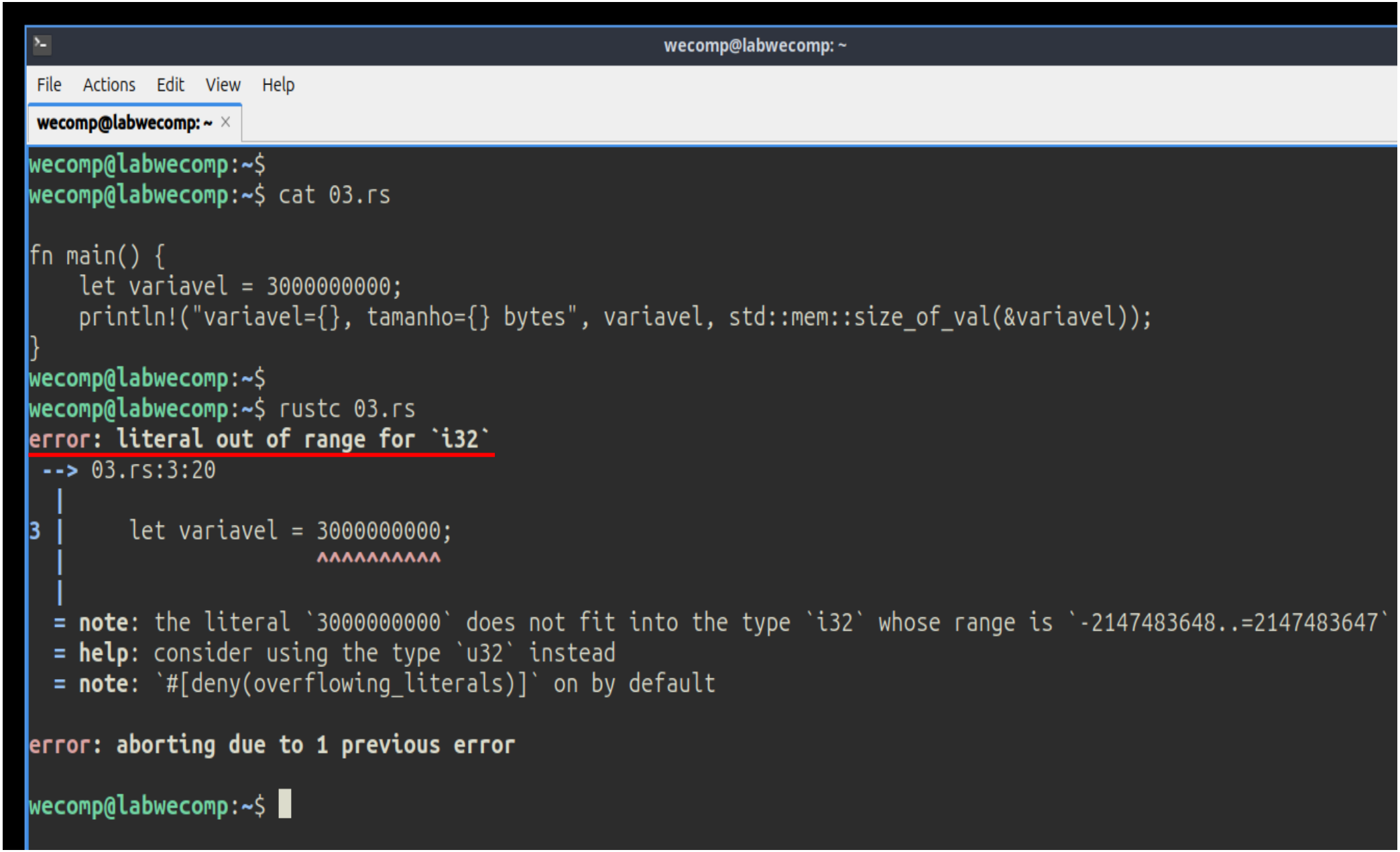
- Retorna **o tamanho em bytes do valor referenciado**.
- Como `variavel` é `i32`, retorna `4` bytes.

🔹 Resumo comparativo

Aspecto	C	Rust
`&variavel`	Endereço de memória	Referência para o valor
Tipo resultante	Ponteiro (`int*` , `char*`)	Referência (`&T`)
Segurança	Baixa (pode gerar dangling pointers)	Alta (verificação em tempo de compilação)
Aritmética	Permitida	Não permitida
Desreferenciar	`*p`	`*r`


File Actions Edit View Help
wecomp@labwecomp: ~
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 03.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./03
variavel=300, tamanho=4 bytes
wecomp@labwecomp:~$
wecomp@labwecomp:~$

Troque o valor atribuído
para um valor que não caiba
em 4 bytes/i32
(p.ex. 3 000 000 000)


wecomp@labwecomp: ~
File Actions Edit View Help
wecomp@labwecomp: ~
wecomp@labwecomp:~$
wecomp@labwecomp:~$ cat 03.rs
fn main() {
let variavel = 3000000000;
println!("variavel={}, tamanho={} bytes", variavel, std::mem::size_of_val(&variavel));
}
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 03.rs
error: literal out of range for `i32`
--> 03.rs:3:20
3 | let variavel = 3000000000;
^^^^^^^^^^
= note: the literal `3000000000` does not fit into the type `i32` whose range is `-2147483648..=2147483647`
= help: consider using the type `u32` instead
= note: `#[deny(overflowing_literals)]` on by default
error: aborting due to 1 previous error
wecomp@labwecomp:~$

Outros tipos:

Arquivo 04.rs

```rust
fn main() {

    let decimal:f32 = 2.5;
    println!("decimal = {}, tamanho={}", decimal, std::mem::size_of_val(&decimal));

    let booleana = false;
    println!("Tamanho booleana = {} bytes", std::mem::size_of_val(&booleana));

    let letra:char = 'C';
    println!("Tamanho do char = {} bytes", std::mem::size_of_val(&letra));
}
```

1️⃣ `let decimal:f32 = 2.5;`

- Declara uma variável `decimal` do tipo `f32` (ponto flutuante de 32 bits).
- Armazena o valor `2.5`.

2️⃣ `println!("decimal = {}, tamanho={}", decimal, std::mem::size_of_val(&decimal));`

- Imprime o valor da variável (`2.5`) e o **tamanho em bytes**.
- `&decimal` passa uma referência (endereço) da variável para a função `size_of_val`.
- `std::mem::size_of_val(&decimal)` retorna **4 bytes**, porque `f32` ocupa 4 bytes na memória.

Arquivo 04.rs

```rust
fn main() {

    let decimal:f32 = 2.5;
    println!("decimal = {}, tamanho={}", decimal, std::mem::size_of_val(&decimal));

    let booleana = false;
    println!("Tamanho booleana = {} bytes", std::mem::size_of_val(&booleana));

    let letra:char = 'C';
    println!("Tamanho do char = {} bytes", std::mem::size_of_val(&letra));
}
```

3 `let booleana = false;`

- Declara uma variável booleana `booleana`.
- Rust infere o tipo como `bool`.

4 `println!("Tamanho booleana = {} bytes", std::mem::size_of_val(&booleana));`

- Retorna o tamanho da variável booleana em bytes.
- `bool` ocupa **1 byte** em Rust.

Arquivo 04.rs

```rust
fn main() {

    let decimal:f32 = 2.5;
    println!("decimal = {}, tamanho={}", decimal, std::mem::size_of_val(&decimal));

    let booleana = false;
    println!("Tamanho booleana = {} bytes", std::mem::size_of_val(&booleana));

    let letra:char = 'C';
    println!("Tamanho do char = {} bytes", std::mem::size_of_val(&letra));
}
```

5 `let letra:char = 'C';`

- Declara uma variável `letra` do tipo `char` (caractere Unicode de 4 bytes).

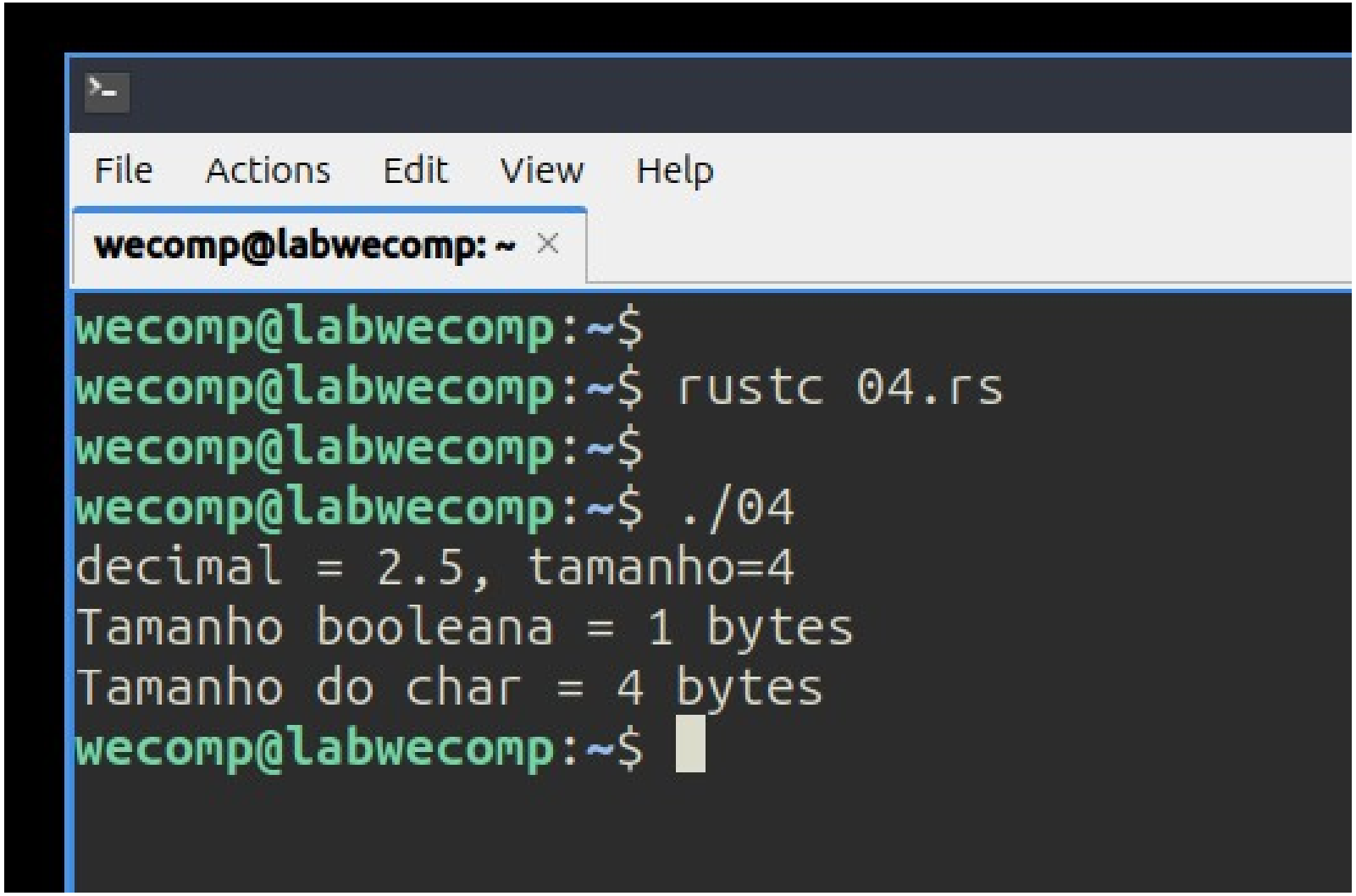
6 `println!("Tamanho do char = {} bytes", std::mem::size_of_val(&letra));`

- Retorna o tamanho do `char`, que é **4 bytes** em Rust.
- Diferente de C/Java, `char` em Rust sempre ocupa 4 bytes, para suportar **Unicode completo**.

“Unicode completo” significa que o tipo `char` em Rust consegue representar **todos os caracteres possíveis definidos pelo padrão Unicode**, não apenas os do ASCII ou BMP (Basic Multilingual Plane).

Detalhes:

1. **Unicode**
   - É um padrão que atribui um número único (código) para **praticamente todos os símbolos usados em qualquer língua**, além de emojis, símbolos matemáticos, setas, etc.
   - Cada caractere Unicode tem um **code point**, como `U+0041` para `'A'` ou `U+1F600` para 😀.
2. **Rust** `char`
   - Em Rust, `char` ocupa **4 bytes (32 bits)**.
   - Isso permite armazenar qualquer **code point Unicode** de `U+0000` até `U+10FFFF`.
   - Diferente de C (`char` = 1 byte) ou Java (`char` = 2 bytes, UTF-16), que não conseguem representar todos os caracteres Unicode sem usar sequências especiais (surrogate pairs no caso do Java).


File Actions Edit View Help
wecomp@labwecomp: ~
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 04.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./04
decimal = 2.5, tamanho=4
Tamanho booleana = 1 bytes
Tamanho do char = 4 bytes
wecomp@labwecomp:~$

Brincando com char...

Arquivo 05.rs

```rust
fn main() {

    let sorrindo: char          = '\u{1F60A}';
    let olhando_pra_cima: char = '\u{1F644}';
    let assustado: char         = '\u{1F631}';
    let chorando: char          = '\u{1F622}';

    println!("Sorrindo: {}", sorrindo);
    println!("Olhando pra cima: {}", olhando_pra_cima);
    println!("Assustado: {}", assustado);
    println!("Chorando: {}", chorando);
}

```

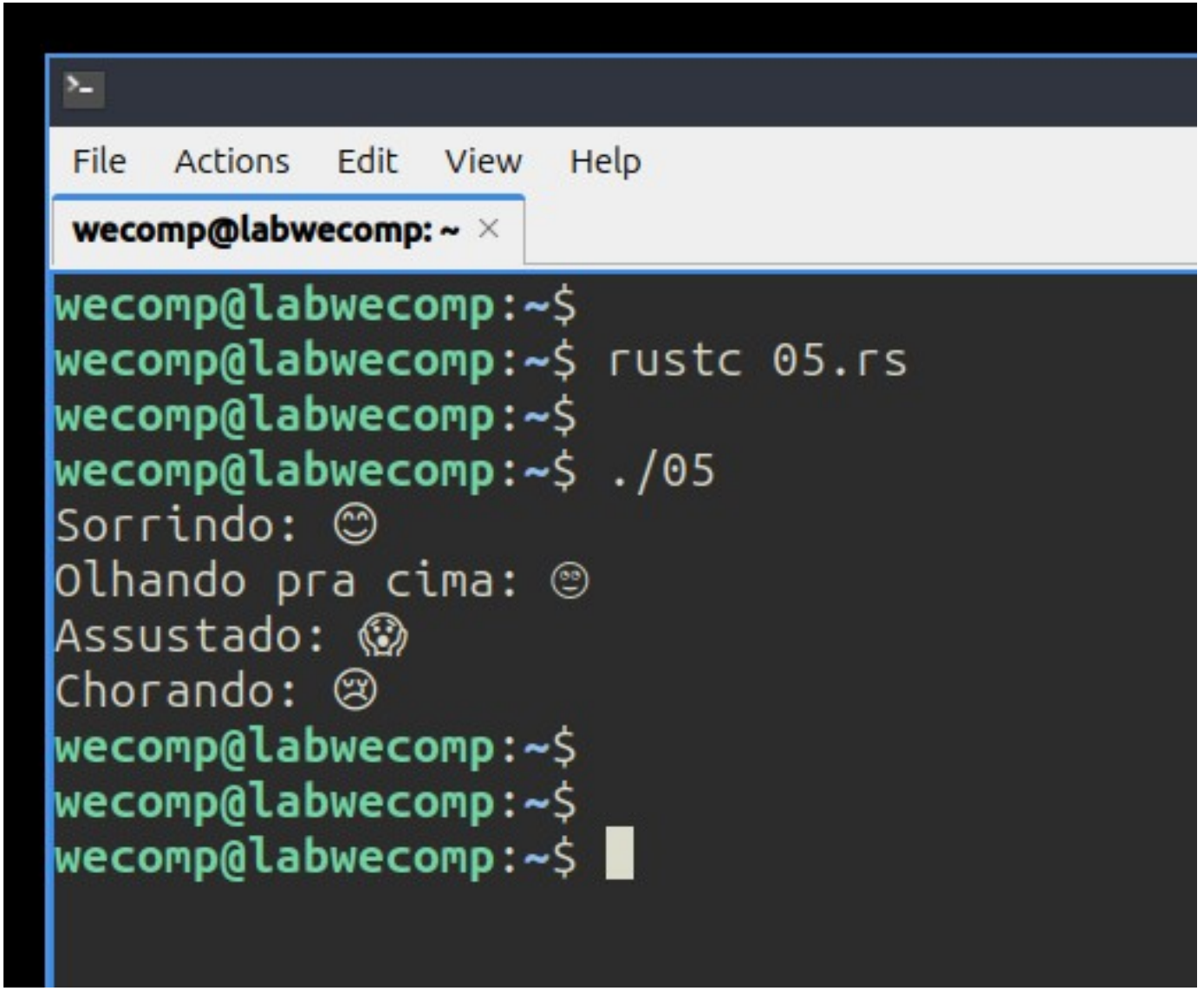
```rust
let sorrindo: char = '\u{1F60A}';
```

- Declara a variável `sorrindo` do tipo `char`.
- O valor `'\u{1F60A}'` é um **literal Unicode** usando a notação `\u{...}`.
  - Dentro das chaves vai o **code point Unicode** em hexadecimal.
  - `1F60A` corresponde ao emoji 😀 **sorrindo com olhos fechados**.

O mesmo vale para as outras variáveis:

Variável	Code point	Emoji correspondente
`olhando_pra_cima`	1F644	🙄
`assustado`	1F631	😱
`chorando`	1F622	😢

- Todas são do tipo `char` em Rust, que suporta **Unicode completo**, inclusive emojis fora do BMP.


File Actions Edit View Help
wecomp@labwecomp: ~
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 05.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./05
Sorrindo:
Olhando pra cima:
Assustado:
Chorando:
wecomp@labwecomp:~$
wecomp@labwecomp:~$
wecomp@labwecomp:~$

Rodando no terminal do meu Ubuntu 25.04...



(i)Mutabilidade

imutabilidade

- Por padrão, variáveis em RUST são imutáveis.
- Não confunda com constante
- A variável pode ter seu valor atribuído na primeira vez, mas depois não pode mais ter seu valor alterado.
- Se quiser poder mudar o valor atribuído, precisa usar a palavra chave “mut” na definição da variável.

Arquivo 06.rs

```
 1
 2 fn main() {
 3
 4     let variavel = 100;
 5
 6     println!("variavel = {}", variavel);
 7
 8     variavel = 150;
 9
10     println!("variavel = {}", variavel);
11
12 }
```

Arquivo 06.rs

```
1
2 fn main() {
3
4     let variavel = 100;
5
6     println!("variavel = {}", variavel);
7
8     variavel = 150;
9
10    println!("var
11
12 }
```

```
wecomp@labwecomp: ~
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 06.rs
error[E0384]: cannot assign twice to immutable variable `variavel`
 --> 06.rs:8:5
  |
4 |     let variavel = 100;
  |         -------- first assignment to `variavel`
...
8 |     variavel = 150;
  |     ^^^^^^^^^^^^^^ cannot assign twice to immutable variable
  |
help: consider making this binding mutable
  |
4 |     let mut variavel = 100;
  |         +++

error: aborting due to 1 previous error

For more information about this error, try `rustc --explain E0384`.
wecomp@labwecomp:~$
```

Defina a variável como mutable (mut):

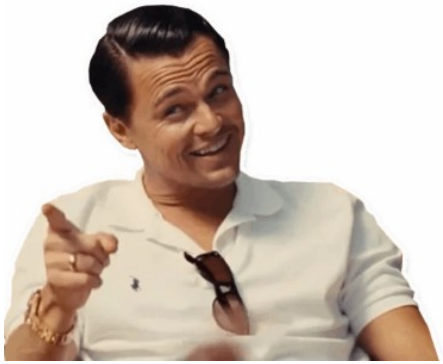
```
1
2 fn main() {
3
4     let mut variavel = 100;
5
6     println!("variavel = {}", variavel);
7
8     variavel = 150;
9
10    println!("variavel = {}", variavel);
11
12 }
```

Defina a variável como mutable (mut):

```
1
2  fn main() {
3
4      let mut variavel = 100;
5
6      println!("variavel = {}", variavel);
7
8      variavel = 150;
9
10     println!("variavel =
11
12 }
```



Defina a variável como mutable (mut):



```
1
2  fn main() {
3
4      let mut variavel = 100;
5
6      println!("variavel = {}", variavel);
7
8      variavel = 150;
9
10     println!("variavel =
11
12 }
```

```
wecomp@labwecomp:~
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 06.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./06
variavel = 100
variavel = 150
wecomp@labwecomp:~$
```

Variáveis locais, globais, static, unsafe

Resumo – `let`, `const` e `static` em Rust

- `let`
  - Cria variáveis locais (escopo limitado).
  - Imutável por padrão, pode ser `mut`.
  - Vive apenas enquanto o escopo existir.
- `const`
  - Valor fixo, substituído em tempo de compilação.
  - Pode ser declarado no escopo global ou local.
  - Sempre imutável e sem endereço em memória.
- `static`
  - Variável global única, alocada em memória fixa.
  - Vive durante toda a execução do programa.
  - Imutável por padrão; `static mut` existe, mas exige `unsafe`.

1. Variável normal (local)

```rust
fn main() {
    let x = 10;          // imutável por padrão
    let mut y = 20;      // mutável
}
```

- Vivem no **stack** (ou heap, se for algo grande, mas controlado pelo escopo).
- Só existem enquanto o escopo/função está rodando.
- Nunca são globais por si só.

`const`

- Valor imutável, resolvido em *compile time*.
- Não ocupa memória de variável em tempo de execução, vira literalmente uma substituição de texto pelo compilador.

2. `const`

```rust
const PI: f64 = 3.1415;
```

- **Substituição em tempo de compilação**: não há endereço fixo em memória para ele.
- É literalmente "inline" em todos os pontos do código.
- Pode ser declarado no **nível global** ou dentro de funções:

```rust
const GLOBAL: i32 = 1;

fn main() {
    const LOCAL: i32 = 2;
}
```

3. `static`

rust Copiar código

```rust
static MENSAGEM: &str = "Olá!";
```

- **Mora em memória fixa** durante toda a execução do programa (na *data section* do binário).
- É **global** por natureza.
- Ao contrário de `const`, tem **um endereço único** em memória.

👉 Isso significa que **não faz sentido** declarar `static` dentro de uma função e esperar que seja "local", porque o compilador ainda a tratará como **global única**. Você até pode escrever:

rust Copiar código

```rust
fn main() {
    static X: i32 = 42;
    println!("{}", X);
}
```

Mas ela não é "local" de verdade — continua sendo única e global. A diferença é só de escopo de visibilidade do nome, não de tempo de vida.

07.rs

```rust

// Constante global
// (substituída em tempo de compilação, não ocupa memória real)

const PI: f64 = 3.1415;


// Static global
// (tem endereço fixo em memória, imutável por padrão)

static VALOR_FIXO: i32 = 100;


// Static mut global
// (mutável, mas acesso exige unsafe)

static mut CONTADOR: i32 = 0;


fn main() {
```

```
1
2  // Constante global
3  // (substituída em tempo de compilação, não ocupa memória real)
4
5  const PI: f64 = 3.1415;
6
7
8  // Static global
9  // (tem endereço fixo em memória, imutável por padrão)
10
11 static VALOR_FIXO: i32 = 100;
12
13
14 // Static mut global
15 // (mutável, mas acesso exige
16
17 static mut CONTADOR: i32 = 0;
18
19
20 fn main() {
```

```
20 fn main() {
21
22     // ---------------- let ----------------
23     let x = 10;         // variável local, imutável por padrão
24     let mut y = 20;     // variável local mutável
25
26     println!("let x = {}", x);
27     println!("let mut y = {}", y);
28     y += 1;
29     println!("y após incremento = {}", y);
30
31     // ---------------- const ----------------
32     println!("const PI = {}", PI);
33
34     // ---------------- static ----------------
35     println!("static VALOR_FIXO = {}", VALOR_FIXO);
36
37     // ---------------- static mut (unsafe) ----------------
38     CONTADOR += 1;
39     println!("static mut CONTADOR = {}", CONTADOR);
40
41 }
```

```
wecomp@labwecomp: ~ X
wecomp@labwecomp:~$ rustc 07.rs
error[E0133]: use of mutable static is unsafe and requires unsafe function or block
  --> 07.rs:38:5
   |
38 |     CONTADOR += 1;
   |     ^^^^^^^^ use of mutable static
   |
   = note: mutable statics can be mutated by multiple threads: aliasing violations or data races will cause undefi
ned behavior

error[E0133]: use of mutable static is unsafe and requires unsafe function or block
  --> 07.rs:39:42
   |
39 |     println!("static mut CONTADOR = {}", CONTADOR);
   |                                          ^^^^^^^^ use of mutable static
   |
   = note: mutable statics can be mutated by multiple threads: aliasing violations or data races will cause undefi
ned behavior

error: aborting due to 2 previous errors

For more information about this error, try `rustc --explain E0133`.
wecomp@labwecomp:~$
```

```
wecomp@labwecomp:~$ rustc 07.rs
error[E0133]: use of mutable static is unsafe and requires unsafe function or block
  --> 07.rs:38:5
   |
38 |     CONTADOR += 1;
   |     ^^^^^^^^ use of mutable static
   |
   = note: mutable statics can be mutated by multiple threads: aliasing violations or data races will cause undefi
ned behavior

error[E0133]: use of mutable static is unsafe and requires unsafe function or block
  --> 07.rs:39:42
   |
39 |     println!("static mut CONTADOR = {}", CONTADOR);
   |                                          ^^^^^^^^ use of mutable static
   |
   = note: mutable statics can be mutated by multiple threads: aliasing violations or data races will cause undefi
ned behavior

error: aborting due

For more informatio
wecomp@labwecomp:~$
```

Em Rust, usar `mut` em uma variável `static` é considerado inseguro porque a variável é global e **pode ser acessada de qualquer thread simultaneamente**. Como o compilador não consegue garantir que duas partes do código não vão tentar ler e escrever nela ao mesmo tempo, podem ocorrer **data races**, ou seja, leituras e escritas simultâneas que corrompem o valor. Por isso, todo acesso a `static mut` exige `unsafe`, sinalizando que o programador é responsável por garantir a segurança. Na prática, isso significa que, sem mecanismos de sincronização como `Mutex` ou tipos atômicos (`Atomic*`), qualquer mutável global pode causar bugs difíceis de detectar em programas concorrentes.

Se você garante que tudo vai dar certo
no seu programa,
pode forçar a compilação/execução
utilizando um bloco “unsafe”.

08.rs

```
20 fn main() {
21
22     // ---------------- let ----------------
23     let x = 10;         // variável local, imutável por padrão
24     let mut y = 20;     // variável local mutável
25
26     println!("let x = {}", x);
27     println!("let mut y = {}", y);
28     y += 1;
29     println!("y após incremento = {}", y);
30
31     // ---------------- const ----------------
32     println!("const PI = {}", PI);
33
34     // ---------------- static ----------------
35     println!("static VALOR_FIXO = {}", VALOR_FIXO);
36
37     // ---------------- static mut (unsafe) ----------------
38     unsafe {
39         CONTADOR += 1;
40         println!("static mut CONTADOR = {}", CONTADOR);
41     }
42
43 }
44
```

```rust

// Constante global
// (substituída em tempo de compilação, não ocupa memória real)

const PI: f64 = 3.1415;


// Static global
// (tem endereço fixo em memória, imutável por padrão)

static VALOR_FIXO: i32 = 100;


// Static mut global
// (mutável, mas acesso exige unsafe)

static mut CONTADOR: i32 = 0;


fn main() {
```

```
wecomp@labwecomp:~$ rustc 08.rs
warning: creating a shared reference to mutable static
  --> 08.rs:40:46
   |
40 |         println!("static mut CONTADOR = {}", CONTADOR);
   |                                              ^^^^^^^^ shared reference to mutable static
   |
   = note: for more information, see <https://doc.rust-lang.org/nightly/edition-guide/rust-2024/static-mut-references.html>
   = note: shared references to mutable statics are dangerous; it's undefined behavior if the static is mutated or if a mutable reference is created for it while the shared reference lives
   = note: `#[warn(static_mut_refs)]` on by default

warning: 1 warning emitted

wecomp@labwecomp:~$ ./08
let x = 10
let mut y = 20
y após incremento = 21
const PI = 3.1415
static VALOR_FIXO = 100
static mut CONTADOR = 1
wecomp@labwecomp:~$
```

Conceito de escopo

Em Rust, **escopo** é a região do código onde uma variável, constante, função ou recurso é válido e pode ser acessado. Ele começa no ponto onde o item é declarado e termina quando o bloco em que foi definido se encerra.

De forma prática:

- Uma variável declarada dentro de um bloco `{ ... }` só pode ser usada dentro desse bloco e deixa de existir quando o bloco termina.
- Esse comportamento está diretamente ligado ao sistema de **ownership** e **lifetime** de Rust: quando o escopo acaba, os valores alocados são liberados automaticamente.

Independentemente de qualquer coisa, chaves ({ ... }) definem um bloco.

shadowing

Shadowing em Rust

- **Shadowing** é o ato de declarar uma nova variável com o mesmo nome de uma já existente.
- A nova variável **sombra** a anterior dentro do mesmo escopo ou em um escopo mais interno.
- A variável original não desaparece: ela apenas fica inacessível enquanto a nova estiver ativa.
- Quando o escopo da variável mais recente termina, a versão anterior volta a ser visível (se ainda existir).
- Esse recurso permite reutilizar nomes de forma organizada, evitando conflitos e mantendo o código claro.

09.rs

```rust

// Criando uma variável global:
static GLOBAL:i16 = 999;

fn main() {

    println!("Valor de GLOBAL em main: {}", GLOBAL);

    // Variável local 'a':
    let a = 111;
    println!("Valor de 'a' em main: {}", a);

    // Variável local 'b':
    let b = 222;
    println!("Valor de 'b' em main, antes de chamar a fn qualquer: {}", b);

    qualquer();

    println!("Valor de 'b' em main, quando voltou da fn qualquer: {}", b);

}

```

09.rs

```
 1
 2 // Criando uma variável global:
 3 static GLOBAL:i16 = 999;
 4
 5 fn main() {
 6
 7     println!("Valor de GLOBAL em main: {}", GLOBAL);
 8
 9     // Variável local 'a':
10     let a = 111;
11     println!("Valor de 'a' em main: {}", a);
12
13     // Variável local 'b':
14     let b = 222;
15     println!("Valor de 'b' em main, antes de chamar a fn qualquer: {}", b);
16
17     qualquer();
18
19     println!("Valor de 'b' em main, quando voltou da fn qualquer: {}", b);
20
21 }
22
```

09.rs

```
fn qualquer() {

    println!("Entrei na funcao qualquer");
    println!("Valor de GLOBAL em qualquer: {}", GLOBAL);

    // Se descomentar a linha abaixo, dá erro de compilação,
    // pois a variável 'a' é local em main:
    // println!("Valor de 'a' em qualquer: {}", a);

    // Criando outra variável local 'b':
    let b = 333;
    println!("Valor de 'b' criado em qualquer: {}", b);

    // Criando um escopo com chaves:

    {
        // Criando outra variável local 'b':
        let b = 444;
        println!("Valor de 'b' no escopo extra em qualquer: {}", b);
    }

    println!("Valor de 'b' depois que saiu do escopo extra: {}", b);
}
```

09.rs

```
fn qualquer() {

    println!("Entrei na funcao qualquer");
    println!("Valor de GLOBAL em qualquer: {}", GLOBAL);

    // Se descomentar a linha abaixo, dá erro de compilação,
    // pois a variável 'a' é local em main:
    // println!("Valor de 'a' em qualquer: {}", a);

    // Criando outra variável local 'b':
    let b = 333;
    println!("Valor de 'b' criado em qualquer: {}", b);

    // Criando um escopo com chaves:

    {
        // Criando outra variável local 'b':
        let b = 444;
        println!("Valor de 'b' no escopo extra em qualquer: {}", b);
    }

    println!("Valor de 'b' depois que saiu do escopo extra: {}", b);
}
```

09.rs

```
fn qualquer() {

    println!("Entrei na funcao qualquer");
    println!("Valor de GLOBAL em qualquer: {}", GLOBAL);

    // Se descomentar a linha abaixo, dá erro de compilação,
    // pois a variável 'a' é local em main:
    // println!("Valor de 'a' em qualquer: {}", a);

    // Criando outra variável local 'b':
    let b = 333;
    println!("Valor de 'b' criado em qualquer: {}", b);

    // Criando um escopo com chaves:

    {
        // Criando outra variável local 'b':
        let b = 444;
        println!("Valor de 'b' no escopo extra em qualquer: {}", b);
    }

    println!("Valor de 'b' depois que saiu do escopo extra: {}", b);
}
```

09.rs

```
fn qualquer() {

    println!("Entrei na funcao qualquer");
    println!("Valor de GLOBAL em qualquer: {}", GLOBAL);

    // Se descomentar a linha abaixo, dá erro de compilação,
    // pois a variável 'a' é local em main:
    // println!("Valor de 'a' em qualquer: {}", a);

    // Criando outra variável local 'b':
    let b = 333;
    println!("Valor de 'b' criado em qualquer: {}", b);

    // Criando um escopo com chaves:

    {
        // Criando outra variável local 'b':
        let b = 444;
        println!("Valor de 'b' no escopo extra em qualquer: {}", b);
    }

    println!("Valor de 'b' depois que saiu do escopo extra: {}", b);
}
```

Primeiro teste:
Vamos descomentar a exibição da variável local ‘a’ dentro da função qualquer:

```
3 fn qualquer() {
4
5     println!("Entrei na funcao qualquer");
6     println!("Valor de GLOBAL em qualquer: {}", GLOBAL);
7
8     // Se descomentar a linha abaixo, dá erro de compilação,
9     // pois a variável 'a' é local em main:
0     println!("Valor de 'a' em qualquer: {}", a);
1
```

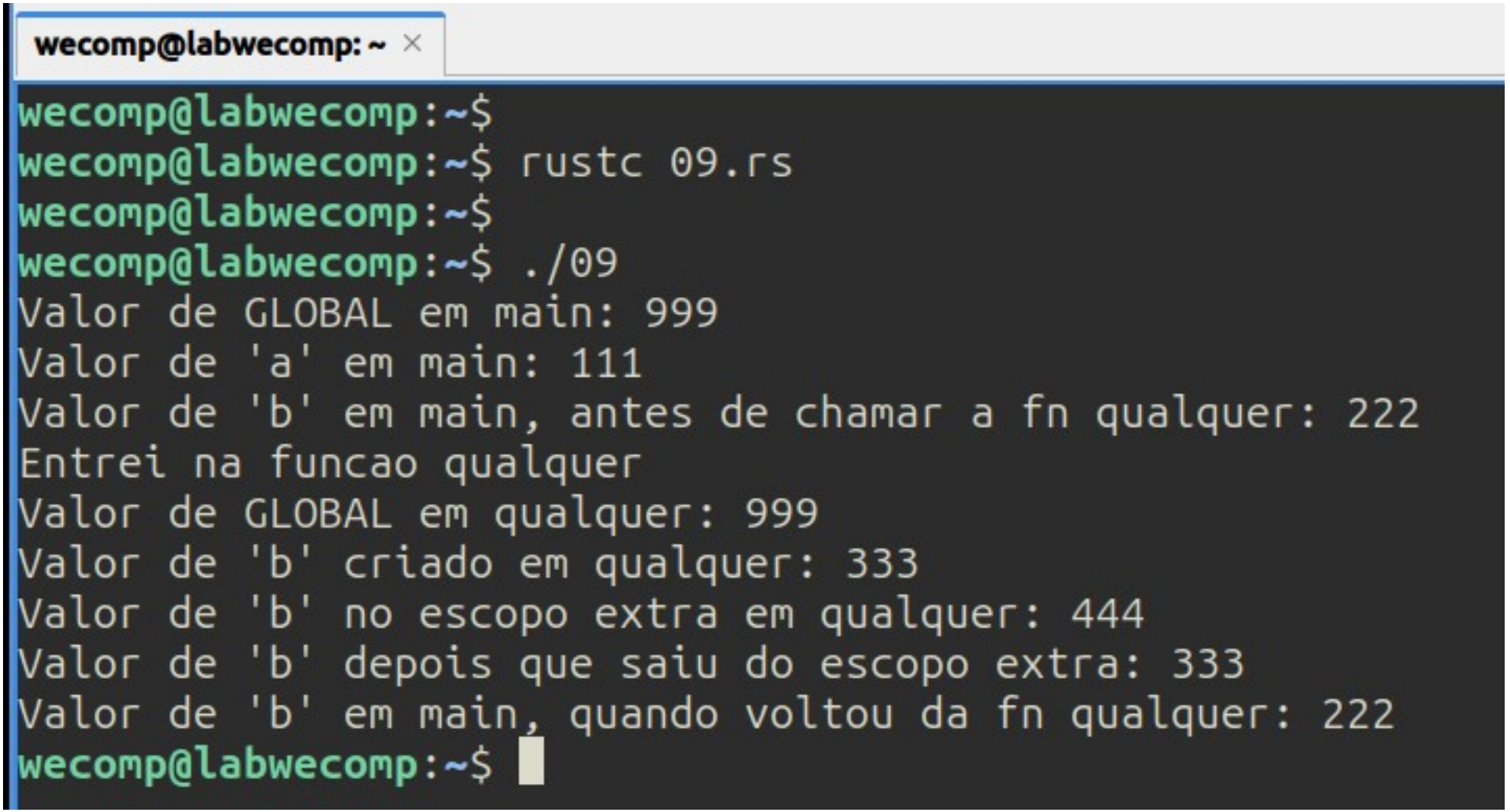
```
3 fn qualquer() {
4
5     println!("Entrei na funcao qualquer");
6     println!("Valor de GLOBAL em qualquer: {}", GLOBAL);
7
8     // Se descomentar a linha abaixo, dá erro de compilação,
9     // pois a variável 'a' é local em main:
0     println!("Valor de 'a' em qualquer: {}", a);
1
```

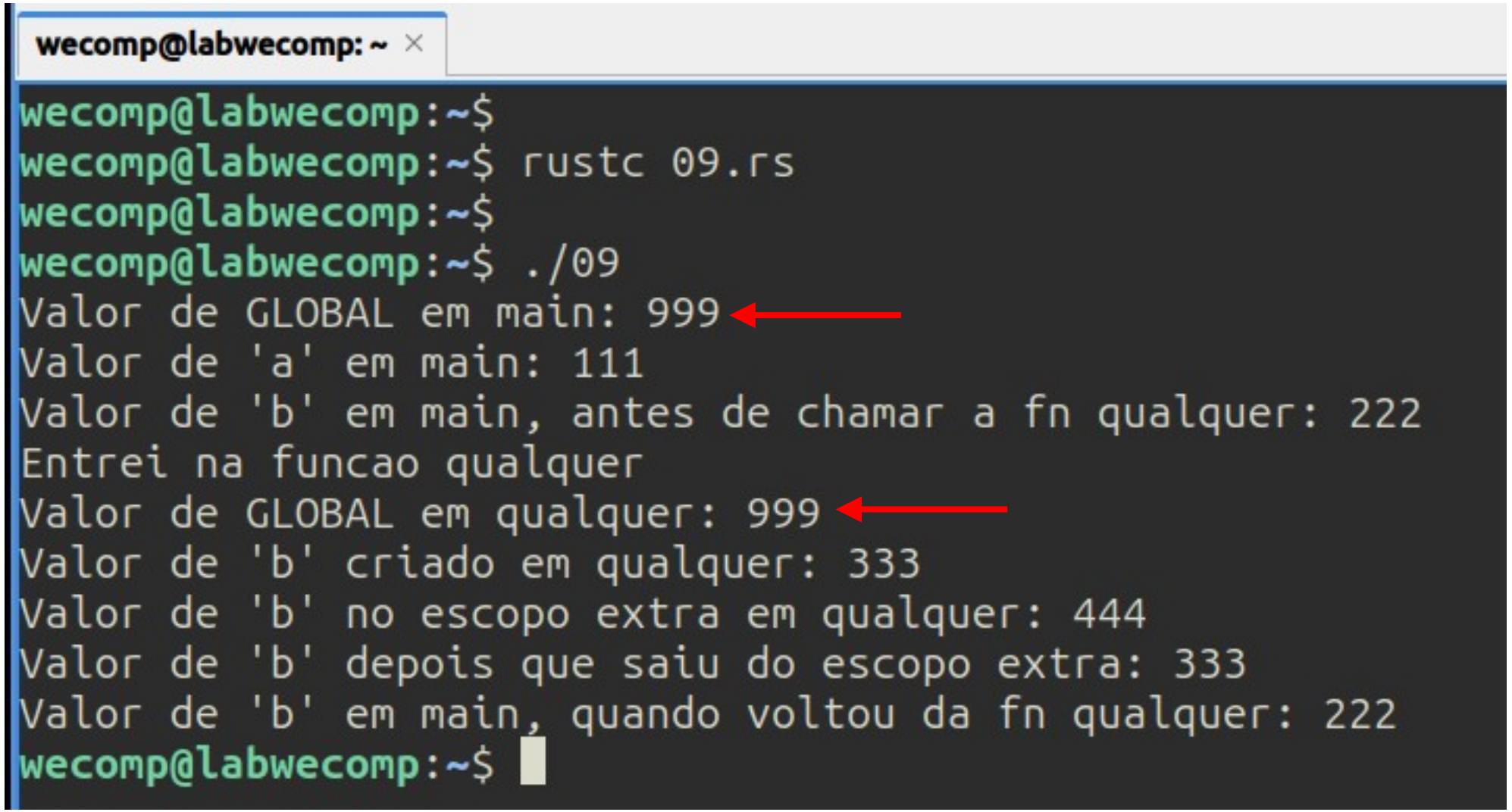
```
wecomp@labwecomp: ~
wecomp@labwecomp:~$ rustc 09.rs
error[E0425]: cannot find value `a` in this scope
  --> 09.rs:30:46
   |
30 |     println!("Valor de 'a' em qualquer: {}", a);
   |                                              ^ not found in this scope

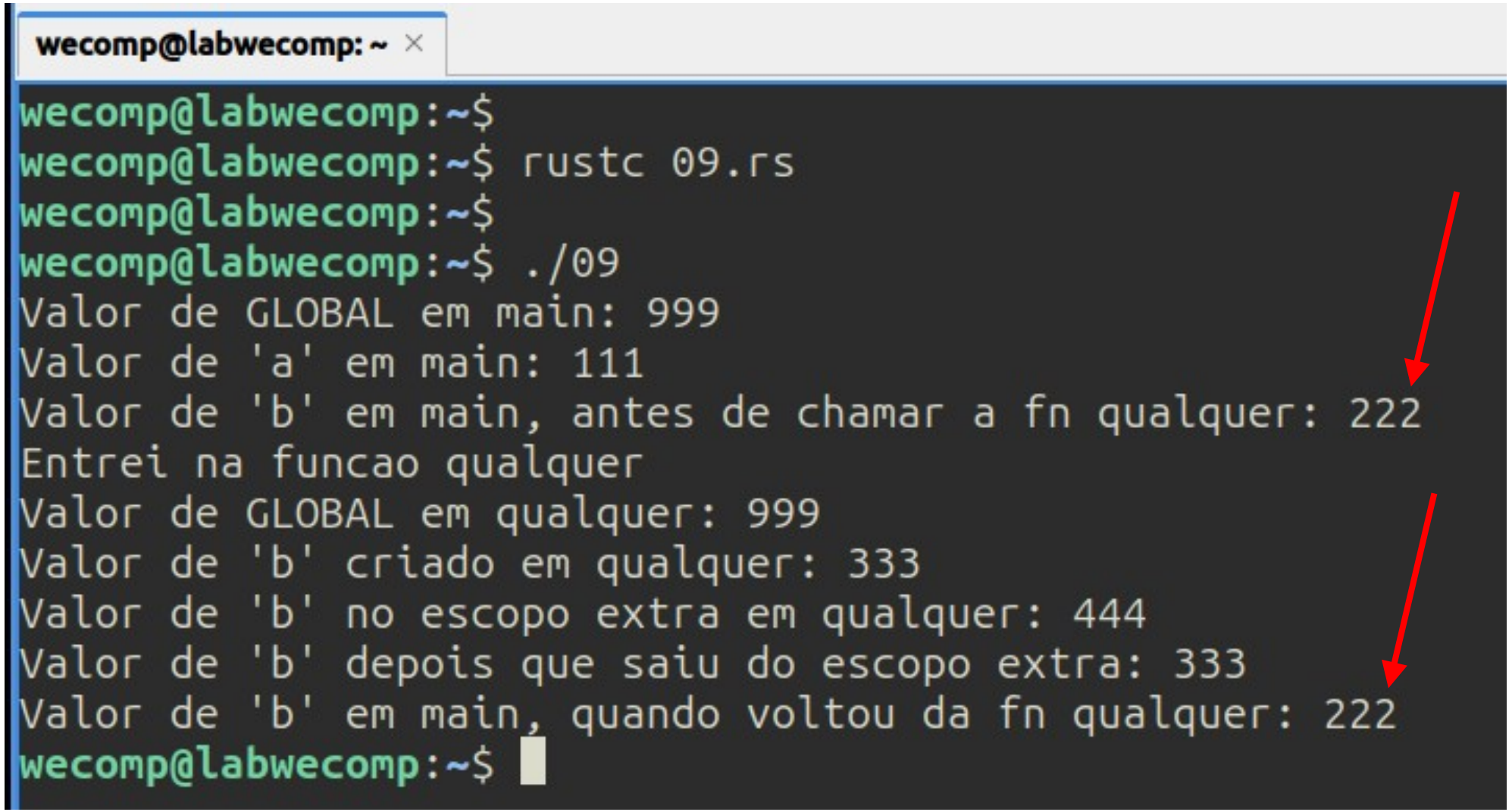
error: aborting due to 1 previous error

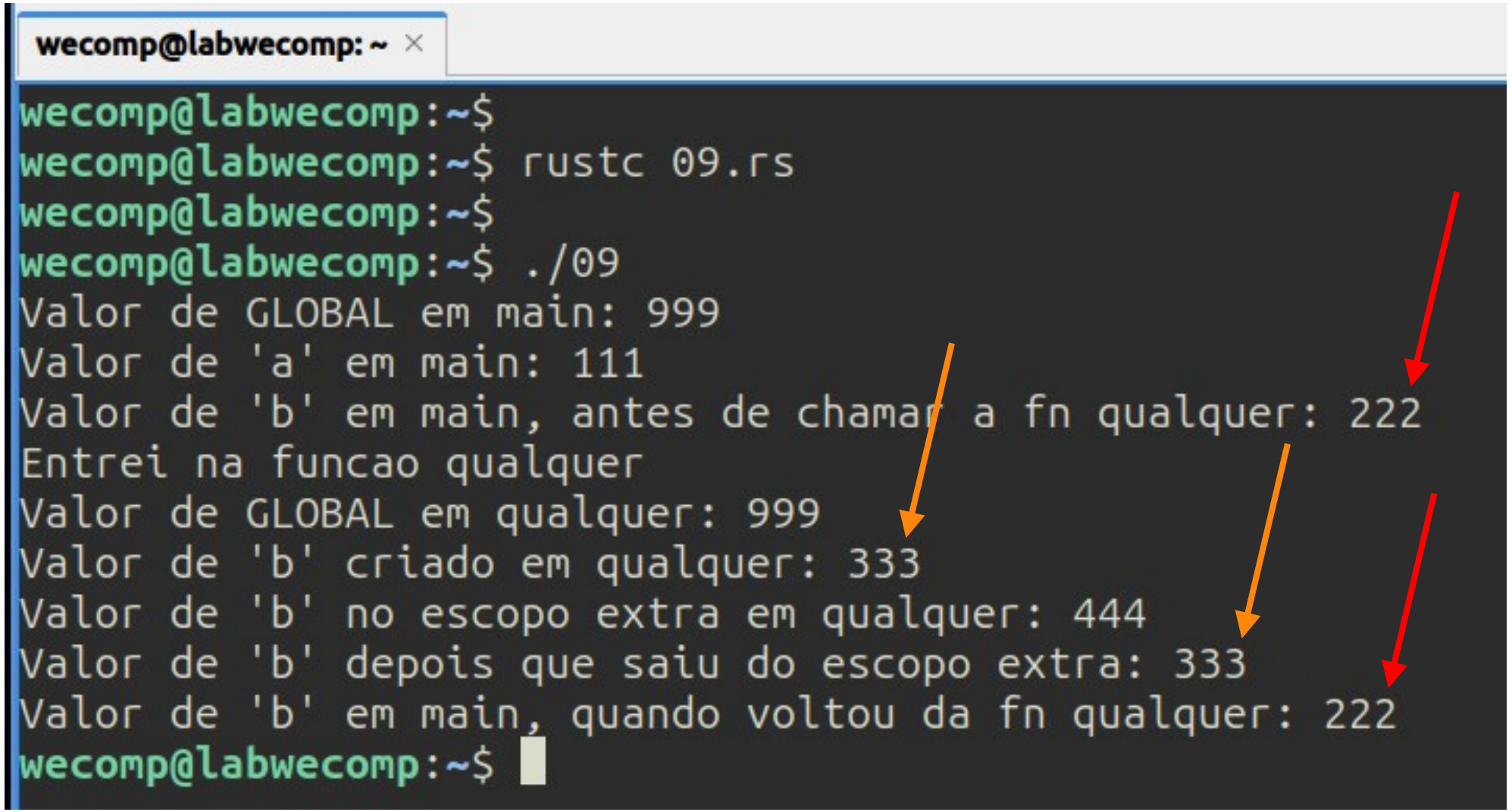
For more information about this error, try `rustc --explain E0425`.
wecomp@labwecomp:~$
```

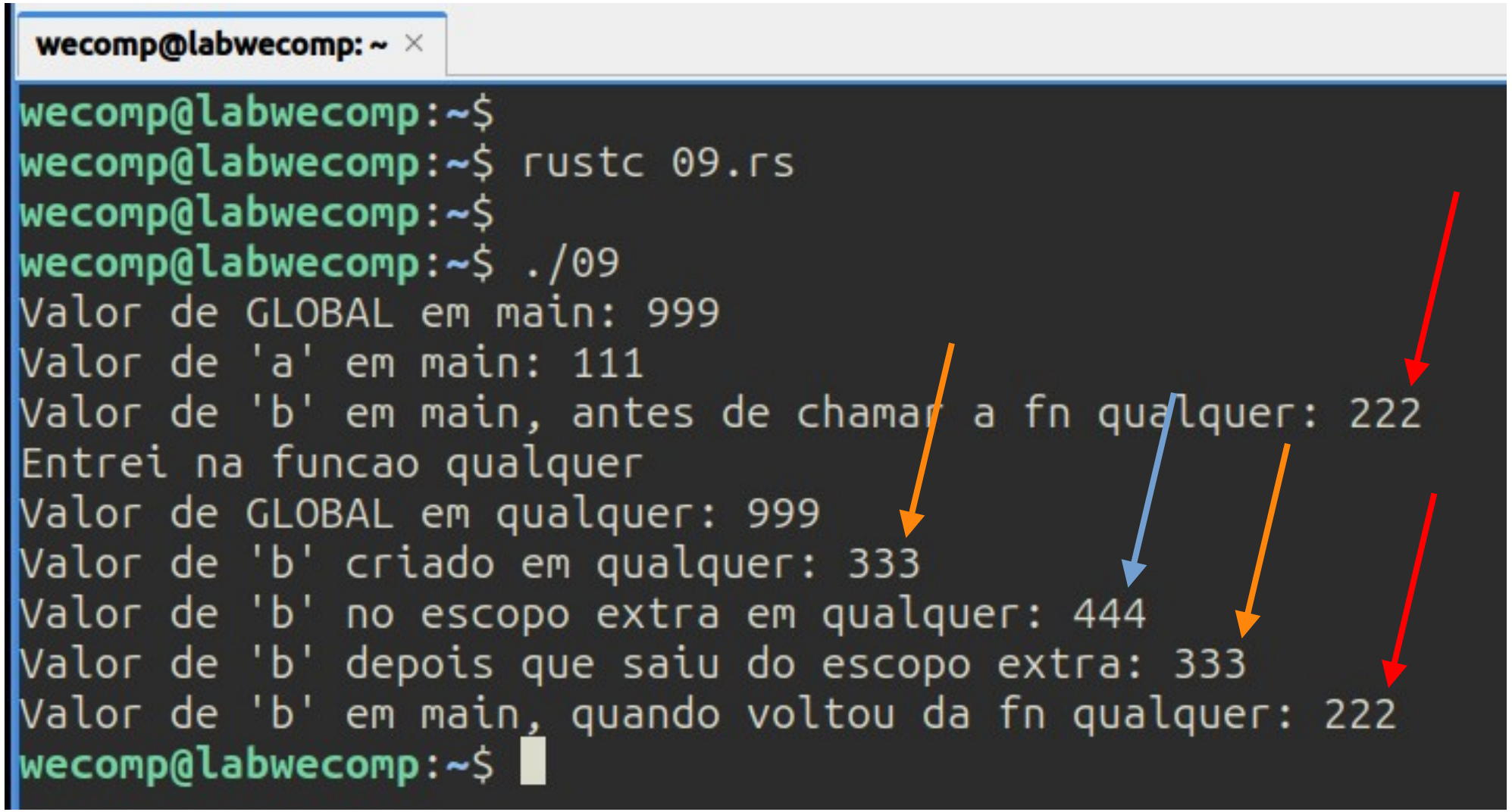
Recomentando a linha...
Execução...


wecomp@labwecomp: ~
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 09.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./09
Valor de GLOBAL em main: 999
Valor de 'a' em main: 111
Valor de 'b' em main, antes de chamar a fn qualquer: 222
Entrei na funcao qualquer
Valor de GLOBAL em qualquer: 999
Valor de 'b' criado em qualquer: 333
Valor de 'b' no escopo extra em qualquer: 444
Valor de 'b' depois que saiu do escopo extra: 333
Valor de 'b' em main, quando voltou da fn qualquer: 222
wecomp@labwecomp:~$


wecomp@labwecomp: ~
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 09.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./09
Valor de GLOBAL em main: 999
Valor de 'a' em main: 111
Valor de 'b' em main, antes de chamar a fn qualquer: 222
Entrei na funcao qualquer
Valor de GLOBAL em qualquer: 999
Valor de 'b' criado em qualquer: 333
Valor de 'b' no escopo extra em qualquer: 444
Valor de 'b' depois que saiu do escopo extra: 333
Valor de 'b' em main, quando voltou da fn qualquer: 222
wecomp@labwecomp:~$


wecomp@labwecomp: ~
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 09.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./09
Valor de GLOBAL em main: 999
Valor de 'a' em main: 111
Valor de 'b' em main, antes de chamar a fn qualquer: 222
Entrei na funcao qualquer
Valor de GLOBAL em qualquer: 999
Valor de 'b' criado em qualquer: 333
Valor de 'b' no escopo extra em qualquer: 444
Valor de 'b' depois que saiu do escopo extra: 333
Valor de 'b' em main, quando voltou da fn qualquer: 222
wecomp@labwecomp:~$


wecomp@labwecomp: ~
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 09.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./09
Valor de GLOBAL em main: 999
Valor de 'a' em main: 111
Valor de 'b' em main, antes de chamar a fn qualquer: 222
Entrei na funcao qualquer
Valor de GLOBAL em qualquer: 999
Valor de 'b' criado em qualquer: 333
Valor de 'b' no escopo extra em qualquer: 444
Valor de 'b' depois que saiu do escopo extra: 333
Valor de 'b' em main, quando voltou da fn qualquer: 222
wecomp@labwecomp:~$


wecomp@labwecomp: ~
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 09.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./09
Valor de GLOBAL em main: 999
Valor de 'a' em main: 111
Valor de 'b' em main, antes de chamar a fn qualquer: 222
Entrei na funcao qualquer
Valor de GLOBAL em qualquer: 999
Valor de 'b' criado em qualquer: 333
Valor de 'b' no escopo extra em qualquer: 444
Valor de 'b' depois que saiu do escopo extra: 333
Valor de 'b' em main, quando voltou da fn qualquer: 222
wecomp@labwecomp:~$

Funções

Mas antes...
tudo em RUST é uma expressão

(e o que o ; tem a ver com isso!!)

Expressão vs. Declaração em Rust

- **Expressão**: algo que *produz um valor*.
  Exemplos:
  - `2 + 3` → resulta em `5`
  - `x * 10`
  - `{ let y = 5; y + 1 }` → resulta em `6`
- **Declaração (statement)**: algo que *não produz valor diretamente*, mas sim faz uma ação (ex.: definir uma variável).
  Exemplo:

```rust
let a = 10; // declaração de variável
```

  Mas observe: `10` é uma **expressão** dentro dessa declaração.

Papel do ponto e vírgula (`;`)

- Quando você **não coloca** `;` no final de uma linha, significa que aquela expressão deve **retornar um valor**.
- Quando você **coloca** `;`, a expressão se transforma em uma declaração que não retorna valor (o tipo vira `()`, chamado de *unit type* em Rust).*

👉 Resumindo:

- **Tudo em Rust é expressão** (até blocos `{ ... }`).
- O `;` **decide** se você quer usar o valor da expressão ou apenas executar e descartar, resultando em `()`.

Expressões e `;` em Rust

- **Tudo é expressão** → produz um valor
- **Blocos `{ ... }` também são expressões**
- O `;` muda o comportamento:

📌 **Sem `;`** → a expressão retorna um valor

📌 **Com `;`** → o valor é descartado, o resultado é `()` (*unit type*)

O **unit type** em Rust é escrito como `()`.

Ele representa:

- **A ausência de um valor significativo** (semelhante ao `void` em C/Java, mas é de fato um tipo).
- **Um único valor possível**, que é o próprio `()` — por isso é chamado *unit* (unidade).

Agora sim... funções!!

10.rs

```
 1
 2 fn soma( a:i32, b:i32 ) -> i32
 3 {
 4     println!( "{} + {} = {}", a, b, a + b );
 5     a + b  // sem o ; é uma expressão, está retornando este valor
 6 }
 7
 8 fn main() {
 9
10     println!( "Soma = {}", soma(3, 7) );
11
12 }
13
```

```rust
fn soma( a:i32, b:i32 ) -> i32
{
    println!( "{} + {} = {}", a, b, a + b );
    a + b  // sem o ; é uma expressão, está retornando este valor
}

fn main() {

    println!( "Soma = {}", soma(3, 7) );

}
```

1. **Definição da função** `soma`

```rust
fn soma(a: i32, b: i32) -> i32
```

- A função recebe dois parâmetros `a` e `b` do tipo `i32`.
- A seta `-> i32` indica que a função deve **retornar um valor inteiro.**

```rust
fn soma( a:i32, b:i32 ) -> i32
{
    println!( "{} + {} = {}", a, b, a + b );
    a + b  // sem o ; é uma expressão, está retornando este valor
}

fn main() {

    println!( "Soma = {}", soma(3, 7) );

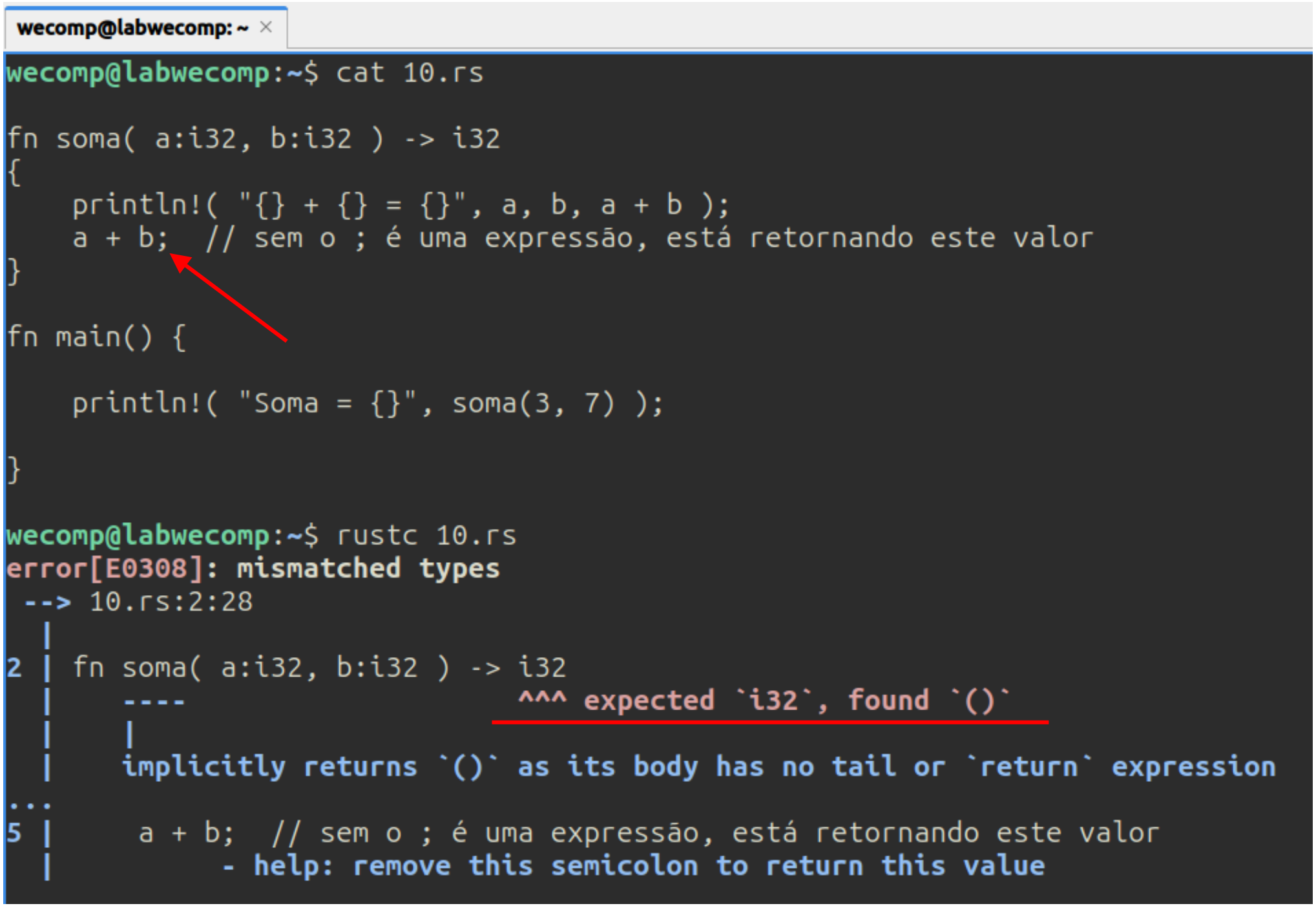
}
```

2. **Corpo da função**

```rust
println!("{} + {} = {}", a, b, a + b);
a + b
```

- Primeiro imprime na tela a soma.
  Exemplo: `3 + 7 = 10`.
- Depois, a última linha `a + b` **é uma expressão sem** `;`, logo o valor calculado é **retornado** pela função.

> Se tivesse `a + b;` (com ponto e vírgula), isso não retornaria nada, e o compilador reclamaria porque a função prometeu devolver um `i32`. **(próximo slide)**


wecomp@labwecomp: ~
wecomp@labwecomp:~$ cat 10.rs
fn soma(a:i32, b:i32) -> i32
{
println!("{} + {} = {}", a, b, a + b);
a + b; // sem o ; é uma expressão, está retornando este valor
}
fn main() {
println!("Soma = {}", soma(3, 7));
}
wecomp@labwecomp:~$ rustc 10.rs
error[E0308]: mismatched types
--> 10.rs:2:28
2 | fn soma(a:i32, b:i32) -> i32
^^^ expected `i32`, found `()`
implicitly returns `()` as its body has no tail or `return` expression
5 | a + b; // sem o ; é uma expressão, está retornando este valor
- help: remove this semicolon to return this value

```rust

fn soma( a:i32, b:i32 ) -> i32
{
    println!( "{} + {} = {}", a, b, a + b );
    a + b  // sem o ; é uma expressão, está retornando este valor
}

fn main() {

    println!( "Soma = {}", soma(3, 7) );

}

```

3. **Função** `main`

```rust
println!("Soma = {}", soma(3, 7));
```

- Chama `soma(3, 7)`.
- A função imprime:

```
3 + 7 = 10
```

- Depois retorna `10`.
- Esse valor é usado no `println!`, resultando em:

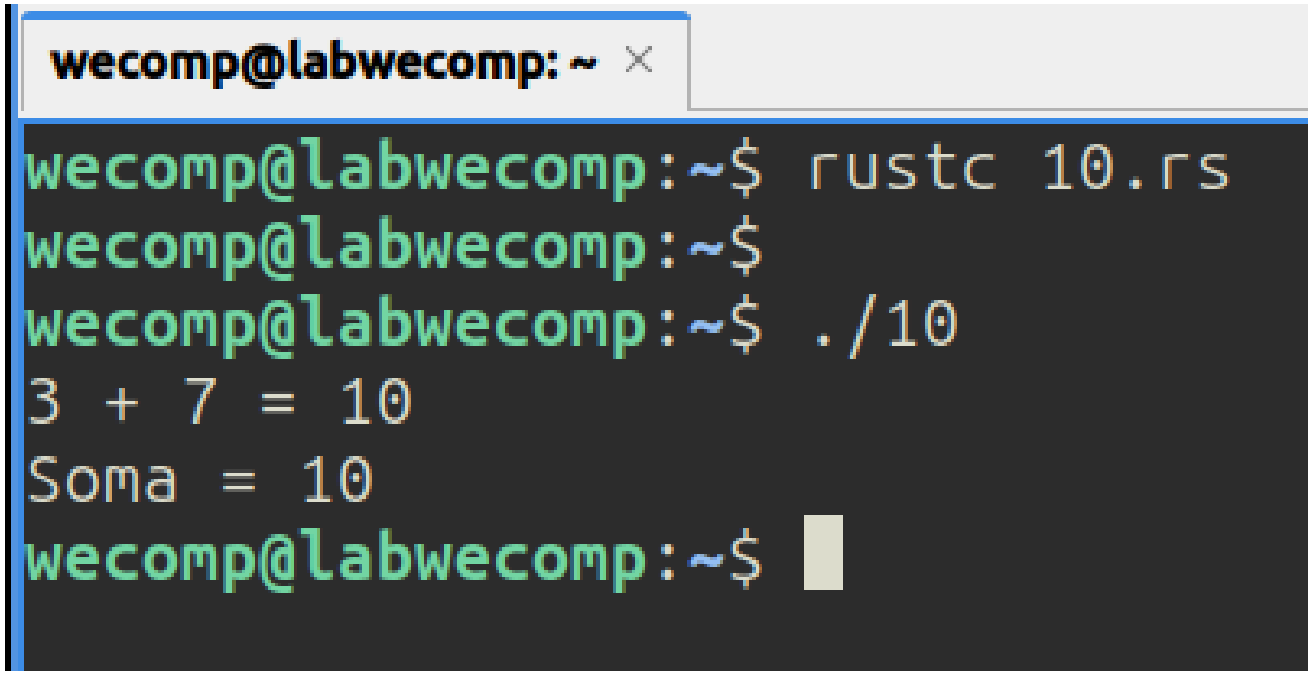
```ini
Soma = 10
```

```rust
fn soma( a:i32, b:i32 ) -> i32
{
    println!( "{} + {} = {}", a, b, a + b );
    a + b  // sem o ; é uma expressão, está retornando este valor
}

fn main() {

    println!( "Soma = {}", soma(3, 7) );

}
```



Condição

11.rs

```
fn main() {

    let mut idade:u8 = 16;

    if idade > 17 {

        println!("Pode dirigir");

    } else {

        println!("Não pode dirigir");

    }

    idade = 20;

    if idade > 17 {

        println!("Pode dirigir");

    } else {

        println!("Não pode dirigir");

    }

}
```

11.rs

```
fn main() {

    let mut idade:u8 = 16;

    if idade > 17 {

        println!("Pode

    } else {

        println!("Não

    }

    idade = 20;

    if idade > 17 {

        println!("Pode dirigir");

    } else {

        println!("Não pode dirigir");

    }

}
```

```
wecomp@labwecomp: ~
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 11.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./11
Não pode dirigir
Pode dirigir
wecomp@labwecomp:~$
```

12.rs

```rust
fn main() {

    // Adultos (18 ou mais anos) podem sair do IFSP.
    // Jovens (15 até 17 anos) só podem sair se autorizados.

    let idade:u8 = 15;
    let saida_autorizada = true;

    // OU lógico: ||
    // E  lógico: &&

    if idade > 17 || idade > 14 && saida_autorizada {

        println!("Pode sair do IFSP");

    } else {

        println!("Não pode sair do IFSP");

    }

}
```

12.rs

```rust
fn main() {

    // Adultos (18 ou mais anos) podem sair do IFSP.
    // Jovens (15 até 17 anos) só podem sair se autorizados.

    let idade:u8 = 15;
    let saida_autorizada = true;

    // OU lógico: ||
    // E  lógico: &&

    if idade > 17 || idade > 14 && saida_autorizada {

        println!("Pode sair do IFSP");

    } else {

        println!("Não

    }

}
```

```
wecomp@labwecomp: ~
wecomp@labwecomp:~$
wecomp@labwecomp:~$ rustc 12.rs
wecomp@labwecomp:~$
wecomp@labwecomp:~$ ./12
Pode sair do IFSP
wecomp@labwecomp:~$
wecomp@labwecomp:~$
```

Lembrando que em RUST tudo pode ser uma expressão (retorna um valor), basta não usar o ;

13.rs

```
1
2 fn main() {
3
4     let mut idade = 16;
5
6     let mut maioridade = if idade > 17 { "maior" } else { "menor" };
7
8     println!("A pessoa é {} de idade", maioridade);
9
0     idade = 20;
1     maioridade = if idade > 17 { "maior" } else { "menor" };
2     println!("A pessoa é {} de idade", maioridade);
3
4 }
5
```

```rust
fn main() {

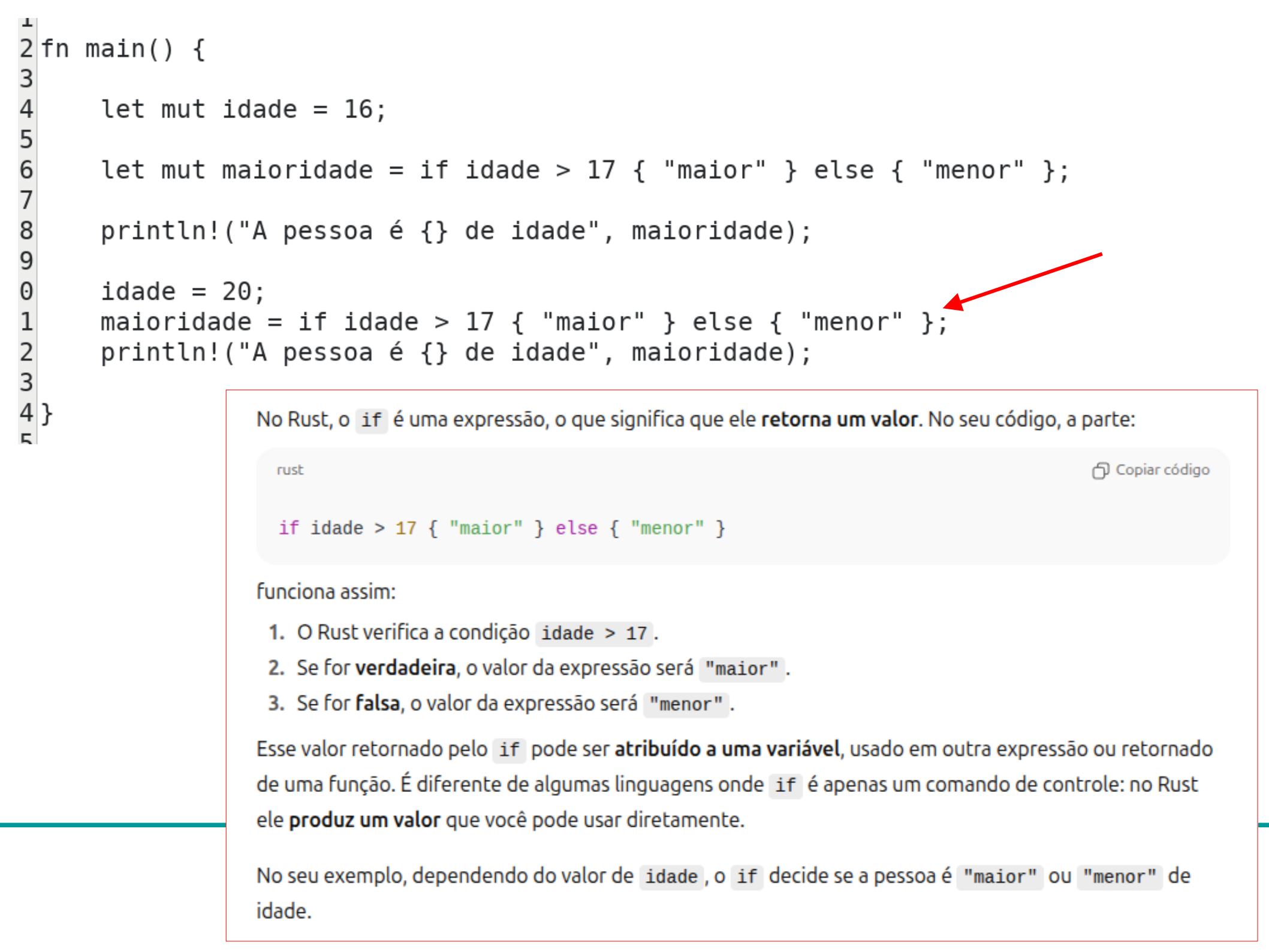
    let mut idade = 16;

    let mut maioridade = if idade > 17 { "maior" } else { "menor" };

    println!("A pessoa é {} de idade", maioridade);

    idade = 20;
    maioridade = if idade > 17 { "maior" } else { "menor" };
    println!("A pessoa é {} de idade", maioridade);

}
```



No Rust, o `if` é uma expressão, o que significa que ele **retorna um valor**. No seu código, a parte:

rust — Copiar código

```rust
if idade > 17 { "maior" } else { "menor" }
```

funciona assim:

1. O Rust verifica a condição `idade > 17`.
2. Se for **verdadeira**, o valor da expressão será `"maior"`.
3. Se for **falsa**, o valor da expressão será `"menor"`.

Esse valor retornado pelo `if` pode ser **atribuído a uma variável**, usado em outra expressão ou retornado de uma função. É diferente de algumas linguagens onde `if` é apenas um comando de controle: no Rust ele **produz um valor** que você pode usar diretamente.

No seu exemplo, dependendo do valor de `idade`, o `if` decide se a pessoa é `"maior"` ou `"menor"` de idade.

```rust
fn main() {

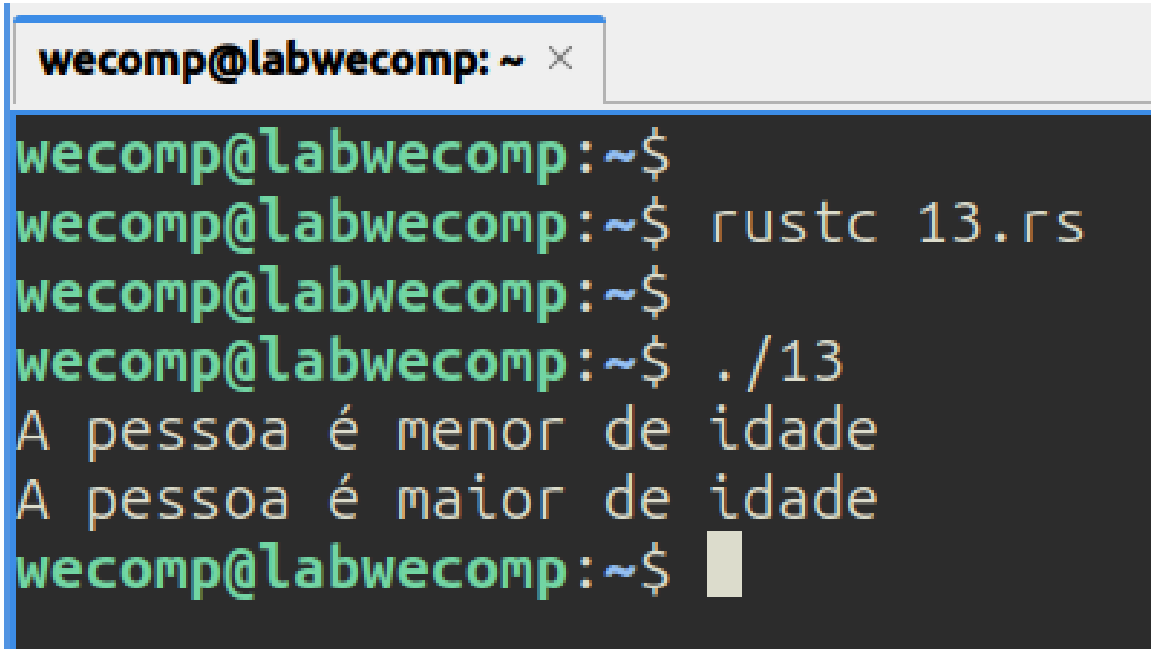
    let mut idade = 16;

    let mut maioridade = if idade > 17 { "maior" } else { "menor" };

    println!("A pessoa é {} de idade", maioridade);

    idade = 20;
    maioridade = if idade > 17 { "maior" } else { "menor" };
    println!("A pessoa é {} de idade", maioridade);

}
```




To be continued...

Como falei no início da aula,
após o intervalo estarei na 105.
Podemos instalar a máquina virtual lá
para experimentação.


obrigado
pela atenção!
RENAN PAVINI